CARFIND

INSTITUCIÓN: Escuela Tecnica N° 32 DE14 "Gral Jose de San Martin"

MATERIA/TALLER: Prácticas Profesionalizantes

DOCENTES: Gonzalo Nicolás Consorti, Damian Olaso y Luciano Alderete

ALUMNO: Ignacio Agustin Córdoba

CURSO: 6°1° Turno Mañana Computación

CICLO LECTIVO: 2026

TÍTULO DEL PROYECTO: CarFind

El presente trabajo desarrolla el proyecto CarFind, una propuesta de sistema integral orientada a la compra y venta de vehículos mediante una plataforma digital organizada, segura y escalable. El objetivo principal del informe es transformar los distintos elementos del proyecto —como el prototipo de interfaces realizado en Figma, el diseño de la base de datos relacional en SQL y el análisis funcional del sistema— en una memoria descriptiva estructurada bajo criterios académicos.

A lo largo del documento se explican los principales módulos que componen la plataforma, incluyendo la gestión de usuarios, publicaciones, carrito de compras, mensajería, métodos de pago, perfiles, reportes y configuración. Cada uno de estos componentes es analizado tanto desde la experiencia de usuario como desde su relación con el modelo de datos, permitiendo comprender cómo las decisiones de diseño impactan directamente en el funcionamiento del sistema.

Además, se aborda el estudio de herramientas y tecnologías utilizadas en el desarrollo, tales como frameworks, gestores de bases de datos, ORMs y plataformas de administración. Esto permite contextualizar el proyecto dentro de un entorno tecnológico real y en constante evolución.

Por último, el trabajo también establece una relación entre el proyecto y la formación técnica en el área de bases de datos, destacando la importancia de conceptos como normalización, seguridad, transacciones, procedimientos almacenados y trazabilidad de la información.

Palabras clave: base de datos, compra-venta, vehículos, SQL, Figma, prototipado, marketplace, sistemas.

Presentación del proyecto

¿Qué problema busco resolver?

Cuando comenzamos a desarrollar este proyecto en equipo, identificamos que el problema principal no es simplemente la compra y venta de vehículos, sino la forma en la que estas operaciones se realizan actualmente en plataformas digitales. Muchas de estas plataformas son generalistas, lo que significa que los vehículos aparecen mezclados con otros productos, generando desorganización y dificultando la búsqueda de información relevante.

Además, detectamos una falta importante de confianza entre compradores y vendedores. En este tipo de transacciones, es fundamental contar con información clara y completa, como el estado del vehículo, su documentación, kilometraje o antecedentes. Sin embargo, en muchos casos estos datos no están bien estructurados o directamente no se encuentran disponibles, lo que aumenta el riesgo de fraude o malas decisiones.

Por otro lado, los vendedores también enfrentan dificultades, ya que muchas veces no logran destacar sus publicaciones ni gestionar correctamente la comunicación con posibles compradores. Esto genera procesos poco eficientes y una experiencia negativa para ambas partes.

¿Por qué es importante resolverlo?

Resolver este problema es importante porque impacta directamente en la seguridad, el tiempo y la experiencia de los usuarios. La compra o venta de un vehículo no es una decisión menor, ya que implica un valor económico alto y requiere confianza entre las partes.

Una plataforma desorganizada o poco confiable puede generar pérdidas económicas, decisiones incorrectas o incluso situaciones de fraude. Además, la falta de claridad y estructura provoca que los usuarios pierdan tiempo buscando información o realizando consultas innecesarias.

Por eso, mejorar este tipo de sistemas permite no solo optimizar el proceso de compra y venta, sino también generar un entorno más seguro, transparente y eficiente para todos los usuarios.

¿Quién sería el usuario o beneficiario?

Los principales beneficiarios del proyecto son las personas que desean comprar o vender vehículos a través de medios digitales. Esto incluye tanto a compradores particulares como a vendedores individuales o concesionarias.

También se benefician aquellos usuarios que buscan una experiencia más organizada, con información clara y herramientas que faciliten la comunicación, la comparación de opciones y la toma de decisiones. En general, cualquier persona involucrada en el proceso de compra-venta de vehículos puede aprovechar las ventajas de una plataforma más especializada.

¿Cuál es el objetivo general?

El objetivo general del proyecto es diseñar una plataforma digital especializada en la compra y venta de vehículos, que permita mejorar la organización de la información, la comunicación entre usuarios y la seguridad de las operaciones, mediante el uso de un prototipo de interfaz y un modelo de base de datos estructurado.

¿Cuáles son los objetivos específicos?

- Analizar las principales problemáticas de las plataformas actuales de compra y venta.
- Diseñar un sistema que permita organizar mejor la información de los vehículos.
- Desarrollar un prototipo de interfaz que facilite la navegación del usuario.
- Elaborar un diagrama de casos de uso que represente las interacciones entre los usuarios y el sistema.
- Colaborar en la planificación del proyecto mediante herramientas como el diagrama de Gantt.
- Relacionar las vistas del sistema con las funcionalidades principales.

¿Cuál es el alcance del proyecto?

El proyecto incluye el diseño de una plataforma conceptual que abarca funcionalidades como registro de usuarios, publicaciones de vehículos, visualización de información, mensajería, carrito y métodos de pago.

Sin embargo, no incluye una implementación completa en producción, sino que se centra en el diseño del sistema, el prototipado de interfaces y la estructura de la base de datos como modelo teórico.

Objetivos

Objetivo general:

El objetivo general de este proyecto es diseñar y documentar una plataforma digital integral orientada a la compra y venta de vehículos, mediante la articulación de un prototipo de interfaz, un modelo de base de datos relacional y la implementación de lógica SQL. Se busca desarrollar una solución que permita mejorar la organización, la seguridad y la eficiencia de las operaciones dentro del entorno automotor digital, incorporando además criterios de usabilidad, validación de datos y proyección de mejoras futuras.

Objetivos específicos:

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

- Analizar el funcionamiento de las plataformas actuales de compra y venta de vehículos, identificando sus principales limitaciones y oportunidades de mejora.
- Diseñar un prototipo de interfaz que represente las principales funcionalidades del sistema, priorizando la experiencia del usuario y la claridad en la navegación.
- Modelar una base de datos relacional que permita almacenar de forma organizada la información del sistema, asegurando consistencia, integridad y escalabilidad.
- Relacionar las vistas del frontend con las entidades del backend, comprendiendo cómo cada acción del usuario impacta en la estructura de datos.
- Implementar y documentar la lógica SQL, incluyendo claves primarias, claves foráneas, restricciones, índices, vistas, triggers, procedimientos almacenados y transacciones.
- Investigar frameworks y herramientas que puedan utilizarse en el desarrollo y administración del sistema, evaluando sus ventajas y aplicaciones.
- Incorporar criterios de seguridad, control y validación de datos para reducir riesgos dentro de la plataforma.
- Vincular el desarrollo del proyecto con los conocimientos adquiridos en el área de bases de datos, análisis de sistemas y diseño de software.

¿Cuál es el alcance del proyecto?

Cuando definimos el alcance del proyecto, lo primero que tuvimos en cuenta como grupo fue establecer hasta dónde queríamos llegar con CarFind en esta etapa. No se trataba de hacer una aplicación completamente terminada, sino de desarrollar una base sólida que represente cómo funcionaría el sistema en un entorno real.

En este sentido, el proyecto incluye el diseño de una plataforma digital orientada a la compra y venta de vehículos, donde se contemplan distintas funcionalidades principales. Entre ellas se encuentran el registro e inicio de sesión de usuarios, la gestión de perfiles, la navegación de publicaciones y la visualización detallada de cada vehículo con información relevante como marca, modelo, año, precio y estado.

También se incluye la posibilidad de que los vendedores puedan crear y gestionar sus propias publicaciones, utilizando formularios que permiten organizar mejor los datos. A su vez, el sistema contempla herramientas como el buscador con filtros, la mensajería entre comprador y vendedor, el carrito de compra y la selección de métodos de pago a nivel conceptual.

Otro aspecto importante dentro del alcance es la incorporación de elementos que aportan seguridad y confianza al sistema, como la reputación de usuarios, verificaciones, reportes y detección de actividad sospechosa. Además, se considera un rol de administrador encargado de moderar el contenido y gestionar el funcionamiento general de la plataforma.

Sin embargo, es importante aclarar que el proyecto no incluye una implementación completa lista para producción. Es decir, muchas de estas funcionalidades no están desarrolladas de forma operativa, sino que se encuentran diseñadas a nivel conceptual, mediante prototipos y estructuras de base de datos.

Por ejemplo, no se realiza una integración real con sistemas de pago ni con servicios externos para validar información de vehículos. Tampoco se desarrollan funcionalidades avanzadas como analítica de datos compleja o automatización de procesos en tiempo real.

En resumen, el alcance del proyecto busca un equilibrio entre una propuesta funcional bastante completa y una implementación realista acorde a los conocimientos y recursos disponibles. El objetivo principal es definir cómo debería funcionar el sistema, más que desarrollarlo completamente.

Investigación Inicial (Antes de comenzar)

Antes de comenzar con el desarrollo del proyecto, realizamos una investigación inicial para entender mejor el problema y analizar posibles soluciones. En esta etapa, nos enfocamos principalmente en observar cómo funcionan actualmente las plataformas de compra y venta de vehículos, y qué aspectos podíamos mejorar en nuestro sistema.

Para esto, utilizamos principalmente herramientas como Google, donde investigamos diferentes páginas y plataformas existentes. Una de las principales referencias fue Mercado Libre, ya que es una de las plataformas más utilizadas, aunque no está especializada exclusivamente en vehículos. A partir de su análisis, pudimos identificar tanto aspectos positivos, como la facilidad de uso y la variedad de publicaciones, como también ciertas limitaciones, especialmente en la organización de la información y la falta de enfoque en el rubro automotor.

Además, también analizamos otras páginas web dedicadas específicamente a la venta de autos, lo que nos permitió comparar distintas formas de presentar la información y entender qué elementos son realmente importantes para los usuarios al momento de buscar o publicar un vehículo.

En cuanto a las herramientas de desarrollo, se decidió utilizar Figma para la creación del prototipo de interfaces. Esta elección se basó principalmente en que es una herramienta accesible,

fácil de usar y que permite trabajar de forma colaborativa. Gracias a Figma, pudimos diseñar las distintas vistas del sistema, organizar la navegación y representar de manera visual cómo sería la interacción del usuario con la plataforma.

Durante esta etapa no solo se buscó recopilar información, sino también tomar decisiones sobre cómo encarar el proyecto. En base a lo investigado, se definió que la mejor solución era desarrollar una plataforma más específica y organizada, enfocada únicamente en la compra y venta de vehículos, mejorando la estructura de la información y la experiencia del usuario.

Planificación

Para llevar adelante el desarrollo del proyecto, fue necesario organizar el trabajo de manera clara y dividir las tareas entre los integrantes del grupo. Al tratarse de un proyecto amplio, la planificación fue fundamental para poder avanzar de forma ordenada y cumplir con los objetivos propuestos.

En primer lugar, definimos distintas etapas de trabajo, que incluyeron la investigación inicial, el diseño del prototipo en Figma, la elaboración de diagramas, la organización de la base de datos y la documentación del proyecto. Cada una de estas etapas permitió avanzar progresivamente, evitando desorden o superposición de tareas.

Como grupo, decidimos organizarnos mediante una distribución de roles. Éramos cinco integrantes, y se asignó el rol de Scrum Master a Ludmila, quien se encargó de coordinar el trabajo, organizar las tareas y asegurar que el equipo mantuviera un buen ritmo de avance. Esta forma de trabajo nos ayudó a tener una mejor comunicación y a resolver problemas de manera más rápida.

En cuanto a las herramientas utilizadas, trabajamos principalmente con Figma para el diseño de las interfaces, mientras que para la organización general del proyecto utilizamos esquemas, documentos compartidos y planificación por etapas. También se utilizó Google como fuente de investigación y apoyo durante todo el proceso.

A nivel personal, cada integrante se enfocó en diferentes partes del proyecto. En mi caso, participé en la elaboración del diagrama de casos de uso, que permite representar las interacciones entre los usuarios y el sistema. Además, colaboré en el diseño de algunas vistas del prototipo y en la planificación general del proyecto.

En cuanto al tiempo, si bien no se manejó un cronograma extremadamente rígido, se intentó seguir una secuencia lógica de trabajo, avanzando primero con la investigación, luego con el diseño y finalmente con la documentación. Esto permitió mantener una estructura clara durante todo el desarrollo.

Documentación de Procesos (Desarrollo Paso a Paso)

En esta sección se describe el proceso de desarrollo del proyecto, organizado por etapas. Cada una representa una parte del trabajo realizado y permite entender cómo se fue construyendo la solución desde el inicio hasta su diseño actual.

Etapa 1 – Investigación y análisis del problema

¿Qué debía lograr en esta etapa?

El objetivo principal fue comprender el problema y analizar cómo funcionan actualmente las plataformas de compra y venta de vehículos.

¿Qué procedimiento realicé?

En esta etapa se realizó una investigación utilizando principalmente Google, donde se analizaron diferentes páginas web. Se observaron tanto plataformas generalistas como Mercado Libre, como también sitios específicos de venta de autos.

Se revisaron aspectos como la organización de las publicaciones, la información disponible, la forma de navegación y la interacción entre usuarios.

¿Qué herramientas utilicé?

- Google
- Páginas web de compra-venta
- Mercado Libre como referencia principal

¿Qué decisiones técnicas tomé y por qué?

A partir del análisis, se decidió que el sistema debía ser más específico para el rubro automotor, con mejor organización de la información y funcionalidades más claras para el usuario.

<u>Etapa 2 – Diseño del prototipo en Figma</u>

¿Qué resultado obtuve?

Se logró identificar las principales problemáticas del sistema actual, lo que permitió definir una base clara para el desarrollo del proyecto.

¿Qué debía lograr en esta etapa?

Diseñar visualmente cómo iba a ser la plataforma y definir las distintas vistas del sistema.

¿Qué procedimiento realicé?

Se comenzó a diseñar el prototipo utilizando Figma, creando distintas pantallas como el login, registro, vista principal, publicaciones, carrito, mensajería y otras funcionalidades.

También se trabajó en la organización de la navegación para que sea clara y fácil de usar.

¿Qué herramientas utilicé?

- Figma

¿Qué decisiones técnicas tomé y por qué?

Se eligió Figma por ser una herramienta accesible, fácil de usar y que permite trabajar en equipo. Además, se decidió basarse en interfaces conocidas como Mercado Libre para facilitar la experiencia del usuario.

¿Qué resultado obtuve?

Se logró un prototipo completo que representa el funcionamiento general del sistema y permite visualizar cómo interactúan los usuarios con la plataforma.

¿Qué fuentes consulte?

Para fundamentar el desarrollo del proyecto, se consultaron diferentes tipos de fuentes.

En primer lugar, se usaron libros de ingeniería de software como los de Pressman y Sommerville, los cuales ayudaron a comprender el análisis de requisitos y el diseño del sistema. También se investigó sobre patrones de diseño y arquitectura a partir de autores como Gamma y Martin Fowler, y sobre arquitecturas web gracias a los aportes de Roy Fielding.

En cuanto a la implementación técnica, se consulto la documentación oficial de herramientas como React, Spring Boot, MySQL y PostgreSQL, además del sitio MDN para tecnologías web.

Por otro lado, se profundizo metodologías ágiles como Scrum, utilizando la guía oficial para entender la organización del trabajo en equipo.

Finalmente, una fuente clave fue la entrevista realizada a un cliente del rubro automotor, que me permitió identificar el problema principal. También incorporé conocimientos adquiridos en una capacitación sobre bases de datos.

Resumen de fuentes:

-
- Beck, K. (2000). Extreme Programming Explained: Embrace Change. Addison-Wesley.
- Fielding, R. (2000). Architectural Styles and the Design of Network-based Software Architectures. University of California.
- Gamma, E., Helm, R., Johnson, R., & Johnson, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Oracle Corporation. (2023). MySQL Reference Manual. https://dev.mysql.com/doc/
- PostgreSQL Global Development Group. (2023). PostgreSQL Documentation. https://www.postgresql.org/docs/
- Pressman, R. (2010). Ingeniería del Software. McGraw-Hill.
- React Documentation Team. (2024). React Official Documentation. https://react.dev/
- Sommerville, I. (2011). Ingeniería de Software. Pearson Educación.
- Spring Team. (2024). Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/docs/current/reference/html/
- Mozilla Developer Network. (2024). HTML, CSS y JavaScript Documentation. https://developer.mozilla.org/
- Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. Scrum.org. https://scrumguides.org/
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

¿Qué procedimiento realice paso a paso?

Primero, se realizó la categorización de requisitos, separándolos en requerimientos funcionales (como el registro de usuarios o la carga de imágenes) y no funcionales (como la disponibilidad 24/7 o el diseño responsive).

Después, se elaboró un análisis FODA, que permitió identificar tanto las fortalezas del equipo como las amenazas externas, especialmente frente a plataformas ya posicionadas. A partir de esto, se decidió enfocar el proyecto específicamente en el nicho automotor.

Luego, se definieron las reglas de negocio, estableciendo criterios como la aprobación obligatoria de publicaciones por parte de un administrador para garantizar la calidad de la información.

Por último, se configuró el tablero en JIRA, cargando el Product Backlog y transformando los requerimientos en tareas técnicas (issues), asignadas según el rol de cada integrante del equipo.

Evidencias

FODA

Herramienta de análisis estratégico que permite identificar las fortalezas, oportunidades, debilidades y amenazas de un proyecto, facilitando la toma de decisiones y la planificación.

Fortalezas:

- Disponibilidad de tecnologías modernas para el desarrollo web.
- Existencia de una necesidad real en el mercado detectada mediante entrevista.
- Posibilidad de centralizar y estructurar la información de los vehículos.

Debilidades:

- Falta de experiencia previa del cliente con sistemas especializados.
- Dependencia de la calidad de los datos ingresados por los usuarios.
- Complejidad técnica del desarrollo inicial.

Oportunidades:

- Crecimiento del comercio digital en el sector automotor.
- Posibilidad de escalar el sistema con nuevas funcionalidades (pagos, promociones, reputación).
- Diferenciación frente a plataformas generalistas.

Amenazas:

- Alta competencia de plataformas ya posicionadas en el mercado.

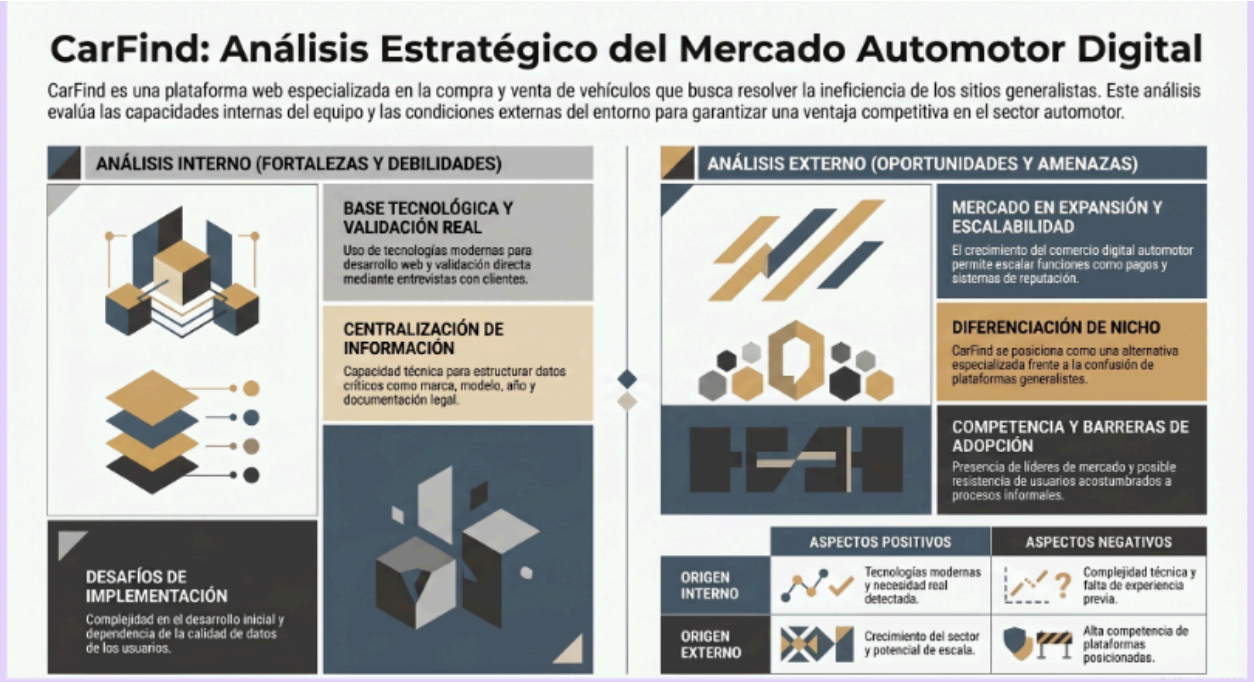
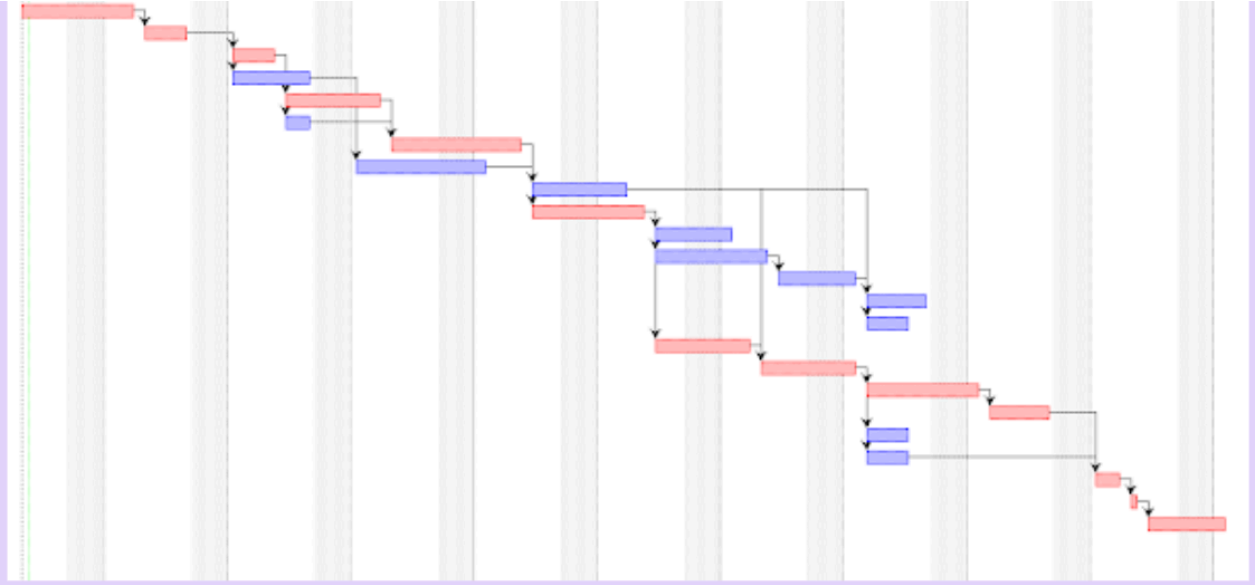


Diagrama gantt

Representación gráfica que muestra la planificación temporal del proyecto, incluyendo tareas, duración, secuencia y dependencias, permitiendo organizar y controlar su ejecución.



ID	Tarea	Duración (Días)	Recursos Humanos	Recursos Físicos	Costo (Pesos)	Dependencia
T1	Relevamiento de requerimientos	5	Analista + Scrum Master	PC, internet	$770.000	-
T2	Análisis del sistema	3	Analista	PC	$420.000	T1
T3	Definición de arquitectura	3	Desarrollador	PC	$462.000	T2

T4	Diseño UI/UX	5	Desarrollador	PC	$630.000	T2
T5	Modelado de base de datos	4	DBA	PC	$616.000	T3
T6	Configuración del entorno	2	Desarrollador	PC	$224.000	T3
T7	Desarrollo backend base	6	Desarrollador	PC	$840.000	T5, T6
T8	Desarrollo frontend base	6	Desarrollador	PC	$840.000	T4, T6
T9	Registro y login	4	Desarrollador	PC	$560.000	T7, T8
T10	Publicación de vehículos	5	Desarrollador	PC	$700.000	T7, T8
T11	Carga de imágenes	3	Desarrollador	PC	$420.000	T10
T12	Búsqueda y filtros	5	Desarrollador	PC	$700.000	T10

T13	Visualización de vehículos	3	Desarrollador	PC	$420.000	T12
T14	Sistema de contacto	4	Desarrollador	PC, internet	$560.000	T9, T13
T15	Sistema de favoritos	3	Desarrollador	PC	$420.000	T13
T16	Validación de datos	4	Desarrollador	PC	$560.000	T10
T17	Integración del sistema	4	Desarrollador + Scrum Master	PC	$672.000	T9, T16
T18	Testing funcional	5	Tester	PC	$490.000	T17
T19	Corrección de errores	4	Desarrollador	PC	$560.000	T18
T20	Seguridad	3	Desarrollador + DBA	PC	$504.000	T17
T21	Optimización	3	DBA	PC	$462.000	T17

T22	Preparación deploy	2	Desarrollador + Scrum Master	PC, servidor	$308.000	T19, T21
T23	Deploy	1	Desarrollador	Servidor, hosting	$140.000	T22
T24	Documentación	4	Analista	PC	$448.000	T23

Metodología

Para el desarrollo del proyecto se utilizó una metodología basada en el trabajo colaborativo y la organización por etapas, combinando análisis, diseño y planificación. Si bien no se aplicó un marco metodológico de manera estricta, se tomaron algunos conceptos de metodologías ágiles, como la división de tareas, la asignación de roles y el trabajo iterativo.

En primer lugar, se identificaron los actores principales del sistema, como comprador, vendedor, visitante y administrador. A partir de esto, se comenzó a pensar en cómo interactúan estos usuarios con la plataforma, lo cual fue fundamental para el desarrollo del diagrama de casos de uso y la definición de funcionalidades.

Luego, se trabajó sobre el diseño del prototipo en Figma, donde se representaron las distintas vistas del sistema, como el registro, login, navegación principal, publicaciones, carrito, mensajería y configuración. Este proceso permitió visualizar de forma concreta cómo sería la experiencia del usuario dentro de la plataforma.

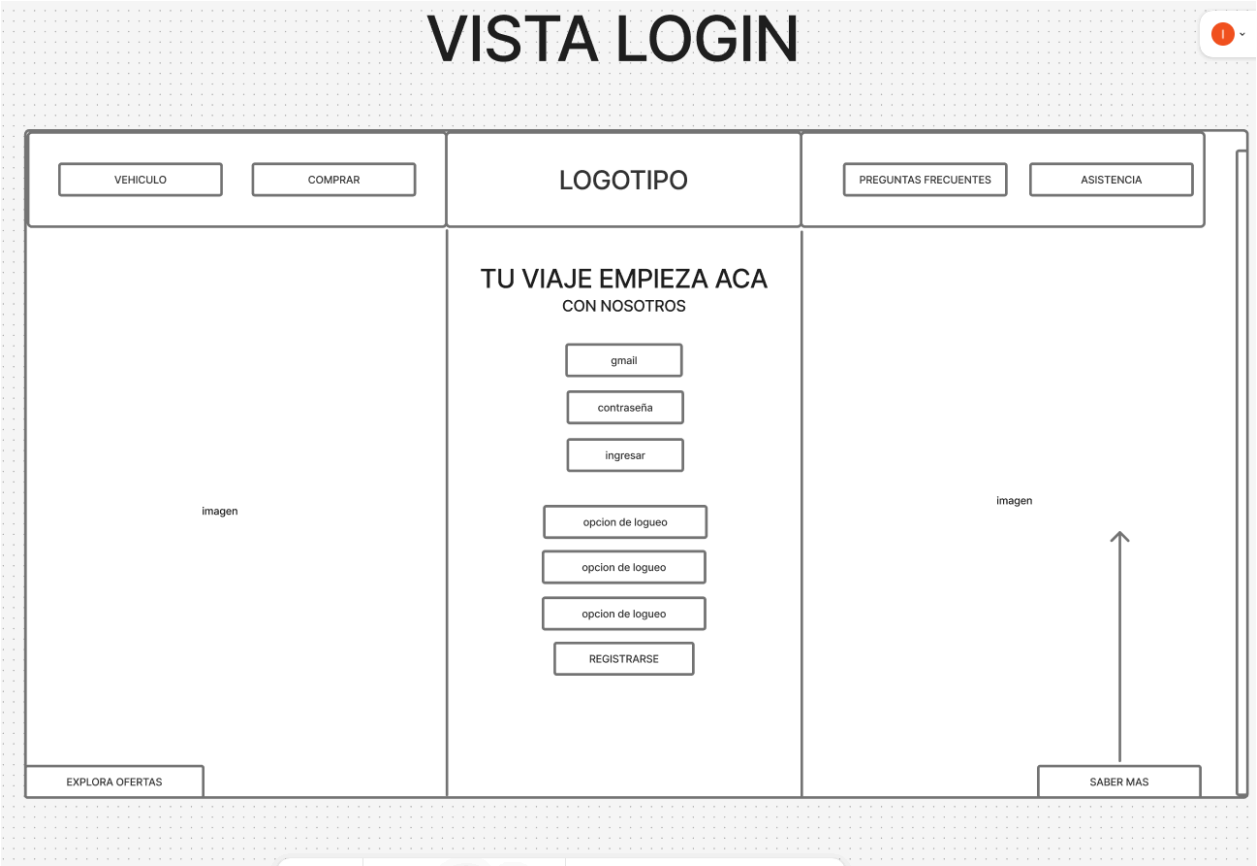
A medida que se avanzaba en el diseño, se fueron identificando las necesidades de datos del sistema, lo que permitió relacionar las vistas con la estructura de la base de datos. De esta manera, cada pantalla no se pensó de forma aislada, sino en función de la información que debía manejar.

Además, el trabajo se organizó mediante la asignación de roles dentro del equipo, donde se destacó la figura del Scrum Master, encargada de coordinar las tareas y facilitar la comunicación entre los integrantes. Esto permitió mantener un orden y un seguimiento del avance del proyecto.

En general, la metodología utilizada combinó investigación, análisis funcional, diseño visual y organización grupal, permitiendo desarrollar el proyecto de manera progresiva y estructurada.

Un acuerdo: Nuestro profesor nos comentó que todos los integrantes del grupo deben hacer cada cosa del proyecto, entonces cada tarea que nos ayude avanzar con el proyecto lo hicimos juntos. Obviamente nos dividimos las vistas ya que eran un montón

VISTA LOGIN



La vista de login representa el punto de acceso principal al sistema CarFind y cumple un rol fundamental dentro de la plataforma, ya que es la encargada de autenticar a los usuarios antes de permitir el acceso a funcionalidades más avanzadas, como la compra, venta, mensajería o gestión de perfil. Esta pantalla no solo tiene una función técnica, sino también una gran importancia desde el punto de vista de la experiencia del usuario, ya que es una de las primeras interacciones que una persona tiene con el sistema.

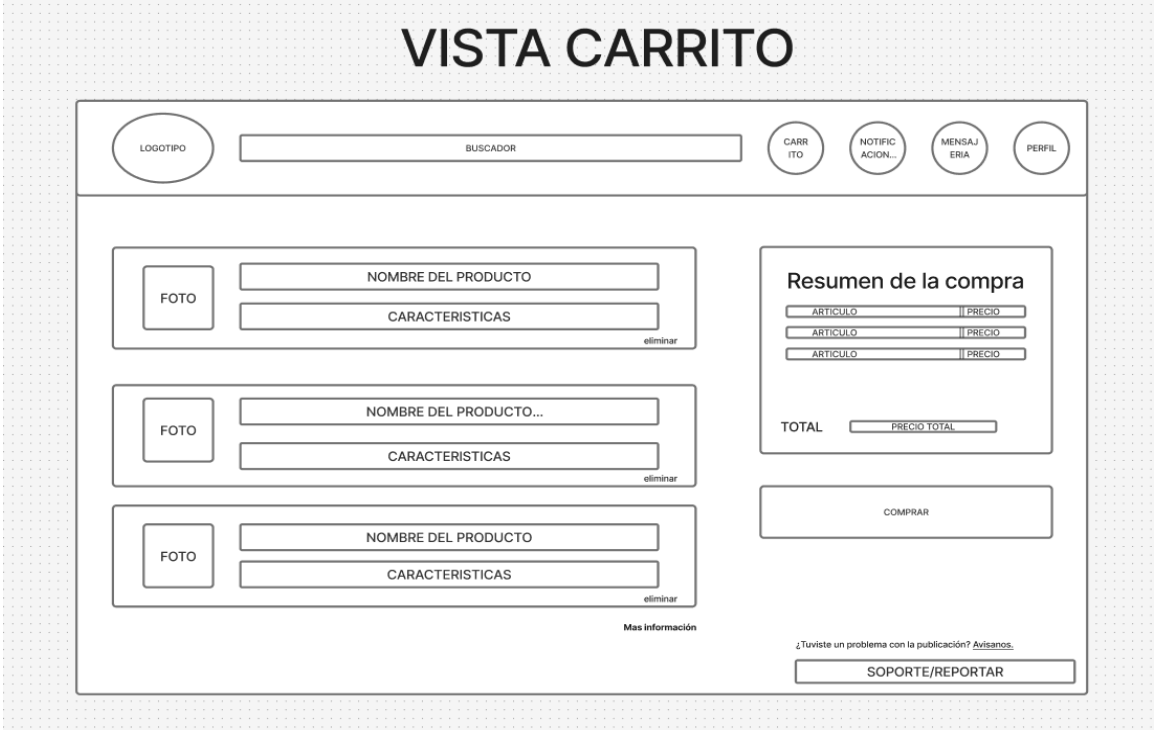
En el diseño del prototipo, la vista de login presenta una estructura clara y organizada, pensada para que cualquier usuario pueda comprender rápidamente qué debe hacer. Generalmente incluye campos para ingresar el correo electrónico o nombre de usuario y la contraseña, acompañados de botones de acción como “Iniciar sesión”. También se contemplan

opciones adicionales como el acceso al registro en caso de no tener una cuenta, o la recuperación de contraseña ante posibles olvidos.

Desde el punto de vista del diseño, se buscó que esta pantalla sea simple, intuitiva y visualmente coherente con el resto del sistema. Se tomó como referencia interfaces conocidas, lo que facilita que el usuario no tenga que aprender desde cero cómo interactuar con la plataforma. La disposición de los elementos, el uso de espacios y la jerarquía visual están pensados para guiar al usuario de manera natural hacia la acción principal, que es iniciar sesión.

A nivel funcional, esta vista se relaciona directamente con la gestión de usuarios dentro del sistema. Cuando una persona ingresa sus datos, el sistema debe validar que la información sea correcta y permitir el acceso únicamente si las credenciales coinciden con las almacenadas en la base de datos. Esto implica la existencia de mecanismos de seguridad, como el almacenamiento de contraseñas en formato cifrado y la validación de datos de entrada.

VISTA CARRITO



La vista de carrito cumple un rol importante dentro del sistema CarFind, ya que funciona como un espacio donde el usuario puede organizar y revisar las publicaciones de vehículos que le interesan antes de avanzar con una posible compra. Aunque en el caso de los vehículos no se trata de compras múltiples como en un e-commerce tradicional, el carrito permite centralizar la intención de compra y facilitar la toma de decisiones.

En el prototipo, esta vista presenta una estructura clara donde se muestran los vehículos que el usuario ha agregado previamente. Cada elemento del carrito incluye información relevante como el nombre del vehículo, su precio y, en algunos casos, una imagen representativa. Además, se incorporan opciones para eliminar productos del carrito o continuar con el proceso, lo que le da al usuario control sobre su selección.

Desde el punto de vista de la experiencia de usuario, el carrito actúa como un paso intermedio que permite comparar opciones y revisar la información antes de confirmar una operación. Esto resulta útil en un contexto como la compra de vehículos, donde las decisiones suelen ser más pensadas y requieren mayor análisis que en otros tipos de productos.

A nivel funcional, esta vista se relaciona con la gestión de publicaciones y usuarios dentro del sistema. Cada elemento agregado al carrito está vinculado a una publicación específica y a un usuario determinado. Por lo tanto, el sistema debe ser capaz de almacenar esta relación y actualizarla en función de las acciones del usuario, como agregar o eliminar vehículos.

Vista Métodos de pago



La vista de métodos de pago es una de las más sensibles dentro del sistema CarFind, ya que está directamente relacionada con el cierre de la operación de compra. Su función principal es permitir que el usuario seleccione cómo desea realizar el pago del vehículo, mostrando de manera clara las opciones disponibles y un resumen de la compra antes de confirmar la transacción.

En el prototipo, esta vista presenta una estructura organizada donde se destacan los distintos métodos de pago posibles, como tarjeta de crédito, tarjeta de débito u otras alternativas. Cada opción está pensada para ser fácilmente identificable por el usuario, facilitando la selección según sus preferencias. Además, se incluye un resumen de la operación, donde se muestra el

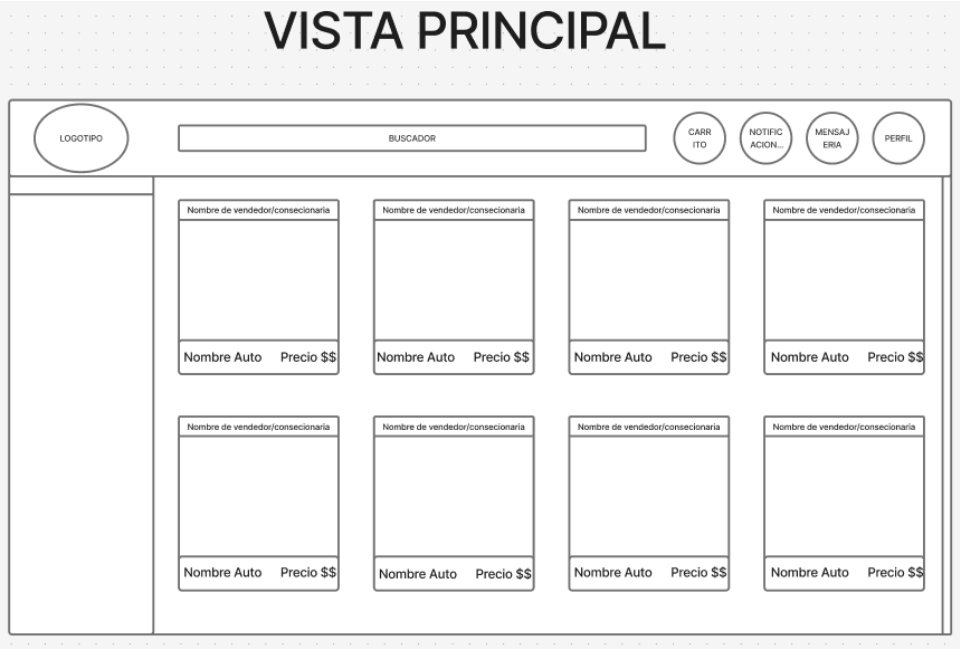
vehículo seleccionado, el precio y el total a pagar, lo que permite al usuario verificar la información antes de continuar.

Desde el punto de vista de la experiencia de usuario, esta pantalla debe transmitir confianza y claridad, ya que implica una decisión importante. Por este motivo, el diseño busca ser simple, evitando sobrecargar al usuario con información innecesaria y destacando únicamente los datos relevantes para completar el proceso.

A nivel funcional, esta vista se relaciona con diferentes componentes del sistema, como las compras, los pagos y los métodos de pago asociados a cada usuario. Aunque en este proyecto la implementación es conceptual, se plantea cómo debería funcionar el flujo en un sistema real, donde los datos ingresados deben ser validados y gestionados de forma segura.

Es importante aclarar que, en esta etapa del proyecto, no se realiza una integración real con plataformas externas de pago. Es decir, la transacción económica no se ejecuta dentro del sistema, sino que se modela su funcionamiento desde el diseño y la estructura de datos. Sin embargo, se contemplan aspectos importantes como la posibilidad de distintos métodos, la validación de información y la relación con las operaciones registradas en el sistema.

VISTA PRINCIPAL



La vista principal constituye el punto de entrada de la plataforma CarFind y es la primera pantalla que visualiza el usuario al acceder al sistema. Presenta una barra de navegación superior con logotipo de la plataforma en el extremo izquierdo, un buscador central de texto libre y accesos rápidos a las secciones de Carrito, Notificaciones, Mensajería y Perfil en el extremo derecho. En el panel lateral izquierdo se contempla un espacio para filtros de búsqueda. El cuerpo principal de la pantalla muestra un catálogo de vehículos distribuido en una grilla de tarjetas, donde cada tarjeta incluye el nombre del vendedor o concesionaria, una imagen representativa del vehículo, el nombre del modelo y el precio. Este diseño de tipo galería permite al usuario obtener una visión general de las publicaciones disponibles de forma rápida e intuitiva, minimizando el tiempo necesario para encontrar opciones relevantes y facilitando la navegación hacia el detalle de cada vehículo.

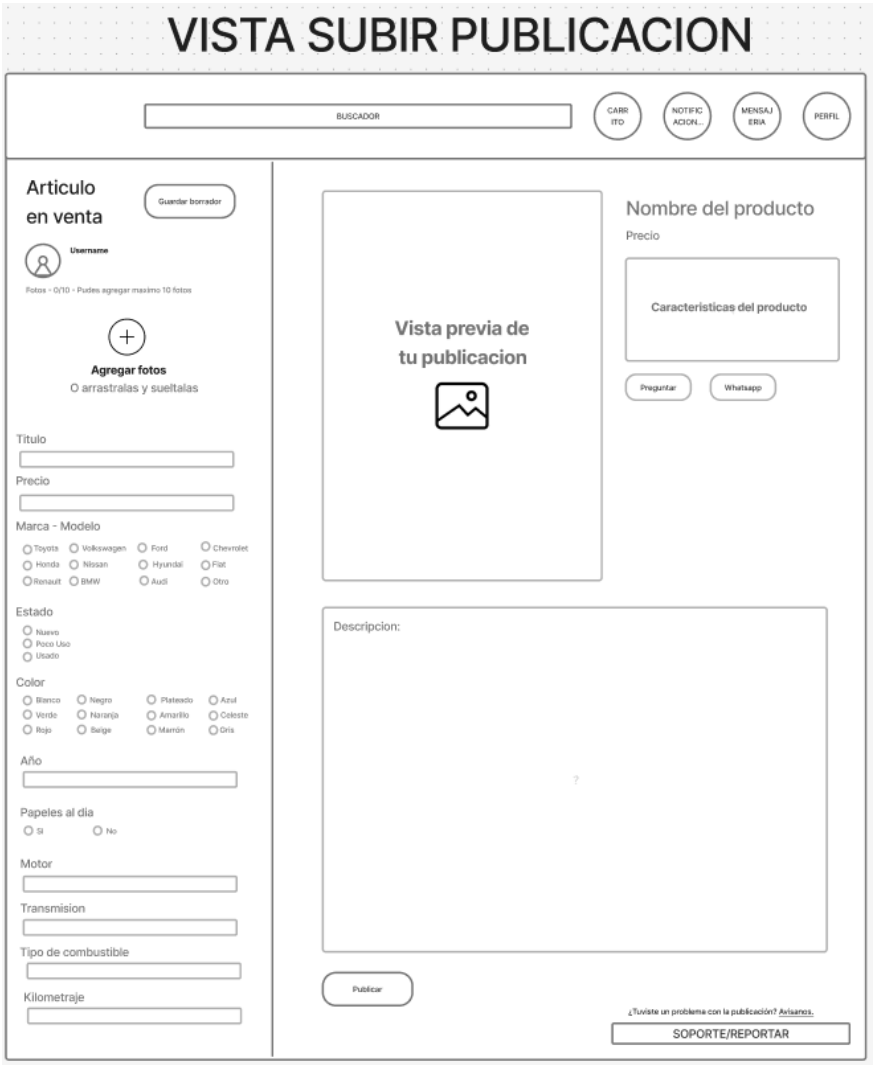
VISTA REGISTRO



La vista de Registro permite la creación de una cuenta nueva dentro de la plataforma. Mantiene la misma estructura visual tripartita que la vista de Login, con imágenes decorativas en los laterales y el formulario centrado. El formulario de registro es más extenso e incluye campos para nombre y apellido, número de teléfono, dirección, localidad, correo electrónico, fecha de nacimiento, género, contraseña y confirmación de contraseña. Al pie del formulario se ofrecen dos opciones de acceso: ingresar con cuenta registrada (para usuarios que ya completaron el

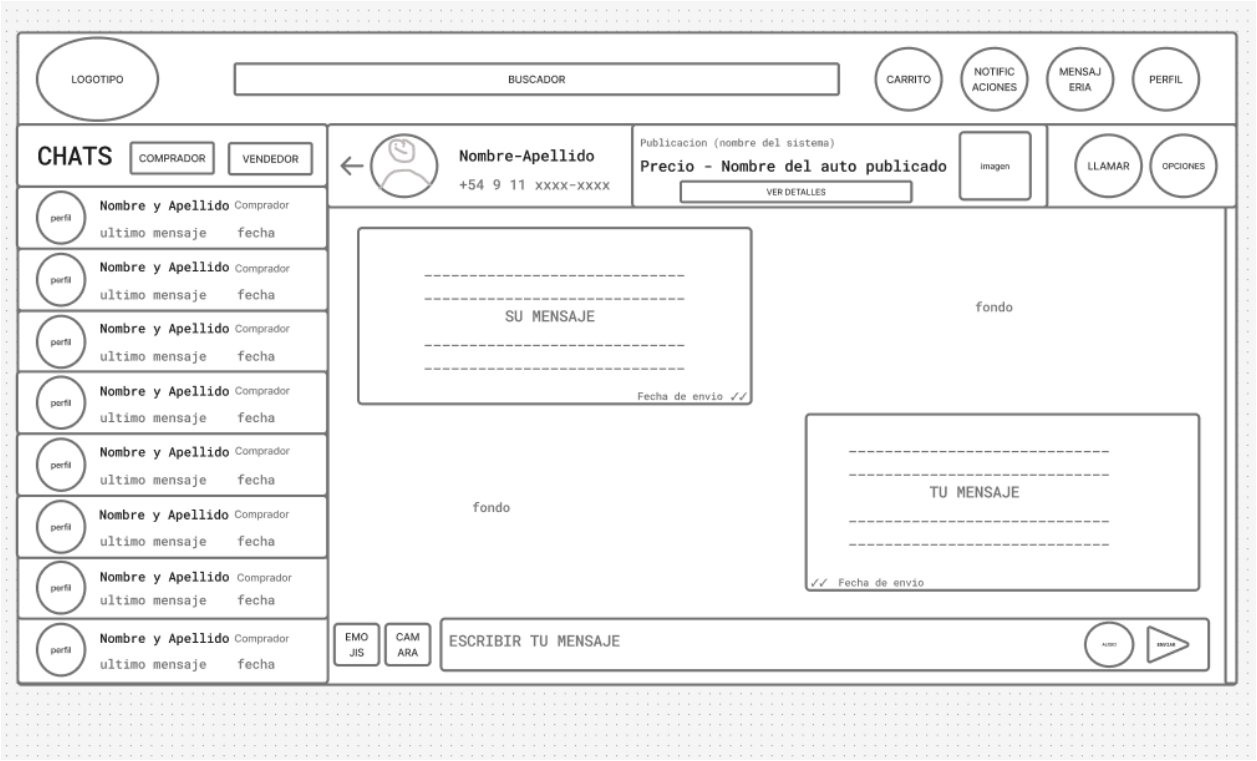
proceso) e ingresar como invitado (para usuarios que desean explorar la plataforma sin registrarse). Este módulo es esencial para garantizar que las publicaciones queden asociadas a un usuario identificable y verificable, lo que constituye un pilar fundamental del sistema de confianza y reputación de la plataforma.

VISTA PUBLICACIÓN



Esta vista permite a los vendedores cargar un nuevo vehículo a la plataforma de forma estructurada y completa. El diseño está dividido en dos paneles que trabajan en conjunto. El panel izquierdo concentra el formulario de carga de datos, que incluye: un módulo de carga de fotos (hasta 10 imágenes, con opción de arrastrar y soltar o seleccionar desde el dispositivo), campos de texto para título y precio, selección de marca mediante radio buttons con las principales marcas del mercado, selección del estado del vehículo (nuevo, poco uso, usado), selección de color, campo de año, indicación de papeles al día (sí/no), y campos adicionales para motor, transmisión, tipo de combustible y kilometraje. El panel derecho muestra una vista previa en tiempo real de cómo lucirá la publicación para el comprador, incluyendo el nombre del producto, precio, características y botones de contacto (Preguntar y WhatsApp). En la parte inferior se ofrecen las opciones de guardar borrador o publicar directamente, y se incluye un enlace de soporte para reportar problemas. Esta vista fue modificada durante el proceso de revisión para corregir la distinción entre marca y modelo.

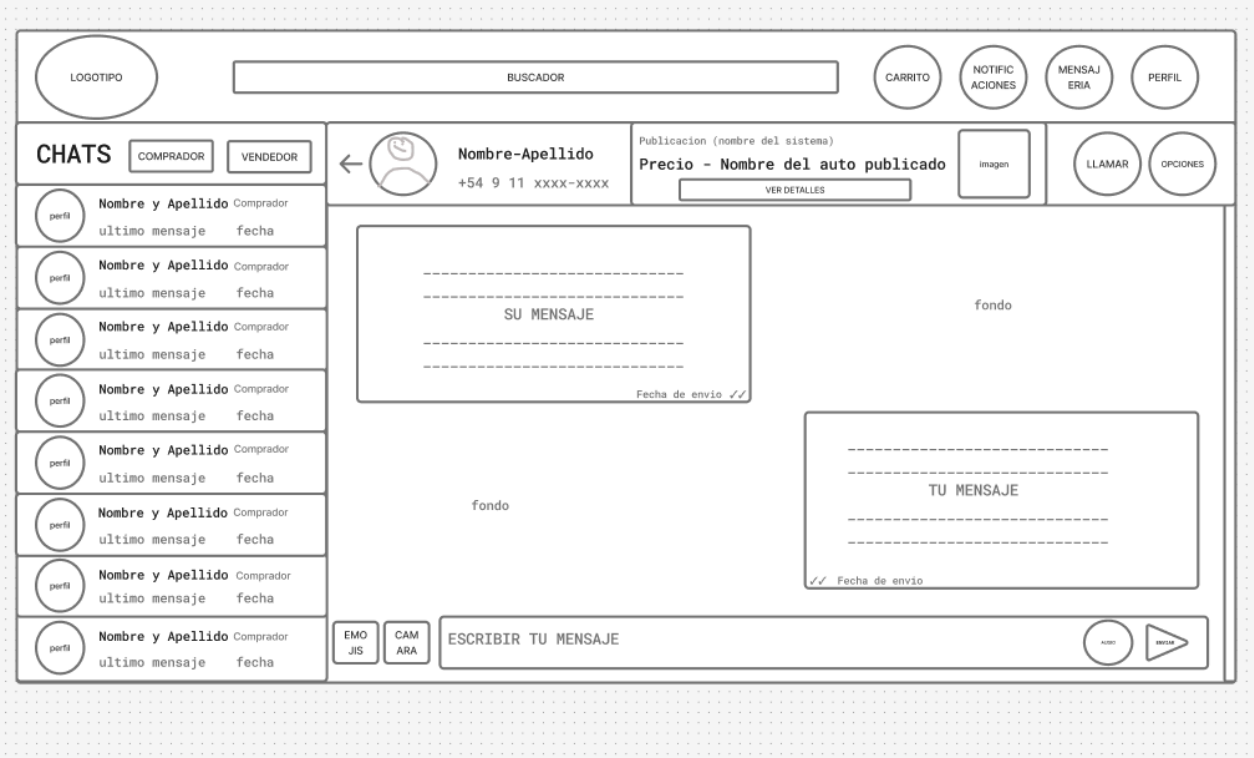
VISTA MENSAJERÍA VENDEDOR



La vista de mensajería para vendedor muestra lista de chats, buscador, nombre y apellido del contacto, último mensaje, fecha de envío, publicación asociada, precio, detalle del auto y opciones de enviar mensaje, audio, emoji, cámara y llamada. Su objetivo es facilitar la negociación. Para el vendedor, el chat debe permitir responder consultas, coordinar visitas y compartir información del vehículo. Las tablas chats y mensajes sostienen esta vista. Una mejora sería vincular cada chat con una publicación concreta.

Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

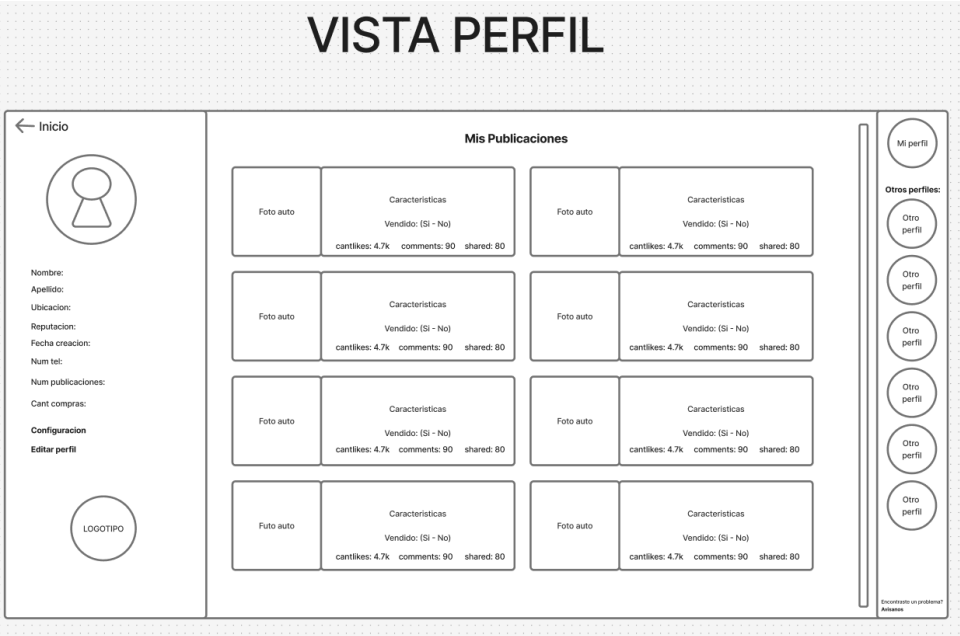
VISTA MENSAJERÍA COMPRADOR



La vista de mensajería para comprador es similar a la del vendedor, pero cambia el rol del usuario que interactúa. El comprador necesita preguntar por estado, papeles, ubicación, precio final, financiación o visita. La pantalla debería mostrar claramente con qué vendedor está hablando y sobre qué publicación. El diseño del prototipo muestra la separación entre “su mensaje” y “tu mensaje”, lo cual ayuda a la lectura conversacional. Desde el backend, los mensajes deben tener control de permisos.

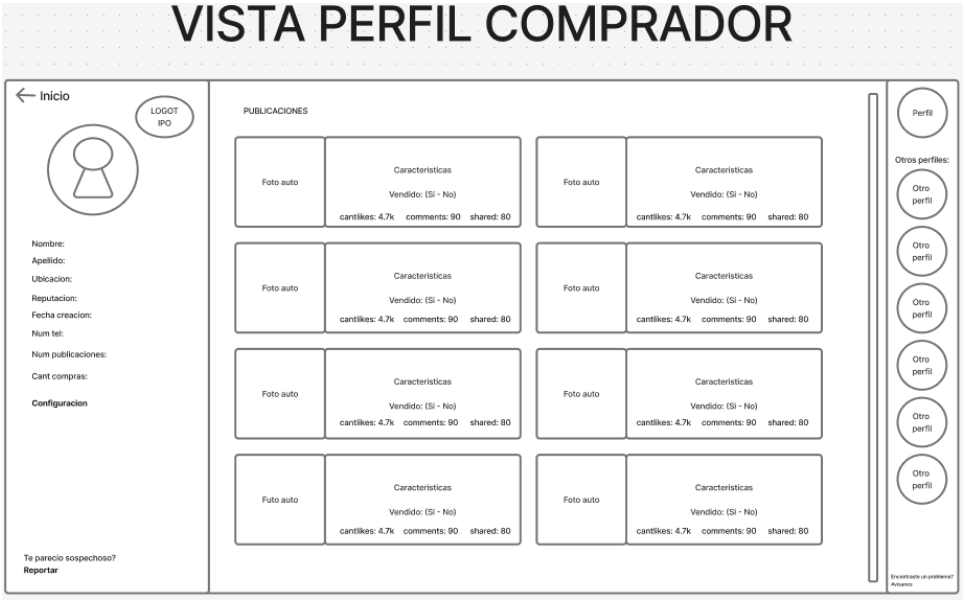
Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás

VISTA PERFIL



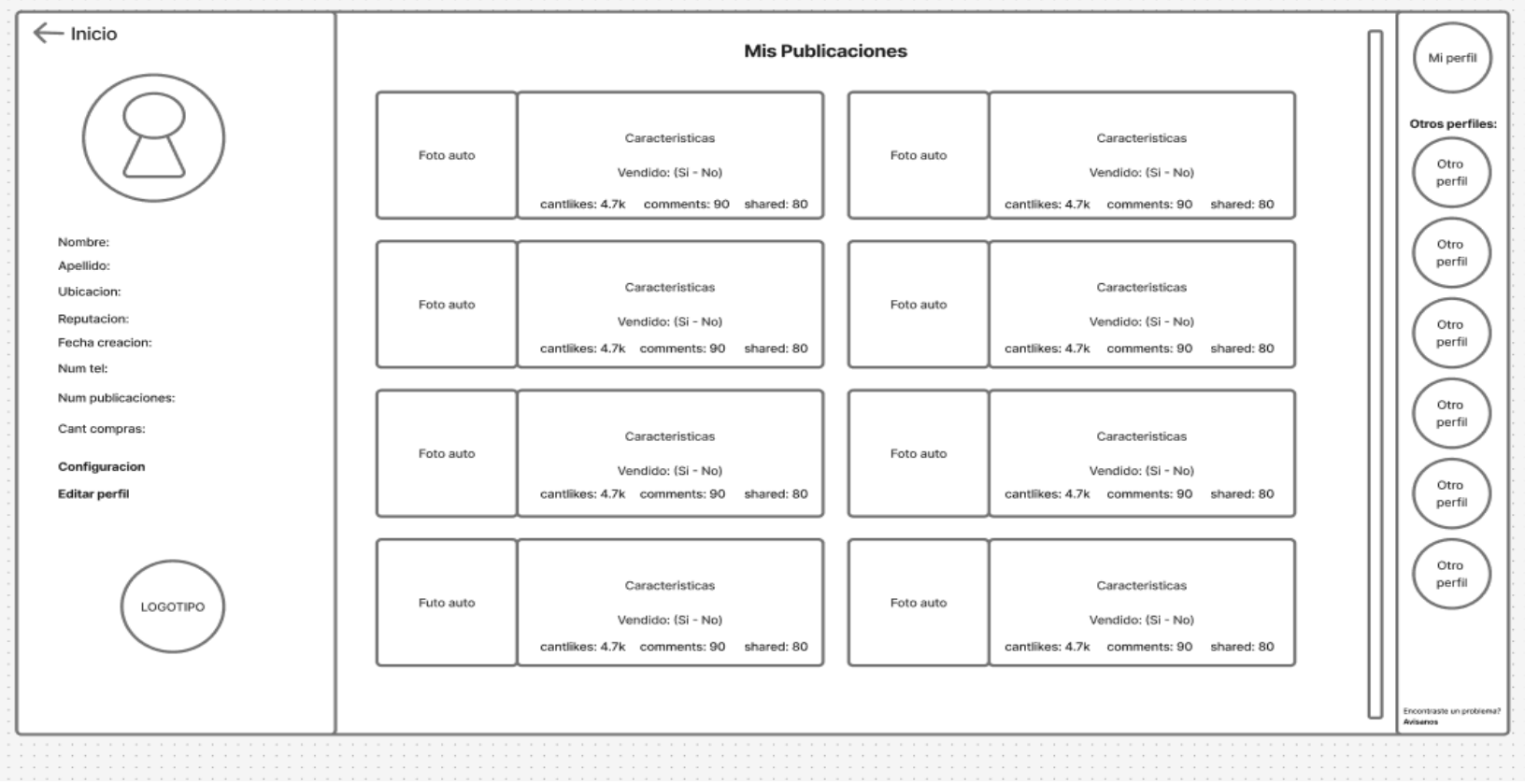
La vista de Perfil del Vendedor centraliza toda la información pública del usuario registrado como vendedor y actúa como su carta de presentación dentro de la plataforma. La pantalla se organiza en tres columnas. La columna izquierda presenta el avatar del usuario, su nombre y apellido, ubicación, puntuación de reputación, fecha de creación de la cuenta, número de teléfono, cantidad de publicaciones activas y cantidad de compras realizadas. Desde este panel también se accede a las opciones de Configuración y Editar Perfil. La columna central, que ocupa la mayor parte del espacio, despliega la grilla de publicaciones del vendedor, donde cada tarjeta muestra la foto del vehículo, sus características principales, el estado de venta (vendido o disponible) y métricas de interacción social (likes, comentarios y compartidos). La columna derecha ofrece acceso a otros perfiles de la comunidad, facilitando la navegación entre usuarios y promoviendo la interacción dentro del ecosistema de la plataforma.

VISTA COMPRADOR



La vista de Perfil del Comprador muestra la información pública de un usuario con rol de comprador, accesible desde los resultados de búsqueda, el listado de perfiles o desde una conversación de mensajería. Su estructura es similar a la del perfil de vendedor, con tres columnas diferenciadas. La columna izquierda presenta los datos personales públicos: nombre, apellido, ubicación, reputación, fecha de creación, número de teléfono, número de publicaciones y cantidad de compras realizadas. En la parte inferior izquierda se incluye la opción de reportar al usuario como sospechoso, lo que activa el sistema de moderación de la plataforma. La columna central lista las publicaciones activas del comprador con foto, características y estado de venta. La columna derecha permite navegar hacia otros perfiles de la comunidad. A diferencia del perfil de vendedor, esta vista no incluye las opciones de edición, ya que está pensada para ser visualizada por terceros. Esta dualidad de perfiles promueve la transparencia y confianza dentro de la plataforma al permitir que ambas partes de una transacción puedan conocer la reputación e historial de la contraparte.

VISTA VENDEDOR

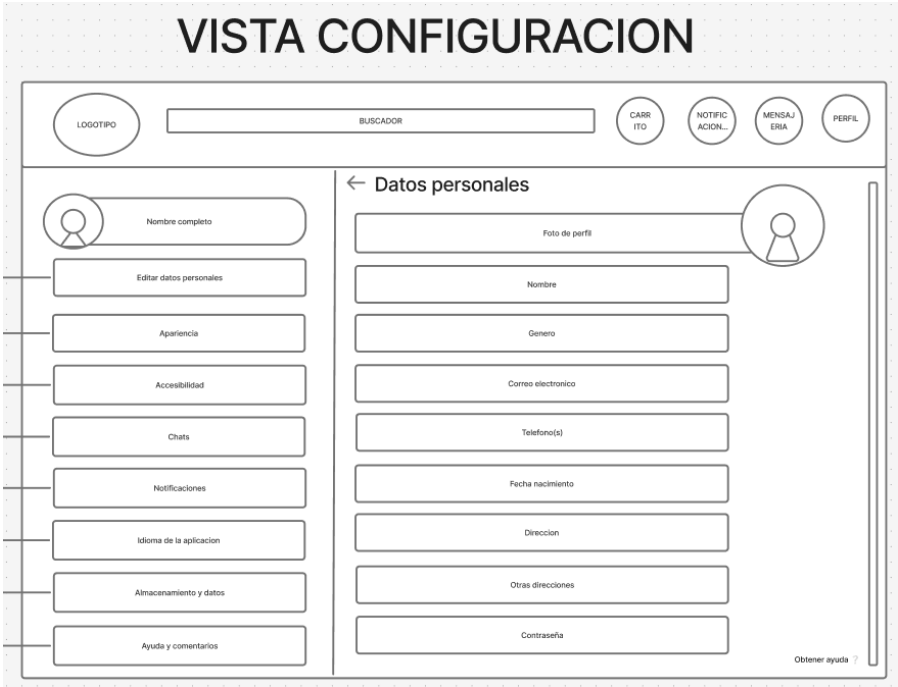


La vista de perfil vendedor muestra datos como nombre, apellido, ubicación, reputación, fecha de creación, número de teléfono, número de publicaciones y cantidad de compras. También muestra publicaciones del usuario, estado vendido o no, likes, comentarios y compartidos. Esta pantalla ayuda a construir confianza. La reputación se relaciona con reseñas, ventas completadas y actividad del usuario. Desde UX, sería conveniente distinguir información pública e información privada.

Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de

una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

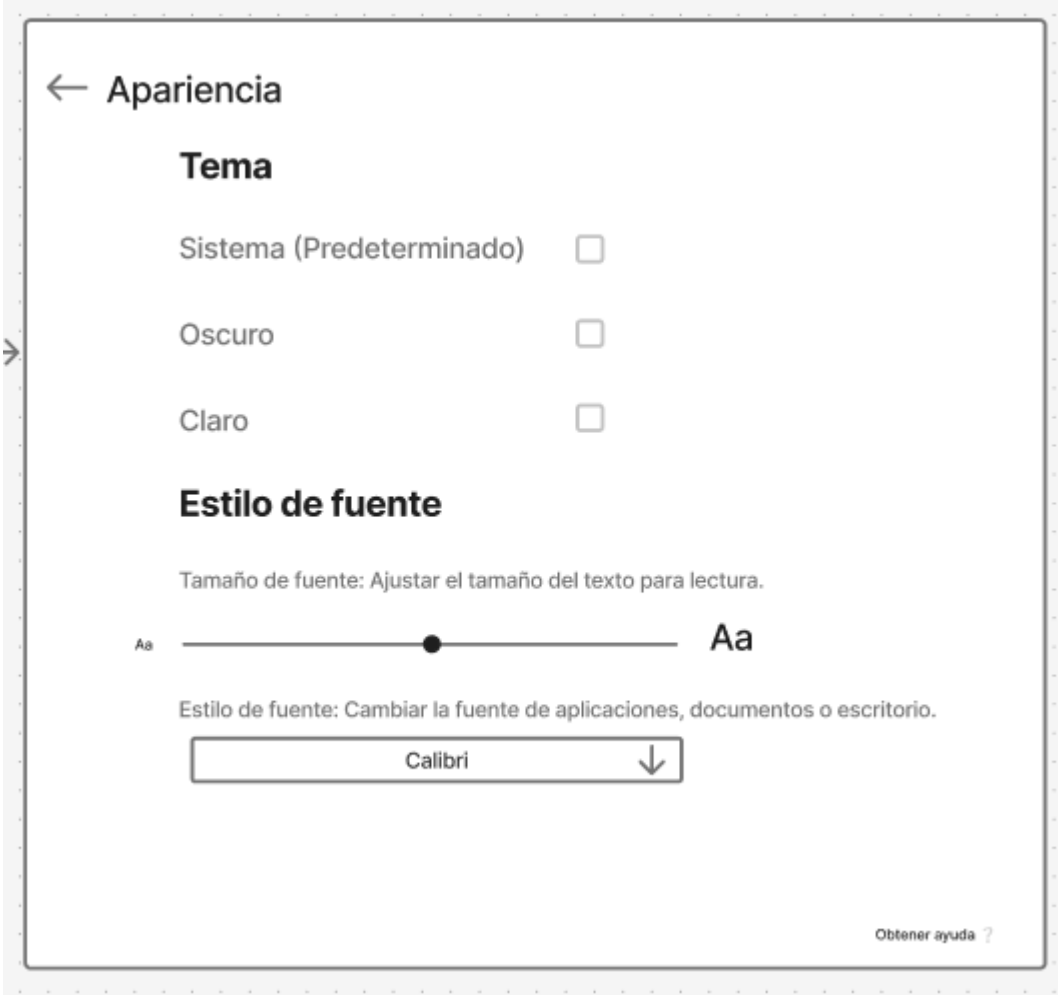
VISTA CONFIGURACIÓN



La vista de Configuración permite al usuario gestionar todos los aspectos personalizables de su cuenta de forma centralizada. La pantalla se divide en dos paneles. El panel izquierdo actúa como menú de navegación de las distintas secciones de configuración, que incluyen: Editar datos personales, Apariencia (para personalizar el tema visual de la aplicación), Accesibilidad (opciones de tamaño de texto, contraste, etc.), Chats (configuración de notificaciones de mensajería), Notificaciones (gestión de alertas del sistema), Idioma de la aplicación, Almacenamiento y datos, y Ayuda y comentarios. El panel derecho, al seleccionar la opción de datos personales, despliega un formulario de edición completo con campos para foto de perfil,

nombre, género, correo electrónico, teléfono(s), fecha de nacimiento, dirección, otras direcciones y contraseña. Esta estructura modular permite al usuario acceder rápidamente a la sección que necesita sin navegar por múltiples pantallas, mejorando significativamente la experiencia de uso del sistema.

VISTA APARIENCIA

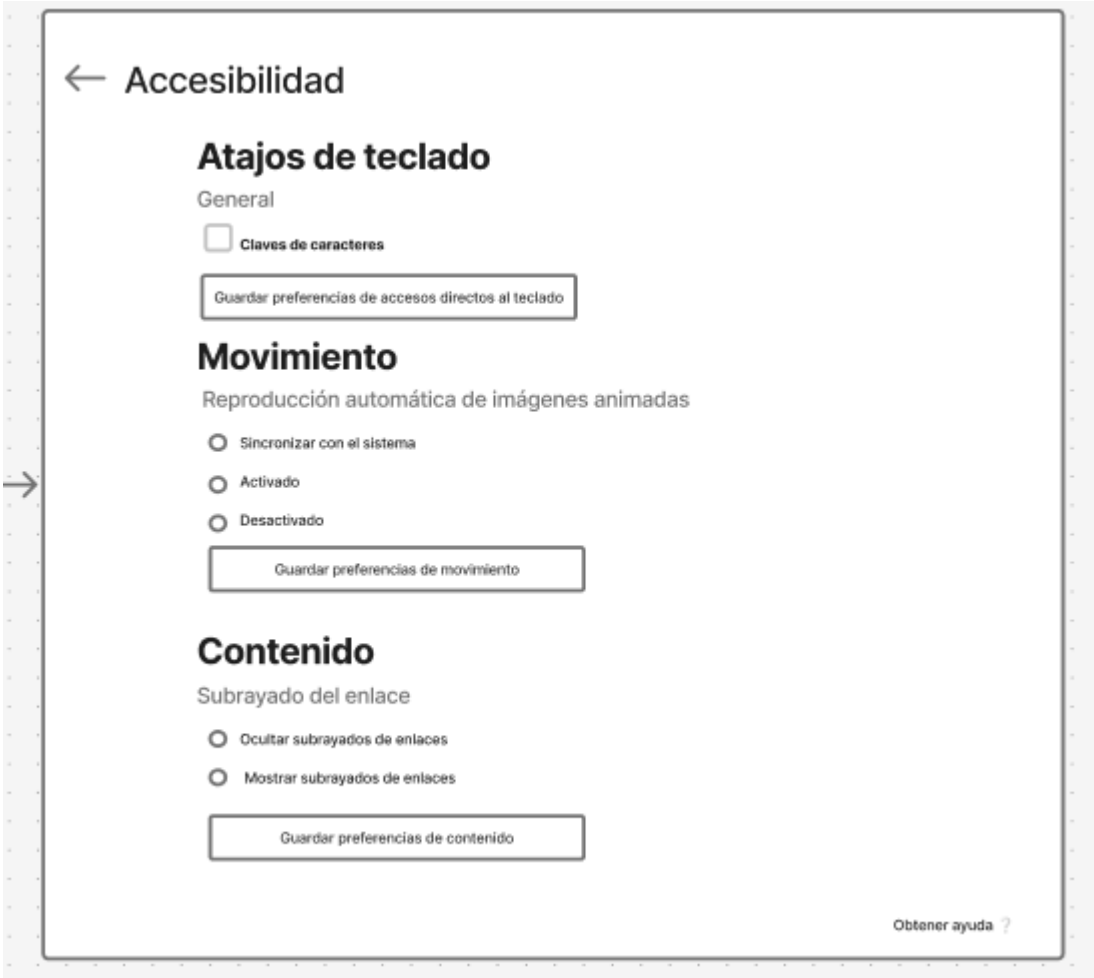


La vista de apariencia permite elegir tema del sistema, oscuro o claro, además de estilo y tamaño de fuente. Esta pantalla apunta a personalización y accesibilidad visual. Aunque parece

secundaria, mejora la experiencia de usuarios con distintas necesidades de lectura. En backend, puede almacenarse como preferencia. En frontend, el framework debería aplicar variables de tema. Una plataforma de autos puede tener usuarios jóvenes y adultos mayores; por eso el tamaño de fuente y el contraste importan.

Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

VISTA ACCESIBILIDAD

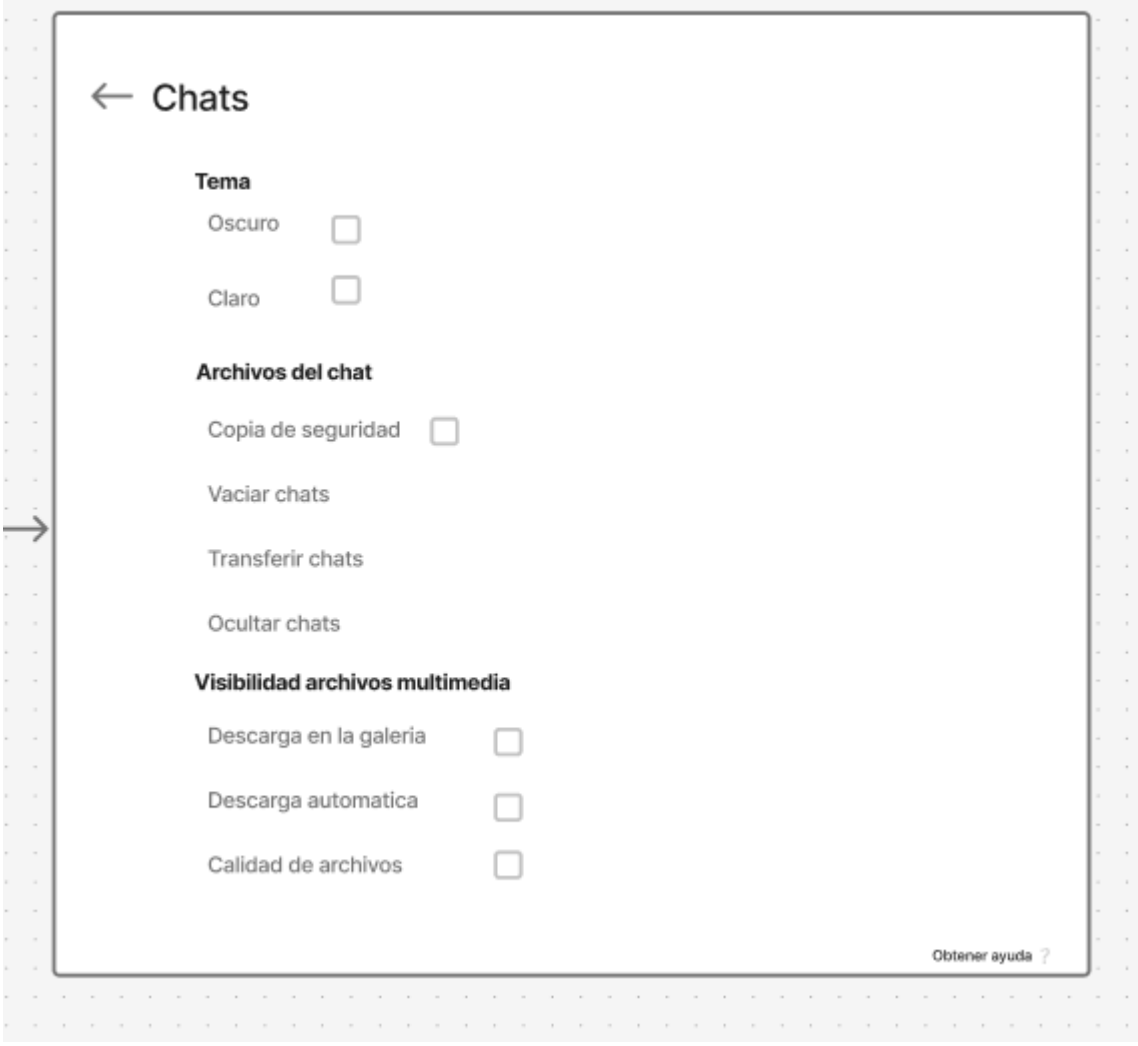


La vista de accesibilidad incluye atajos de teclado, movimiento, subrayado de enlaces y reproducción automática de imágenes animadas. Esta vista es valiosa porque permite que el sistema sea más inclusivo. Desde el punto de vista técnico, estas opciones deberían guardarse por usuario y aplicarse en todas las vistas. Para mejorarla, debería agregarse contraste alto, lector de pantalla, navegación por tabulación y textos alternativos en imágenes de vehículos.

Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de

una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

VISTA DE CHATS EN CONFIGURACIÓN

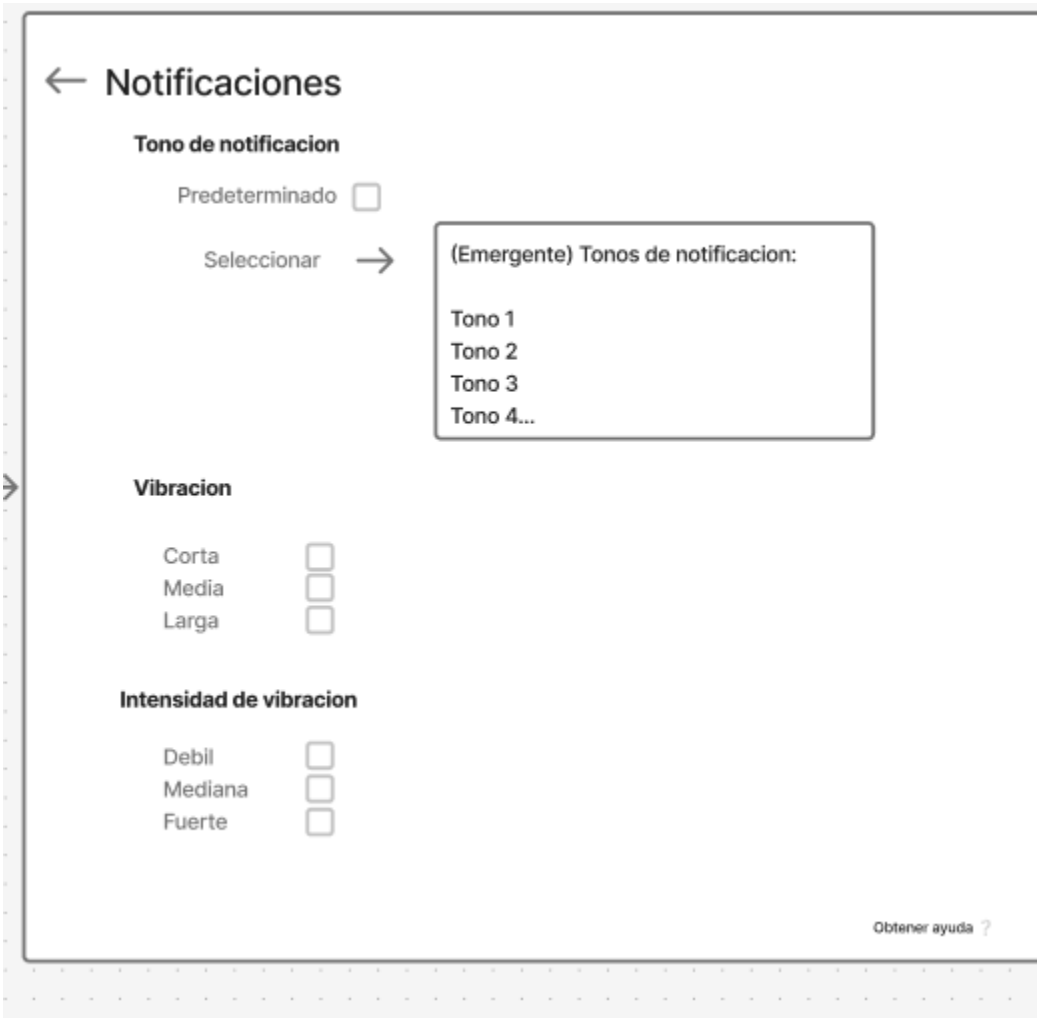


La sección de chats dentro de configuración permite administrar tema, copia de seguridad, vaciar chats, transferir chats, ocultar chats, visibilidad de archivos multimedia y descarga automática.

Esta vista se relaciona con la gestión de información conversacional. En la base actual existen chats y mensajes, pero no se observan tablas para preferencias de chat, backups o archivos adjuntos de mensajes. Una mejora sería crear mensajes_multimedia o archivos_chat.

Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

VISTA DE NOTIFICACIONES



La vista de notificaciones permite configurar tono, vibración, intensidad y tonos emergentes. La tabla notificaciones almacena mensajes asociados al usuario, tipo, leido, entidad_id y fecha. Sin embargo, la configuración de cómo se reciben esas notificaciones debería almacenarse aparte, posiblemente vinculada a dispositivos_usuario. Esta vista es importante porque en un marketplace las notificaciones sostienen la interacción: nuevos mensajes, pagos aprobados, ofertas recibidas, publicación vendida o reporte resuelto.

Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de

una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

VISTA DE ALMACENAMIENTO Y DATOS

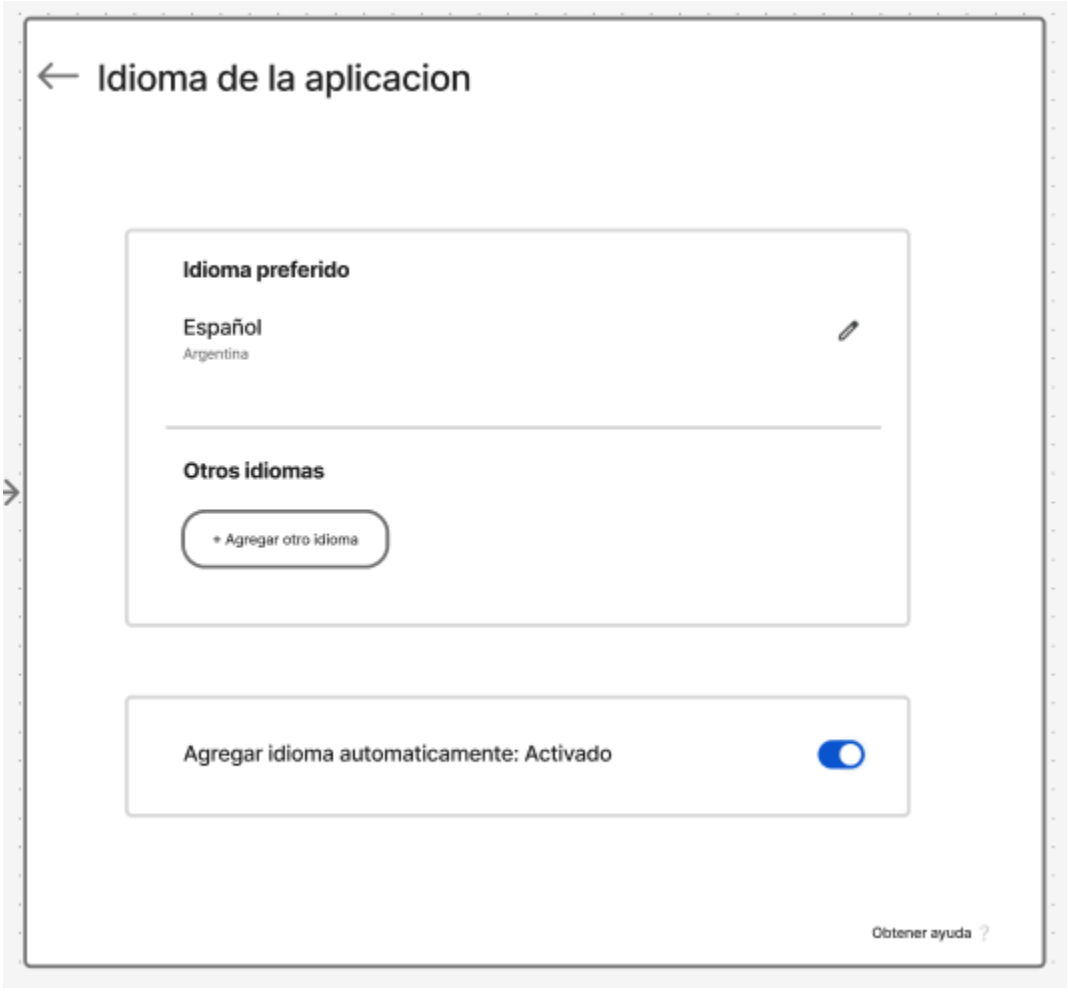


La vista de almacenamiento y datos muestra uso de almacenamiento, datos, media, mensajes, fotos y otros archivos. En un sistema real, esta vista sirve para administrar consumo de espacio y descargas. La base actual tiene multimedia para publicaciones, pero no una tabla de archivos de chat ni métricas por usuario. Una mejora sería registrar tamaño de archivos, tipo, fecha de subida

y propietario. Técnicamente, el almacenamiento de imágenes conviene hacerlo en un servicio de objetos y guardar en la base solo la URL.

Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

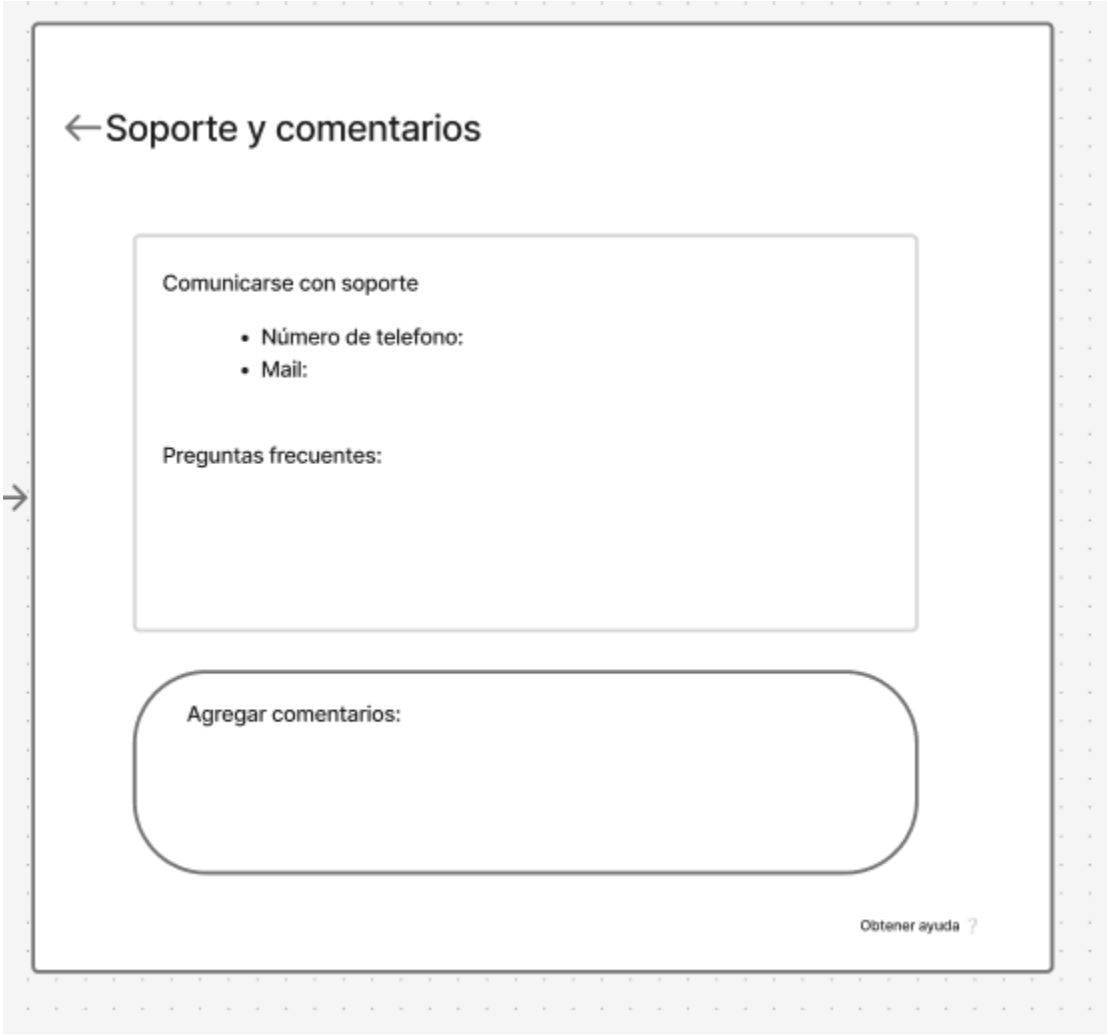
VISTA DE IDIOMA DE LA APLICACIÓN



La vista de idioma permite elegir español de Argentina y agregar otros idiomas. Esta funcionalidad anticipa internacionalización. Para un proyecto local puede parecer innecesaria, pero para una plataforma escalable resulta útil. La base podría guardar idioma_preferido en preferencias_usuario. En el caso de CarFind, también hay terminología regional: auto, coche, vehículo, patente, dominio, VTV, transferencia, prenda y embargo. Si el sistema crece a otros países, esos términos deberían adaptarse.

Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

VISTA DE SOPORTE Y COMENTARIOS



La vista de soporte permite comunicarse con soporte, consultar preguntas frecuentes y agregar comentarios con número de teléfono y mail. Esta vista se vincula con reportes y posiblemente con una futura tabla tickets_soporte. Actualmente reportes permite denunciar publicaciones, pero soporte general podría requerir categorías más amplias: problema con pago, problema con chat, cuenta bloqueada, error en publicación, sospecha de fraude o consulta administrativa.

Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de

una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

VISTA DE REPORTES Y ESTADÍSTICAS

La vista de reportes y estadísticas permite revisar actividad, publicaciones, problemas reportados y métricas. Aunque el prototipo la muestra de forma inicial, la base de datos ya tiene estadisticas_publicacion, reportes, logs_transacciones, actividad_usuario y actividad_sospechosa. Esta pantalla podría ser usada por vendedores para medir rendimiento o por administradores para moderar la plataforma.

Relación con el proyecto: Esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás

VISTA DE CARGA DE PUBLICACIÓN DEL VENDEDOR

La vista de carga de publicación del vendedor muestra un área amplia para completar información del vehículo y publicar. Su importancia está en ordenar el proceso: primero datos básicos, luego fotos, luego características, luego vista previa y finalmente publicación. En términos de base de datos, el sistema debería crear primero un auto, luego una publicación asociada a ese auto y luego las imágenes multimedia. Si el usuario guarda borrador, la publicación podría quedar en estado PAUSADA o BORRADOR si se agrega ese estado.

Relación con el proyecto: Esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

VISTA DE CARGA DE PUBLICACIÓN DEL VENDEDOR

La vista de carga de publicación del vendedor muestra un área amplia para completar información del vehículo y publicar. Su importancia está en ordenar el proceso: primero datos básicos, luego fotos, luego características, luego vista previa y finalmente publicación. En términos de base de datos, el sistema debería crear primero un auto, luego una publicación asociada a ese auto y luego las imágenes multimedia. Si el usuario guarda borrador, la publicación podría quedar en estado PAUSADA o BORRADOR si se agrega ese estado.

Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

VISTA DE PREGUNTAS FRECUENTES Y ASISTENCIA

El prototipo incluye preguntas frecuentes, asistencia y botón de saber más. Estas secciones ayudan a reducir dudas y carga de soporte. Aunque no aparecen tablas específicas para preguntas frecuentes, se podrían administrar desde una tabla ayuda_articulos con título, categoría, contenido, fecha y estado. Para una plataforma de vehículos, las preguntas frecuentes deberían explicar cómo publicar, pagar, denunciar, verificar un vendedor y usar cuotas.

Relación con el proyecto: esta vista no debe evaluarse como una pantalla aislada. Su valor aparece cuando se conecta con el modelo de datos, las reglas de negocio y los riesgos reales de una operación de compra-venta de vehículos. Por eso, cada decisión visual debería tener una justificación técnica detrás.

BACKEND

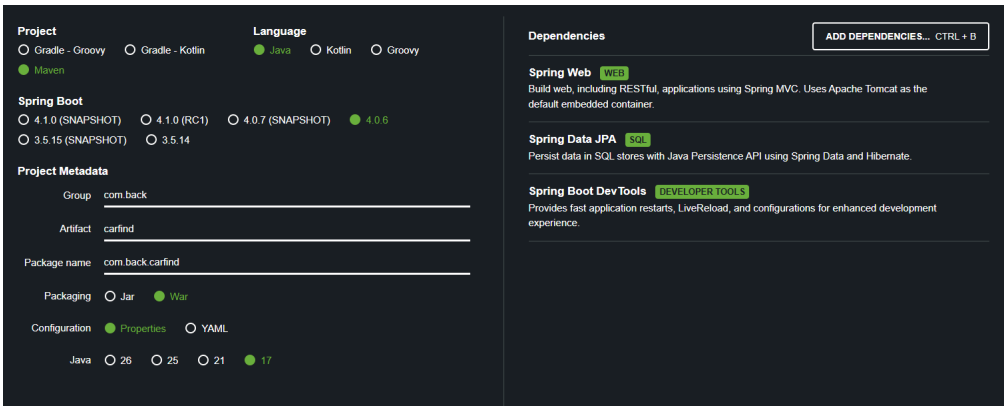
El desarrollo del backend de CarFind requirió una etapa previa de investigación tecnológica orientada a seleccionar las herramientas, frameworks y patrones de diseño más adecuados para los requerimientos del proyecto. Esta investigación contempló el análisis de distintas alternativas para el desarrollo de APIs REST en Java, la evaluación de frameworks de persistencia de datos y la comprensión de los patrones arquitectónicos que mejor se adaptan a una aplicación de la escala y complejidad de CarFind.

Investigación sobre Spring Boot

Spring Boot es un framework de código abierto basado en el ecosistema Spring de Java, diseñado para simplificar la configuración y el despliegue de aplicaciones empresariales. Su principal ventaja es que reduce drásticamente el tiempo necesario para configurar un proyecto, automatizando la mayor parte de las tareas de configuración mediante convenciones predeterminadas (convention over configuration). Esto permite a los desarrolladores enfocarse en la lógica de negocio del sistema en lugar de dedicar tiempo a tareas de infraestructura.

Para la investigación inicial se consultaron distintas fuentes académicas y técnicas especializadas, entre ellas la documentación oficial de Spring Boot (spring.io), tutoriales de plataformas reconocidas como GeeksForGeeks, FreeCodeCamp, Tokio School e Ionos Digital Guide. Estas fuentes permitieron comprender los conceptos fundamentales del framework, incluyendo la inyección de dependencias, la configuración automática (auto-configuration), los starters de Spring Boot y el sistema de capas de la arquitectura MVC aplicada al desarrollo de APIs REST.

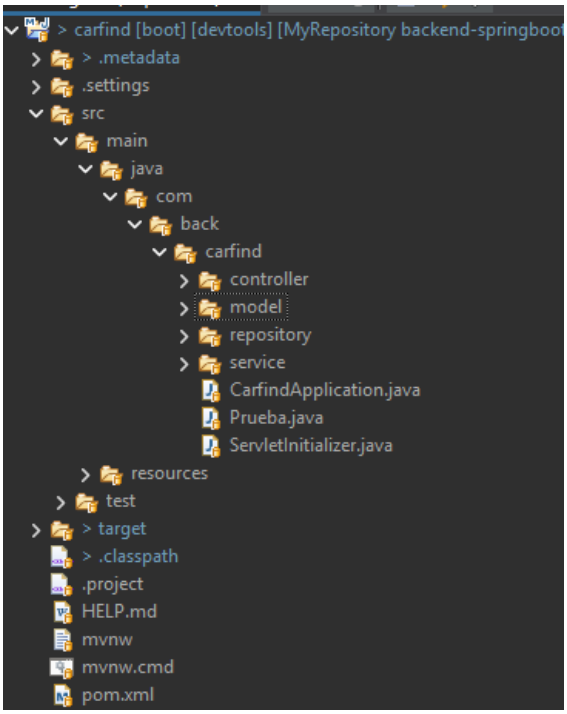
A partir de esta investigación se identificaron las dependencias necesarias para el proyecto: Spring Web (para el desarrollo de endpoints REST), Spring Data JPA (para la capa de persistencia), el driver de conexión a la base de datos relacional seleccionada y Spring Security (para la gestión de autenticación y autorización). Estas dependencias fueron configuradas mediante la herramienta Spring Initializr, que automatiza la generación de la estructura inicial del proyecto.



Estructura del Proyecto

Una vez generada la estructura base mediante Spring Initializr, el proyecto quedó organizado siguiendo la arquitectura en capas propia de las aplicaciones Spring Boot. Esta arquitectura divide el código en cuatro capas principales: el modelo (model), que contiene las clases que representan las entidades de la base de datos; el repositorio (repository), que gestiona el acceso y la persistencia de los datos mediante JPA; el servicio (service), que implementa la lógica de negocio del sistema; y el controlador (controller), que expone los endpoints de la API REST y gestiona las peticiones HTTP entrantes.

Esta separación de responsabilidades es fundamental en el desarrollo de aplicaciones empresariales, ya que facilita el mantenimiento del código, permite el testeo independiente de cada capa y hace el sistema más escalable ante cambios futuros en los requerimientos. Cada clase en el modelo se mapea directamente a una tabla de la base de datos mediante anotaciones JPA, lo que automatiza la conversión entre objetos Java y registros relacionales.



Migración de la Base de Datos a Spring Boot

Una de las tareas más significativas de esta etapa fue la migración completa de la base de datos al modelo de capas de Spring Boot. Esta migración implicó la creación de clases Java para representar las 56 tablas del modelo relacional, la configuración de las relaciones entre entidades mediante anotaciones JPA (@OneToMany, @ManyToOne, @ManyToMany, etc.), la generación de los repositorios correspondientes para cada entidad, la implementación de los servicios necesarios y la exposición de los endpoints REST en los controladores.

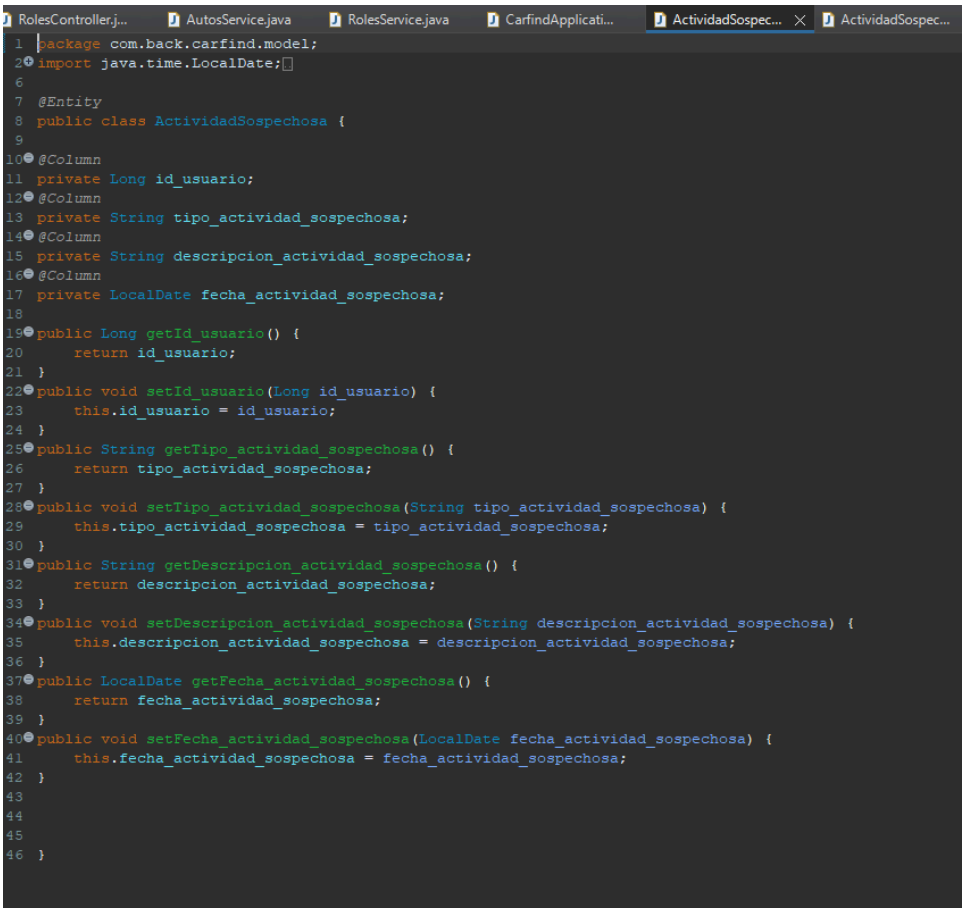
El proceso de migración abarcó 56 clases en la capa de modelo (representando la base de datos completa), 56 repositorios (uno por cada entidad), 13 clases de servicio con la lógica de

negocio más relevante y un controlador inicial para verificar el funcionamiento del sistema. A continuación se presentan ejemplos representativos de cada capa.

Capa Model

ActividadSospechosa.java:

La clase ActividadSospechosa pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información correspondiente a una actividad sospechosa registrada en el sistema, incluyendo el identificador del usuario asociado, el tipo de actividad, su descripción y la fecha en que fue detectada.



Esta estructura permite organizar los datos de manera clara y facilitar su tratamiento en las distintas capas de la aplicación. El uso de la anotación @Entity indica que la clase será gestionada como una tabla dentro de la base de datos, mientras que las anotaciones @Column permiten definir los atributos que serán almacenados. Los métodos de acceso y modificación (getters y setters) resultan necesarios para interactuar con los datos de forma controlada desde otras partes del sistema. De esta manera, la entidad cumple una función fundamental, ya que constituye la base sobre la cual se construyen las operaciones de registro, consulta y seguimiento.

ActividadUsuario.java:

La clase ActividadUsuario pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información correspondiente a una actividad sospechosa registrada en el sistema, incluyendo el identificador del usuario asociado, la accion , su descripción y la fecha en que fue detectada.

```java
package com.back.carfind.model;

import jakarta.persistence.Column;


@Entity
public class ActividadUsuario {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_actividadUsuario;
@Column
private String accion;
@Column
private String descripcion;
@Column
private LocalDate Fecha_accion;


public Long getId_actividadUsuario() {
    return id_actividadUsuario;
}
public void setId_actividadUsuario(Long id_actividadUsuario) {
    this.id_actividadUsuario = id_actividadUsuario;
}
public String getAccion() {
    return accion;
}
public void setAccion(String accion) {
    this.accion = accion;
}
public String getDescripcion() {
    return descripcion;
}
public void setDescripcion(String descripcion) {
    this.descripcion = descripcion;
}
public LocalDate getFecha_accion() {
    return Fecha_accion;
}
public void setFecha_accion(LocalDate fecha_accion) {
    Fecha_accion = fecha_accion;
}

}
```

Esta estructura permite organizar los datos de manera clara y facilitar su tratamiento en las distintas capas de la aplicación. El uso de la anotación @Entity indica que la clase será gestionada como una tabla dentro de la base de datos, mientras que las anotaciones @Column permiten definir los atributos que serán almacenados. Los métodos de acceso y modificación (getters y setters) resultan necesarios para interactuar con los datos de forma controlada desde

otras partes del sistema. De esta manera, la entidad cumple una función fundamental, ya que constituye la base sobre la cual se construyen las operaciones de registro, consulta y seguimiento.

Autos.java:

La clase Autos pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información correspondiente a una actividad sospechosa registrada en el sistema, incluyendo el identificador del usuario asociado, la acción , su descripción y la fecha en que fue detectada.

```java
package com.back.carfind.model;
import java.time.LocalDate;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
@Entity
public class Autos {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_auto;
@Column
private Long modelo_id ;
@Column
private Double version;
@Column
private Integer anio;
@Column
private Integer kms;
@Column
private Boolean vtv;
@Column
private Boolean grabado;
@Column
private Long combustible_id;
@Column
private Long transmision_id;
@Column
private String color;
@Column
private Integer puertas;
@Column
private String descripcion;
@Column
private LocalDate fecha_creacion;

public Long getId_auto() {
    return id_auto;
}
public void setId_auto(Long id_auto) {
    this.id_auto = id_auto;
}
public Long getModelo_id() {
    return modelo_id;
}
public void setModelo_id(Long modelo_id) {
    this.modelo_id = modelo_id;
}
public Double getVersion() {
    return version;
}
public void setVersion(Double version) {
    this.version = version;
}
public Integer getAnio() {
    return anio;
}
```

Esta estructura permite organizar los datos de manera clara y facilitar su tratamiento en las distintas capas de la aplicación. El uso de la anotación @Entity indica que la clase será gestionada como una tabla dentro de la base de datos, mientras que las anotaciones @Column permiten definir los atributos que serán almacenados. Los métodos de acceso y modificación

(getters y setters) resultan necesarios para interactuar con los datos de forma controlada desde otras partes del sistema. De esta manera, la entidad cumple una función fundamental, ya que constituye la base sobre la cual se construyen las operaciones de registro, consulta y seguimiento.

Banco.java:

La clase Banco pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con los bancos registrados en el sistema, incluyendo datos como su identificador, nombre, código, estado de actividad y la fecha en la que fue creado. Esta estructura permite guardar y organizar la información de cada banco para poder utilizarla dentro de las distintas funcionalidades de la aplicación.



```java
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class Banco{

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_banco;

@Column
private String nombre;

@Column
private String codigo;

@Column
private Boolean activo;

@Column
private LocalDate fecha_creacion;

public Long getId_banco() {
    return id_banco;
}

public void setId_banco(Long id_banco) {
    this.id_banco = id_banco;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getCodigo() {
    return codigo;
}

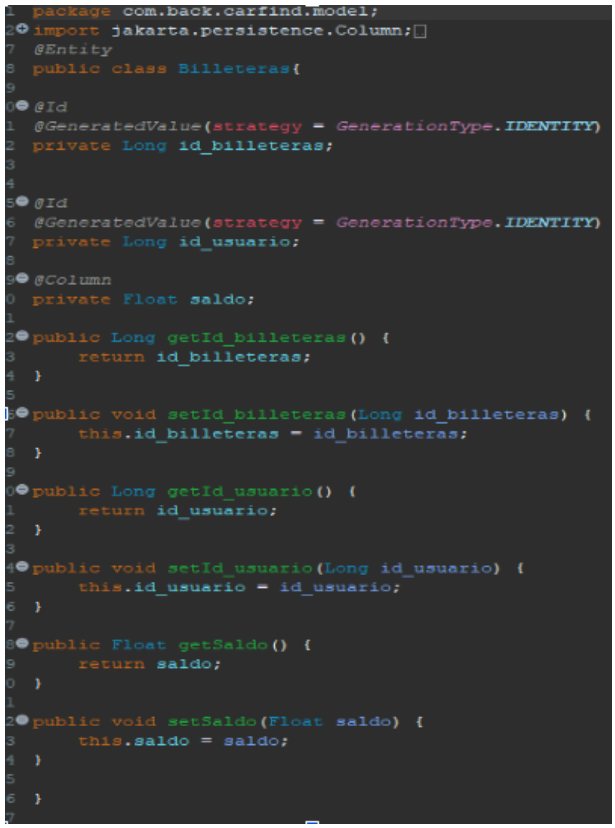
public void setCodigo(String codigo) {
    this.codigo = codigo;
}

```

El uso de la anotación @Entity permite que la clase sea reconocida como una tabla dentro de la base de datos, mientras que @Id y @GeneratedValue indican que el atributo id_banco funciona como clave principal y que su valor se genera automáticamente. Las anotaciones @Column definen los campos que serán almacenados en la tabla. Además, los métodos getters y setters permiten consultar y modificar los datos de manera controlada desde otras partes del sistema. De esta forma, la entidad Banco cumple una función importante, ya que sirve como base para realizar operaciones de registro, consulta, actualización y gestión de la información bancaria dentro de la aplicación.

Billeteras.java:

La clase **Billeteras** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con las billeteras de los usuarios dentro del sistema, incluyendo el identificador de la billetera, el usuario asociado y el saldo disponible. Esta estructura permite organizar los datos financieros de cada usuario y facilitar la gestión de los movimientos o consultas relacionadas con su dinero dentro de la aplicación.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;
@Entity
public class Billeteras{

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_billeteras;


@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_usuario;

@Column
private Float saldo;

public Long getId_billeteras() {
    return id_billeteras;
}

public void setId_billeteras(Long id_billeteras) {
    this.id_billeteras = id_billeteras;
}

public Long getId_usuario() {
    return id_usuario;
}

public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}

public Float getSaldo() {
    return saldo;
}

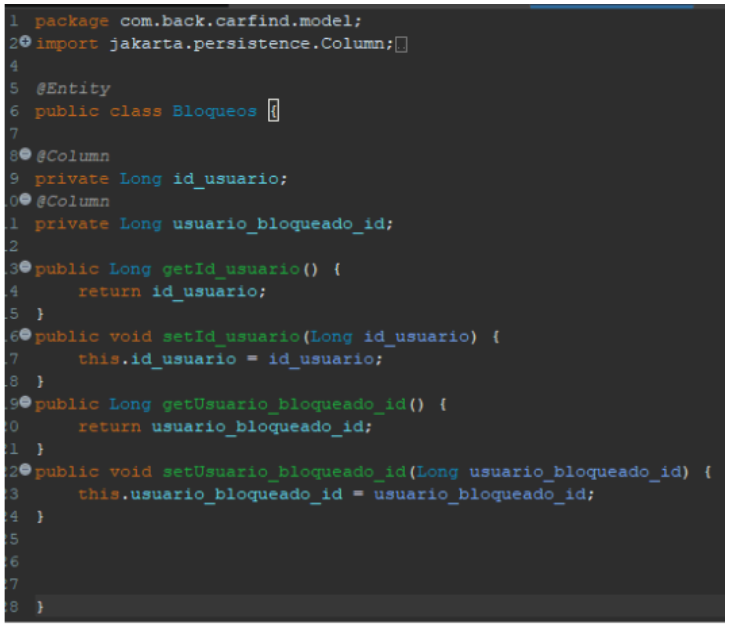
public void setSaldo(Float saldo) {
    this.saldo = saldo;
}

}
```

El uso de la anotación **@Entity** permite que la clase sea tratada como una tabla dentro de la base de datos. Las anotaciones **@Id** y **@GeneratedValue** indican los campos identificadores de la entidad y permiten generar sus valores automáticamente, aunque en este caso ambos atributos están marcados como identificadores y debería definirse correctamente cuál será la clave principal. La anotación **@Column** establece los atributos que serán almacenados, como el saldo de la billetera. Además, los métodos **getters y setters** permiten acceder y modificar la información de forma controlada desde otras partes del sistema. De esta manera, la entidad Billeteras cumple una función importante al servir como base para registrar, consultar y administrar la información de las billeteras asociadas a los usuarios.

Bloqueos.java:

La clase **Bloqueos** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con los bloqueos entre usuarios dentro del sistema, incluyendo el identificador del usuario que realiza el bloqueo y el identificador del usuario bloqueado. Esta estructura permite organizar y controlar las relaciones de bloqueo entre usuarios, facilitando la gestión de restricciones dentro de la aplicación.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;

@Entity
public class Bloqueos {

@Column
private Long id_usuario;
@Column
private Long usuario_bloqueado_id;

public Long getId_usuario() {
    return id_usuario;
}
public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}
public Long getUsuario_bloqueado_id() {
    return usuario_bloqueado_id;
}
public void setUsuario_bloqueado_id(Long usuario_bloqueado_id) {
    this.usuario_bloqueado_id = usuario_bloqueado_id;
}



}
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una entidad dentro de la base de datos, mientras que las anotaciones **@Column** definen los atributos que serán almacenados en la tabla. Los métodos **getters y setters** permiten acceder y modificar los datos de manera controlada desde otras partes del sistema. De esta forma, la entidad Bloqueos cumple una función importante, ya que permite registrar, consultar y administrar las relaciones de bloqueo entre los usuarios para mantener un control adecuado dentro de la aplicación.

BusquedaGuardadas.java:

La clase **BusquedasGuardadas** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con las búsquedas que los usuarios guardan dentro del sistema, incluyendo el identificador del usuario, la consulta realizada y la fecha en la que fue registrada. Esta estructura permite conservar un historial de búsquedas importantes para que puedan ser consultadas nuevamente y mejorar la experiencia del usuario dentro de la aplicación.



```
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class BusquedasGuardadas {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long usuario_id;

@Column
private String query;

@Column
private LocalDate fecha;

public Long getUsuario_id() {
    return usuario_id;
}

public void setUsuario_id(Long usuario_id) {
    this.usuario_id = usuario_id;
}

public String getQuery() {
    return query;
}

public void setQuery(String query) {
    this.query = query;
}

public LocalDate getFecha() {
    return fecha;
}

public void setFecha(LocalDate fecha) {
    this.fecha = fecha;
}

}
```

El uso de la anotación **@Entity** permite que la clase sea gestionada como una tabla dentro de la base de datos, mientras que **@Id** y **@GeneratedValue** indican que el atributo usuario_id funciona como identificador principal y que su valor se genera automáticamente. La anotación **@Column** define los campos que serán almacenados, como la consulta y la fecha de la búsqueda.

Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad BusquedasGuardadas cumple una función fundamental, ya que sirve como base para registrar, consultar y administrar las búsquedas realizadas por los usuarios dentro de la aplicación.

Carrito.java:

La clase **Carrito** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con los productos que los usuarios agregan al carrito de compra dentro del sistema, incluyendo el identificador del usuario, la publicación seleccionada, la cantidad de productos y el precio total. Esta estructura permite organizar los datos de las compras pendientes y facilitar la gestión de los productos antes de completar una operación de compra.

```java
package com.back.carfind.model;
import jakarta.persistence.Column;

@Entity
public class Carrito {

@Column
private Long id_usuario ;
@Column
private Long id_publicacion ;
@Column
private Integer cantidad ;
@Column
private Float precio_total;

public Long getId_usuario() {
    return id_usuario;
}
public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}
public Long getId_publicacion() {
    return id_publicacion;
}
public void setId_publicacion(Long id_publicacion) {
    this.id_publicacion = id_publicacion;
}
public Integer getCantidad() {
    return cantidad;
}
public void setCantidad(Integer cantidad) {
    this.cantidad = cantidad;
}
public Float getPrecio_total() {
    return precio_total;
}
public void setPrecio_total(Float precio_total) {
    this.precio_total = precio_total;
}



}
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una entidad dentro de la base de datos, mientras que las anotaciones **@Column** definen los atributos que serán almacenados en la tabla, como el usuario asociado, la publicación, la cantidad y el precio total. Los métodos **getters y setters** permiten acceder y modificar la información de forma controlada desde otras partes del sistema. De esta manera, la entidad Carrito cumple una función importante, ya que sirve como base para registrar, consultar y administrar los productos seleccionados por los usuarios durante el proceso de compra.

Chat.java:

La clase **Chat** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con las conversaciones entre usuarios dentro del sistema, incluyendo el identificador del chat, los usuarios que participan en la conversación y la fecha en la que fue creado. Esta estructura permite organizar y gestionar la comunicación entre usuarios, facilitando el registro y seguimiento de los chats generados dentro de la aplicación.



```java
package com.back.carfind.model;

import jakarta.persistence.Column;


@Entity
public class Chat {

@Id
@GeneratedValue
private Long id_chat;
@Column
private Long id_usuario1;
@Column
private Long id_usuario2;
@Column
private LocalDate fecha_creacion;

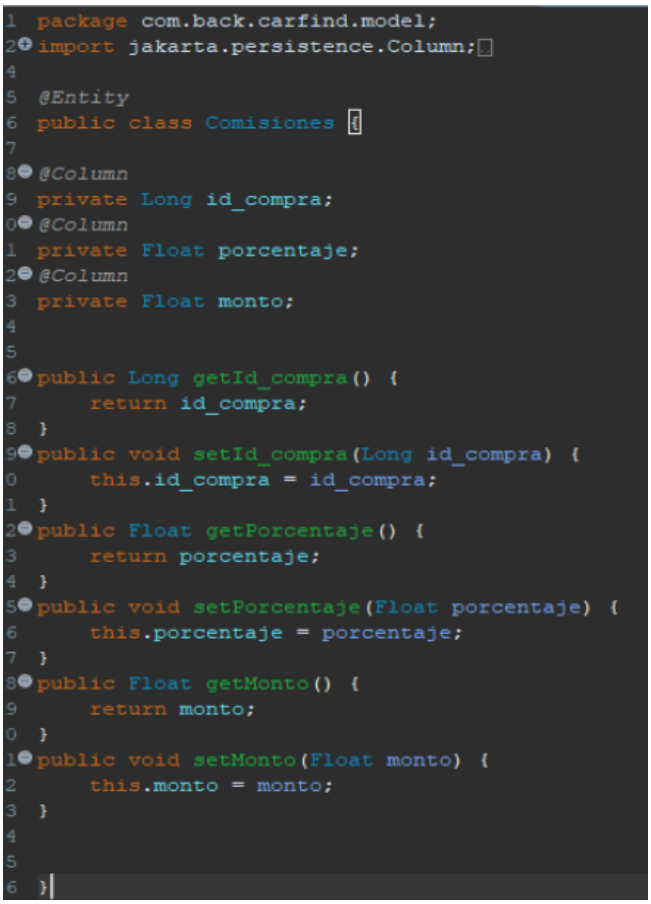

public Long getId_chat() {
    return id_chat;
}
public void setId_chat(Long id_chat) {
    this.id_chat = id_chat;
}
public Long getId_usuario1() {
    return id_usuario1;
}
public void setId_usuario1(Long id_usuario1) {
    this.id_usuario1 = id_usuario1;
}
public Long getId_usuario2() {
    return id_usuario2;
}
public void setId_usuario2(Long id_usuario2) {
    this.id_usuario2 = id_usuario2;
}
public LocalDate getFecha_creacion() {
    return fecha_creacion;
}
public void setFecha_creacion(LocalDate fecha_creacion) {
    this.fecha_creacion = fecha_creacion;
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una tabla dentro de la base de datos, mientras que **@Id** y **@GeneratedValue** indican que el atributo id_chat funciona como identificador principal y que su valor puede generarse automáticamente. Las anotaciones

@Column permiten definir los campos que serán almacenados, como los usuarios asociados y la fecha de creación del chat. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad Chat cumple una función importante, ya que sirve como base para registrar, consultar y administrar las conversaciones entre los usuarios dentro de la aplicación.

Comisiones.java:

La clase **Comisiones** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con las comisiones generadas en las compras realizadas dentro del sistema, incluyendo el identificador de la compra asociada, el porcentaje aplicado y el monto correspondiente. Esta estructura permite organizar los datos de las comisiones y facilitar su control dentro de las operaciones comerciales de la aplicación.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;

@Entity
public class Comisiones {

@Column
private Long id_compra;
@Column
private Float porcentaje;
@Column
private Float monto;


public Long getId_compra() {
    return id_compra;
}
public void setId_compra(Long id_compra) {
    this.id_compra = id_compra;
}
public Float getPorcentaje() {
    return porcentaje;
}
public void setPorcentaje(Float porcentaje) {
    this.porcentaje = porcentaje;
}
public Float getMonto() {
    return monto;
}
public void setMonto(Float monto) {
    this.monto = monto;
}


}
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una entidad dentro de la base de datos, mientras que las anotaciones **@Column** definen los atributos que serán almacenados en la tabla, como la compra relacionada, el porcentaje de comisión y el monto generado. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad Comisiones cumple una función importante, ya que sirve como base para registrar, consultar y administrar los valores de comisión generados en las transacciones realizadas dentro de la aplicación.

Compras.java:

La clase **Compras** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con las compras realizadas dentro del sistema, incluyendo el identificador de la compra, el usuario comprador, la publicación adquirida, el precio final, el estado de la operación y la fecha en la que fue realizada. Esta estructura permite organizar y registrar las transacciones efectuadas por los usuarios, facilitando el seguimiento de cada compra dentro de la aplicación.

Compras.java:

La clase **Compras** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con las compras realizadas dentro del sistema, incluyendo el identificador de la compra, el usuario comprador, la publicación adquirida, el precio final, el estado de la operación y la fecha en la que fue realizada. Esta estructura permite organizar y registrar las transacciones efectuadas por los usuarios, facilitando el seguimiento de cada compra dentro de la aplicación.



```java
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class Compras {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_compra;
@Column
private Long id_comprador;
@Column
private Long id_publicacion;
@Column
private Float precio_final;
@Column
private String estado;
@Column
private LocalDate fecha_compra;


public Long getId_compra() {
    return id_compra;
}
public void setId_compra(Long id_compra) {
    this.id_compra = id_compra;
}
public Long getId_comprador() {
    return id_comprador;
}
public void setId_comprador(Long id_comprador) {
    this.id_comprador = id_comprador;
}
public Long getId_publicacion() {
    return id_publicacion;
}
public void setId_publicacion(Long id_publicacion) {
    this.id_publicacion = id_publicacion;
}
public Float getPrecio_final() {
    return precio_final;
}
public void setPrecio_final(Float precio_final) {
    this.precio_final = precio_final;
}
public String getEstado() {
```

El uso de la anotación **@Entity** permite que la clase sea gestionada como una tabla dentro de la base de datos, mientras que **@Id** y **@GeneratedValue** indican que el atributo id_compra funciona como identificador principal y que su valor se genera automáticamente. Las anotaciones **@Column** permiten definir los campos que serán almacenados, como el comprador, la publicación, el precio, el estado y la fecha de compra. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad Compras cumple una función fundamental, ya que sirve como base para registrar, consultar y administrar las operaciones de compra realizadas dentro de la aplicación.

Cuotas.java:

La clase **Cuotas** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con las cuotas de pago generadas dentro del sistema, incluyendo los identificadores de la cuota y del pago asociado, el número de cuota, el monto, el estado, la fecha de vencimiento y la fecha en la que fue abonada. Esta estructura permite organizar y controlar los pagos realizados en diferentes períodos, facilitando el seguimiento de las obligaciones económicas dentro de la aplicación.



```java
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class Cuotas{

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_cuotas;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_pago_cuotas;

@Column
private Integer numero_cuota;

@Column
private Float monto;

@Column
private String estado;

@Column
private LocalDate fecha_vencimiento;

@Column
private LocalDate fecha_pago;

public Long getId_cuotas() {
    return id_cuotas;
}

public void setId_cuotas(Long id_cuotas) {
    this.id_cuotas = id_cuotas;
}

public Long getId_pago_cuotas() {
    return id_pago_cuotas;
}

public void setId_pago_cuotas(Long id_pago_cuotas) {
    this.id_pago_cuotas = id_pago_cuotas;
}

public Integer getNumero_cuota() {
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una tabla dentro de la base de datos, mientras que **@Id** y **@GeneratedValue** indican los atributos identificadores de la entidad y permiten generar sus valores automáticamente, aunque en este caso ambos campos están marcados como identificadores y debería definirse correctamente cuál será la clave principal. Las anotaciones **@Column** permiten establecer los campos que serán almacenados,

como el número de cuota, el monto, el estado y las fechas correspondientes. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad Cuotas cumple una función importante, ya que sirve como base para registrar, consultar y administrar los pagos divididos en cuotas dentro de la aplicación.

Direcciones.java:

La clase **Direcciones** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con las direcciones registradas dentro del sistema, incluyendo el identificador de la dirección, país, provincia, ciudad, dirección específica, tipo de dirección, si es la principal y la fecha en la que fue creada. Esta estructura permite organizar los datos de ubicación de los usuarios y facilitar su gestión dentro de las distintas funcionalidades de la aplicación.



```java
package com.back.carfind.model;
import java.time.LocalDate;

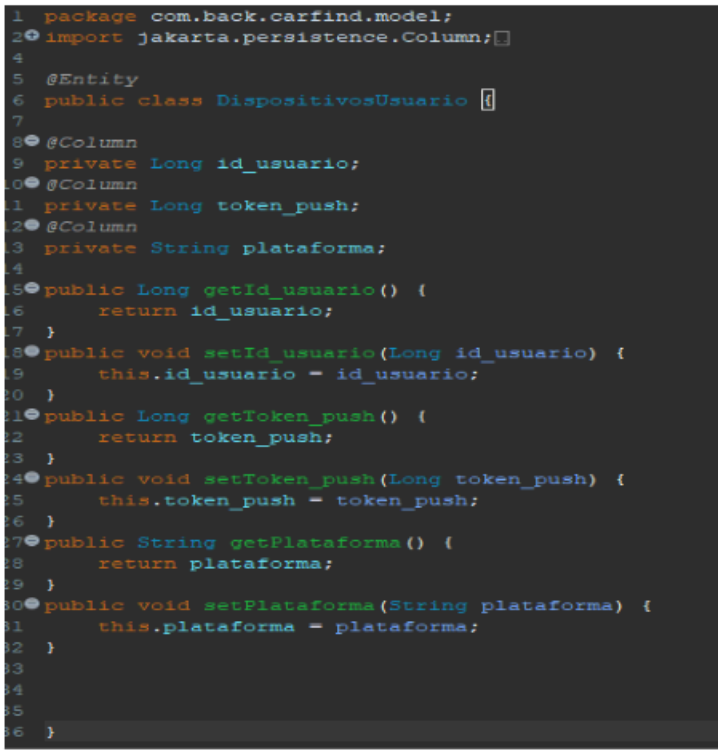
@Entity
public class Direcciones {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_direccion;
@Column
private String pais;
@Column
private String provincia;
@Column
private String ciudad;
@Column
private String direccion;
@Column
private String tipo;
@Column
private Boolean es_principal;
@Column
private LocalDate fecha_creacion_direccion;
public Long getId_direccion() {
    return id_direccion;
}
public void setId_direccion(Long id_direccion) {
    this.id_direccion = id_direccion;
}
public String getPais() {
    return pais;
}
public void setPais(String pais) {
    this.pais = pais;
}
public String getProvincia() {
    return provincia;
}
public void setProvincia(String provincia) {
    thia.provincia = provincia;
}
public String getCiudad() {
    return ciudad;
}
public void setCiudad(String ciudad) {
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una tabla dentro de la base de datos, mientras que **@Id** y **@GeneratedValue** indican que el atributo id_direccion funciona como identificador principal y que su valor se genera automáticamente. Las anotaciones **@Column** permiten definir los campos que serán almacenados, como la ubicación, el tipo de dirección, el estado principal y la fecha de creación. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad Direcciones cumple una función importante, ya que sirve como base para registrar, consultar y administrar las ubicaciones asociadas a los usuarios dentro de la aplicación.

DispositivosUsuario.java:

La clase DispositivosUsuario pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con los dispositivos asociados a los usuarios dentro del sistema, incluyendo el identificador del usuario, el token de notificación y la plataforma del dispositivo utilizado. Esta estructura permite organizar los datos necesarios para gestionar la comunicación con los usuarios mediante sus dispositivos registrados dentro de la aplicación.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;

@Entity
public class DispositivosUsuario {

@Column
private Long id_usuario;
@Column
private Long token_push;
@Column
private String plataforma;

public Long getId_usuario() {
    return id_usuario;
}
public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}
public Long getToken_push() {
    return token_push;
}
public void setToken_push(Long token_push) {
    this.token_push = token_push;
}
public String getPlataforma() {
    return plataforma;
}
public void setPlataforma(String plataforma) {
    this.plataforma = plataforma;
}



}
```

El uso de la anotación @Entity permite que la clase sea reconocida como una entidad dentro de la base de datos, mientras que las anotaciones @Column definen los atributos que serán almacenados en la tabla, como el usuario asociado, el token de notificación y la plataforma del dispositivo. Además, los métodos getters y setters permiten acceder y modificar la información

de manera controlada desde otras partes del sistema. De esta forma, la entidad DispositivosUsuario cumple una función importante, ya que sirve como base para registrar, consultar y administrar los dispositivos vinculados a los usuarios, permitiendo gestionar funcionalidades como notificaciones y comunicación dentro de la aplicación.

Encuentros.java:

La clase **Encuentros** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con los encuentros programados entre usuarios dentro del sistema, incluyendo el identificador del encuentro, la compra asociada, el vendedor, el comprador, la fecha programada, el lugar, el estado, las notas adicionales y la fecha de creación. Esta estructura permite organizar y gestionar los encuentros generados a partir de una compra, facilitando el seguimiento de las reuniones entre las partes involucradas dentro de la aplicación.



```
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class Encuentros{

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_encuentros;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_compra;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_vendedor;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_comprador;

@Column
private LocalDate fecha_programada;

@Column
private String lugar;

@Column
private String estado;

@Column
private String notas;

@Column
private LocalDate fecha_creacion;

public Long getId_encuentros() {
    return id_encuentros;
}

public void setId_encuentros(Long id_encuentros) {
    this.id_encuentros = id_encuentros;
}

public Long getId_compra() {
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una tabla dentro de la base de datos, mientras que **@Id** y **@GeneratedValue** indican los atributos identificadores de la entidad y permiten generar sus valores automáticamente, aunque en este caso varios campos están marcados como identificadores y debería definirse correctamente cuál será la clave principal. Las anotaciones **@Column** permiten establecer los campos que serán almacenados, como las fechas, el lugar, el estado y las notas del encuentro. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad Encuentros cumple una función importante, ya que sirve como base para registrar, consultar y administrar las reuniones relacionadas con las operaciones de compra dentro de la aplicación.

EstadísticasPublicación.java:

La clase **EstadisticasPublicacion** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con las estadísticas de las publicaciones dentro del sistema, incluyendo el identificador de la publicación, la cantidad de vistas, clics, favoritos, contactos generados y veces que fue compartida. Esta estructura permite recopilar y organizar datos sobre el rendimiento de cada publicación, facilitando el análisis de la interacción de los usuarios dentro de la aplicación.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;
@Entity
public class EstadisticasPublicacion {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_publicacion;

@Column
private Integer vistas;

@Column
private Integer clicks;

@Column
private Integer favoritos;

@Column
private Integer contactos;

@Column
private Integer compartidos;

public Long getId_publicacion() {
    return id_publicacion;
}

public void setId_publicacion(Long id_publicacion) {
    this.id_publicacion = id_publicacion;
}

public Integer getVistas() {
    return vistas;
}

public void setVistas(Integer vistas) {
    this.vistas = vistas;
}

public Integer getClicks() {
    return clicks;
}

public void setClicks(Integer clicks) {
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una tabla dentro de la base de datos, mientras que **@Id** y **@GeneratedValue** indican que el atributo id_publicacion funciona como identificador principal y que su valor se genera automáticamente. Las anotaciones **@Column** permiten definir los campos que serán almacenados, como las métricas de interacción de cada publicación. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad EstadisticasPublicacion cumple una función importante, ya que sirve como base para registrar, consultar y administrar los datos de rendimiento de las publicaciones dentro de la aplicación.

Facturas.java:

La clase **Facturas** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con las facturas generadas dentro del sistema, incluyendo el identificador del usuario asociado, la compra correspondiente, el monto total de la factura y la fecha en la que fue creada. Esta estructura permite organizar y registrar los comprobantes de las operaciones realizadas, facilitando el control y seguimiento de las transacciones dentro de la aplicación.



```java
package com.back.carfind.model;
import java.time.LocalDate;

@Entity
public class Facturas {

@Column
private Long id_usuario;
@Column
private Long id_compra;
@Column
private Float total_factura;
@Column
private LocalDate fehca_factura;

public Long getId_usuario() {
    return id_usuario;
}
public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}
public Long getId_compra() {
    return id_compra;
}
public void setId_compra(Long id_compra) {
    this.id_compra = id_compra;
}
public Float getTotal_factura() {
    return total_factura;
}
public void setTotal_factura(Float total_factura) {
    this.total_factura = total_factura;
}
public LocalDate getFehca_factura() {
    return fehca_factura;
}
public void setFehca_factura(LocalDate fehca_factura) {
    this.fehca_factura = fehca_factura;
}



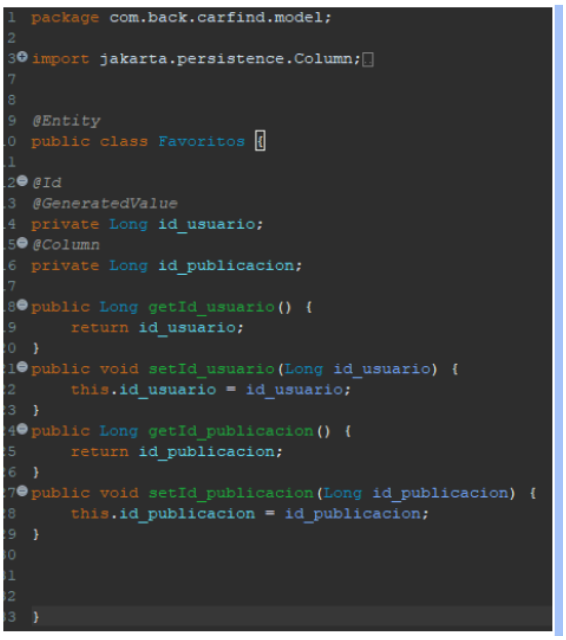
}
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una entidad dentro de la base de datos, mientras que las anotaciones **@Column** definen los atributos que serán almacenados en la tabla, como el usuario, la compra, el total de la factura y la fecha de emisión. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera

controlada desde otras partes del sistema. De esta forma, la entidad Facturas cumple una función importante, ya que sirve como base para registrar, consultar y administrar los comprobantes asociados a las compras realizadas dentro de la aplicación.

Favoritos.java:

La clase **Favoritos** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con las publicaciones que los usuarios marcan como favoritas dentro del sistema, incluyendo el identificador del usuario y el identificador de la publicación seleccionada. Esta estructura permite organizar las preferencias de los usuarios y facilitar el acceso a las publicaciones guardadas para futuras consultas dentro de la aplicación.



```java
package com.back.carfind.model;

import jakarta.persistence.Column;


@Entity
public class Favoritos {

@Id
@GeneratedValue
private Long id_usuario;
@Column
private Long id_publicacion;

public Long getId_usuario() {
    return id_usuario;
}
public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}
public Long getId_publicacion() {
    return id_publicacion;
}
public void setId_publicacion(Long id_publicacion) {
    this.id_publicacion = id_publicacion;
}



}
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una entidad dentro de la base de datos, mientras que **@Id** y **@GeneratedValue** indican que el atributo id_usuario funciona como identificador de la entidad y que su valor puede generarse automáticamente. La anotación **@Column** permite definir los campos que serán almacenados en la tabla, como la publicación asociada al usuario. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad Favoritos cumple una función importante, ya que sirve como base para registrar, consultar y administrar las publicaciones favoritas de los usuarios dentro de la aplicación.

HistorialPrecios.java:

La clase **HistorialPrecios** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con los cambios de precio de las publicaciones dentro del sistema, incluyendo el identificador del historial, la publicación asociada, el precio registrado y la fecha en la que se realizó el cambio. Esta estructura permite mantener un registro de las variaciones de precios, facilitando el seguimiento y control de las modificaciones realizadas sobre las publicaciones dentro de la aplicación.



```java
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class HistorialPrecios{

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_historial_precios;
@Column
private Long id_publicacion;
@Column
private Float precio;
@Column
private LocalDate fecha_precio;


public Long getId_historial_precios() {
    return id_historial_precios;
}
public void setId_historial_precios(Long id_historial_precios) {
    this.id_historial_precios = id_historial_precios;
}
public Long getId_publicacion() {
    return id_publicacion;
}
public void setId_publicacion(Long id_publicacion) {
    this.id_publicacion = id_publicacion;
}
public Float getPrecio() {
    return precio;
}
public void setPrecio(Float precio) {
    this.precio = precio;
}
public LocalDate getFecha_precio() {
    return fecha_precio;
}
public void setFecha_precio(LocalDate fecha_precio) {
    this.fecha_precio = fecha_precio;
}



}
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una tabla dentro de la base de datos, mientras que **@Id** y **@GeneratedValue** indican que el atributo id_historial_precios funciona como identificador principal y que su valor se genera automáticamente. Las anotaciones **@Column** permiten definir los campos que serán almacenados, como la publicación relacionada, el precio y la fecha del registro. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad HistorialPrecios cumple una función importante, ya que sirve como base para registrar, consultar y administrar el historial de cambios de precios realizados en las publicaciones dentro de la aplicación.

HistorialPublicaciones.java:

La clase **HistorialPublicaciones** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con el historial de

publicaciones realizadas dentro del sistema, incluyendo el identificador del usuario asociado, el estado de la publicación y la fecha en la que fue registrada. Esta estructura permite organizar y mantener un seguimiento de las publicaciones creadas por los usuarios, facilitando el control de los cambios y estados dentro de la aplicación.



```java
package com.back.carfind.model;
import java.time.LocalDate;

@Entity
public class HistorialPublicaciones {

@Column
private Long id_usuario;
@Column
private String estado;
@Column
private LocalDate fecha_publicacion;
public Long getId_usuario() {
    return id_usuario;
}
public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}
public String getEstado() {
    return estado;
}
public void setEstado(String estado) {
    this.estado = estado;
}
public LocalDate getFecha_publicacion() {
    return fecha_publicacion;
}
public void setFecha_publicacion(LocalDate fecha_publicacion) {
    this.fecha_publicacion = fecha_publicacion;
}



}
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una entidad dentro de la base de datos, mientras que las anotaciones **@Column** definen los atributos que serán almacenados en la tabla, como el usuario, el estado y la fecha de publicación. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad HistorialPublicaciones cumple una función importante, ya que sirve como base para registrar, consultar y administrar el historial de las publicaciones realizadas por los usuarios dentro de la aplicación.

HistorialVehiculo.java:

La clase **HistorialVehiculo** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con el historial de los vehículos registrados dentro del sistema, incluyendo el identificador del historial, el vehículo asociado, el tipo de historial, una descripción del evento registrado y la fecha en la que ocurrió. Esta estructura permite mantener un registro organizado sobre las modificaciones, mantenimientos u otros acontecimientos relacionados con cada vehículo dentro de la aplicación.



```java
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class HistorialVehiculo {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_historial;
@Column
private Long id_auto;
@Column
private String tipo_historial;
@Column
private String descripcion_historial;
@Column
private LocalDate fecha_historial;

public Long getId_historial() {
    return id_historial;
}
public void setId_historial(Long id_historial) {
    this.id_historial = id_historial;
}
public Long getId_auto() {
    return id_auto;
}
public void setId_auto(Long id_auto) {
    this.id_auto = id_auto;
}
public String getTipo_historial() {
    return tipo_historial;
}
public void setTipo_historial(String tipo_historial) {
    this.tipo_historial = tipo_historial;
}
public String getDescripcion_historial() {
    return descripcion_historial;
}
public void setDescripcion_historial(String descripcion_historial) {
    this.descripcion_historial = descripcion_historial;
}
public LocalDate getFecha_historial() {
    return fecha_historial;
}
public void setFecha_historial(LocalDate fecha_historial) {
```

El uso de la anotación **@Entity** permite que la clase sea reconocida como una tabla dentro de la base de datos, mientras que **@Id** y **@GeneratedValue** indican que el atributo id_historial funciona como identificador principal y que su valor se genera automáticamente. Las anotaciones **@Column** permiten definir los campos que serán almacenados, como el vehículo asociado, el

tipo de historial, la descripción y la fecha del registro. Además, los métodos **getters y setters** permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad HistorialVehiculo cumple una función importante, ya que sirve como base para registrar, consultar y administrar el historial de los vehículos dentro de la aplicación.

Informe_dominio.java:

La clase Informe_dominio pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con los informes de dominio de los vehículos registrados dentro del sistema, incluyendo el identificador del informe, el vehículo asociado, el dominio, los números de chasis y motor, los datos del titular actual, la cantidad de titulares, la información legal del vehículo y las posibles deudas o restricciones. Esta estructura permite organizar y consultar información importante sobre el estado legal y administrativo de un vehículo antes de realizar una operación dentro de la aplicación.



```java
package com.back.carfind.model;

import jakarta.persistence.Column;


@Entity
public class Informe_dominio {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_informe;
@Column
private Long auto;
@Column
private String dominio;
@Column
private Integer numero_chasis;
@Column
private Integer numero_motor;
@Column
private String titular_actual;
@Column
private Integer cantidad_titulares;
@Column
private LocalDate fecha_inscripcion;
@Column
private Boolean tiene_embargo;
@Column
private Boolean tiene_prenda;
@Column
private Boolean es_robado;
@Column
private Boolean titular_inhibido;
@Column
private Double deuda_patentes;
@Column
private Double deuda_multas;
@Column
private String observaciones;
@Column
private LocalDate fecha_emision;
@Column
private LocalDate fecha_vencimiento;
public Long getId_informe() {
    return id_informe;
```

El uso de la anotación @Entity permite que la clase sea reconocida como una tabla dentro de la base de datos, mientras que @Id y @GeneratedValue indican que el atributo id_informe funciona como identificador principal y que su valor se genera automáticamente. Las anotaciones @Column permiten definir los campos que serán almacenados, como los datos del vehículo, información del titular, deudas, observaciones y fechas del informe. Además, los métodos getters y setters permiten acceder y modificar la información de manera controlada desde otras partes del sistema. De esta forma, la entidad Informe_dominio cumple una función importante, ya que sirve como base para registrar, consultar y administrar los informes legales de los vehículos dentro de la aplicación.

Inventario.java:

La clase **Inventario** pertenece a la capa modelo y representa una entidad persistente dentro de la base de datos. Su objetivo es almacenar la información relacionada con el historial de los vehículos registrados dentro del sistema, incluyendo el identificador del historial, el vehículo asociado, el tipo de historial, una descripción del evento registrado y la fecha en la que ocurrió. Esta estructura permite mantener un registro organizado sobre las modificaciones, mantenimientos u otros acontecimientos relacionados con cada vehículo dentro de la aplicación

.

La anotación `@Entity` indica que la clase será reconocida por JPA como una entidad y que sus atributos podrán ser relacionados con una tabla de la base de datos. La anotación `@Id` establece que el atributo `publicacion_id`, de tipo `Long`, funciona como identificador principal de cada registro. Este campo permite relacionar el inventario con una publicación específica.

Inventario.java:

La clase Inventario, ubicada en el paquete com.back.carfind.model, representa la información relacionada con el stock disponible de una publicación dentro del sistema CarFind.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
@Entity
public class Inventario {

@Id
private Long publicacion_id;

@Column
private Integer stock;

public Long getPublicacion_id() {
    return publicacion_id;
}

public void setPublicacion_id(Long publicacion_id) {
    this.publicacion_id = publicacion_id;
}

public Integer getStock() {
    return stock;
}

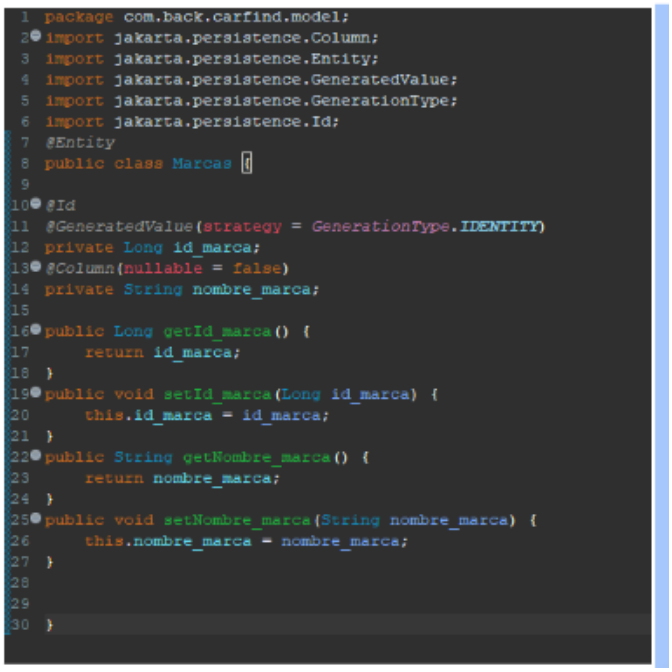
public void setStock(Integer stock) {
    this.stock = stock;
}

}
```

La anotación @Entity indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo publicacion_id, de tipo Long, está marcado con @Id, por lo que funciona como identificador principal del registro y permite asociar el inventario con una publicación específica.

Marcas.java:

La clase Marcas, ubicada en el paquete com.back.carfind.model, representa las marcas de vehículos que pueden registrarse y utilizarse dentro del sistema CarFind.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
@Entity
public class Marcas {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_marca;
@Column(nullable = false)
private String nombre_marca;

public Long getId_marca() {
    return id_marca;
}
public void setId_marca(Long id_marca) {
    this.id_marca = id_marca;
}
public String getNombre_marca() {
    return nombre_marca;
}
public void setNombre_marca(String nombre_marca) {
    this.nombre_marca = nombre_marca;
}


}
```

La anotación @Entity indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo id_marca, de tipo Long, está marcado con @Id, por lo que funciona como identificador principal de cada marca. Además, la anotación @GeneratedValue(strategy = GenerationType.IDENTITY) permite que su valor sea generado automáticamente por la base de datos al insertar un nuevo registro.

Mensajes.java:

La clase Mensajes, ubicada en el paquete com.back.carfind.model, representa los mensajes enviados entre los usuarios dentro de la plataforma CarFind. Su función principal es almacenar la información necesaria para registrar y controlar las conversaciones relacionadas con la compra y venta de vehículos.



```java
package com.back.carfind.model;
import java.time.LocalDate;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
@Entity
public class Mensajes {

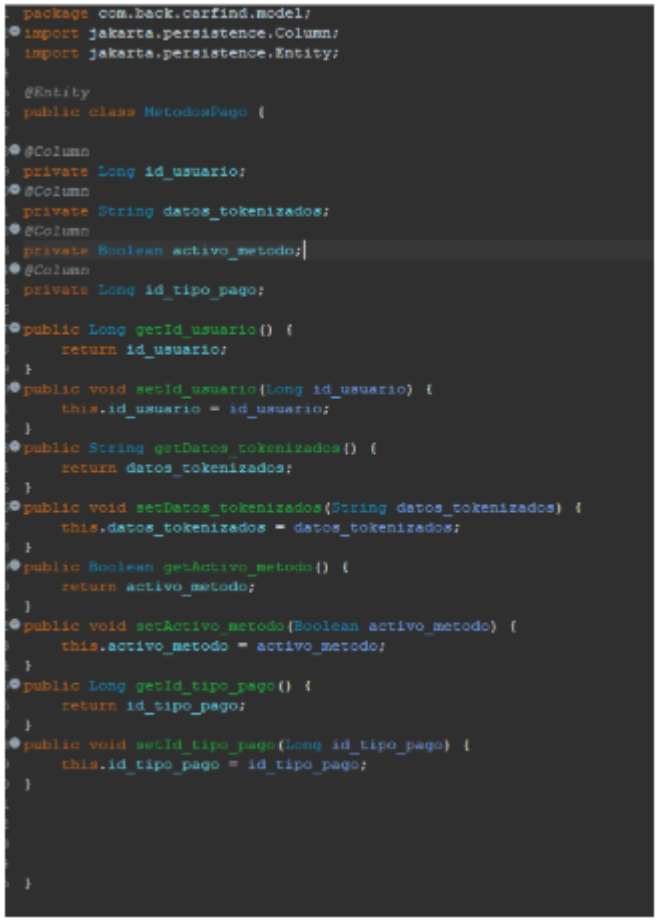
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_mensaje;
@Column
private Long id_chat;
@Column
private Long id_emisor;
@Column
private String contenido;
@Column
private LocalDate fecha;
@Column
private Boolean leido;
public Long getId_mensaje() {
    return id_mensaje;
}
public void setId_mensaje(Long id_mensaje) {
    this.id_mensaje = id_mensaje;
}
public Long getId_chat() {
    return id_chat;
}
public void setId_chat(Long id_chat) {
    this.id_chat = id_chat;
}
public Long getId_emisor() {
    return id_emisor;
}
public void setId_emisor(Long id_emisor) {
    this.id_emisor = id_emisor;
}
public String getContenido() {
    return contenido;
}
public void setContenido(String contenido) {
    this.contenido = contenido;
}
public LocalDate getFecha() {
    return fecha;
}
public void setFecha(LocalDate fecha) {
```

La anotación @Entity indica que la clase será administrada por JPA y estará vinculada con una tabla de la base de datos. El atributo id_mensaje, de tipo Long, está marcado con @Id, por lo que funciona como identificador principal de cada mensaje. Además, la anotación @GeneratedValue(strategy = GenerationType.IDENTITY) permite que este identificador sea generado automáticamente por la base de datos.

MetodosPago.java:

La clase `MetodosPago`, ubicada en el paquete `com.back.carfind.model`, representa los métodos de pago asociados a los usuarios dentro de la plataforma CarFind. Su función es

almacenar la información necesaria para identificar, administrar y activar o desactivar las formas de pago disponibles para realizar operaciones en el sistema.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;

@Entity
public class MetodosPago {

@Column
private Long id_usuario;
@Column
private String datos_tokenizados;
@Column
private Boolean activo_metodo;
@Column
private Long id_tipo_pago;

public Long getId_usuario() {
    return id_usuario;
}
public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}
public String getDatos_tokenizados() {
    return datos_tokenizados;
}
public void setDatos_tokenizados(String datos_tokenizados) {
    this.datos_tokenizados = datos_tokenizados;
}
public Boolean getActivo_metodo() {
    return activo_metodo;
}
public void setActivo_metodo(Boolean activo_metodo) {
    this.activo_metodo = activo_metodo;
}
public Long getId_tipo_pago() {
    return id_tipo_pago;
}
public void setId_tipo_pago(Long id_tipo_pago) {
    this.id_tipo_pago = id_tipo_pago;
}



}
```

La anotación `@Entity` indica que la clase será administrada por JPA y relacionada con una tabla de la base de datos. Los atributos están acompañados por la anotación `@Column`, lo que permite establecer que sus valores serán almacenados en las columnas correspondientes.

Modelos.java:

La clase `Modelos`, ubicada en el paquete `com.back.carfind.model`, representa los modelos de vehículos disponibles dentro de la plataforma CarFind. Su función principal es almacenar el nombre de cada modelo y relacionarlo con la marca correspondiente.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;
@Entity
public class Modelos {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_modelo;
@Column(nullable = false)
private String nombre_modelo;
@Column
private Long marca_id;

public Long getId_modelo() {
    return id_modelo;
}
public void setId_modelo(Long id_modelo) {
    this.id_modelo = id_modelo;
}
public String getNombre_modelo() {
    return nombre_modelo;
}
public void setNombre_modelo(String nombre_modelo) {
    this.nombre_modelo = nombre_modelo;
}
public Long getMarca_id() {
    return marca_id;
}
public void setMarca_id(Long marca_id) {
    this.marca_id = marca_id;
}


}
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará vinculada con una tabla de la base de datos. El atributo `id_modelo`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal de cada modelo. Además, la anotación

`@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que este identificador sea generado automáticamente por la base de datos.

ModeracionMultimedia.java:

La clase ModeracionMultimedia, ubicada en el paquete com.back.carfind.model, representa el proceso de revisión y control del contenido multimedia asociado a las publicaciones de vehículos dentro de la plataforma CarFind.



```
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class ModeracionMultimedia {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_moderacion_multimedia;

@Column
private String estado;

@Column
private String motivo;

@Column
private LocalDate fecha;

public Long getId_moderacion_multimedia() {
    return id_moderacion_multimedia;
}

public void setId_moderacion_multimedia(Long id_moderacion_multimedia) {
    this.id_moderacion_multimedia = id_moderacion_multimedia;
}

public String getEstado() {
    return estado;
}

public void setEstado(String estado) {
    this.estado = estado;
}

public String getMotivo() {
    return motivo;
}

public void setMotivo(String motivo) {
    this.motivo = motivo;
}

public LocalDate getFecha() {
    return fecha;
}

public void setFecha(LocalDate fecha) {
    this.fecha = fecha;
}

}
```

La anotación @Entity indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo id_moderacion_multimedia, de tipo Long, está marcado con @Id, por lo que funciona como identificador principal de cada registro. Además, la anotación @GeneratedValue(strategy = GenerationType.IDENTITY) permite que este identificador sea generado automáticamente por la base de datos.

Movimientos.java:

La clase Movimientos, perteneciente al paquete com.back.carfind.model, representa los movimientos económicos u operaciones realizadas por los usuarios dentro de la plataforma CarFind. Su objetivo es registrar información relacionada con cada operación, como el tipo de movimiento, el monto, una descripción y la fecha en que fue realizada.



```java
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class Movimientos{

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_movimientos;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_usuario;

@Column
private String tipo;

@Column
private Float monto;

@Column
private String descripcion;

@Column
private LocalDate fecha;

public Long getId_movimientos() {
    return id_movimientos;
}

public void setId_movimientos(Long id_movimientos) {
    this.id_movimientos = id_movimientos;
}

public Long getId_usuario() {
    return id_usuario;
}

public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}

public String getTipo() {
    return tipo;
}

public void setTipo(String tipo) {
    this.tipo = tipo;
}

public Float getMonto() {
```

La anotación @Entity indica que la clase será administrada por JPA y asociada a una tabla de la base de datos. El atributo id_movimientos, de tipo Long, identifica de manera única cada movimiento. Al estar acompañado por @Id y @GeneratedValue(strategy = GenerationType.IDENTITY), su valor funciona como clave primaria y se genera automáticamente al insertar un nuevo registro.

Multimedia.java:

La clase Multimedia, ubicada en el paquete com.back.carfind.model, representa los archivos multimedia asociados a las publicaciones de vehículos dentro de la plataforma CarFind. Su

función es almacenar la información necesaria para identificar, organizar y mostrar imágenes, videos u otros recursos relacionados con una publicación.



```java
package com.back.carfind.model;

import jakarta.persistence.Column;


@Entity
public class Multimedia {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_multimedia;
@Column
private Long id_publicacion;
@Column
private String url;
@Column
private String tipo;
@Column
private Integer orden;
@Column
private LocalDate fecha_creacion;

public Long getId_multimedia() {
    return id_multimedia;
}
public void setId_multimedia(Long id_multimedia) {
    this.id_multimedia = id_multimedia;
}
public Long getId_publicacion() {
    return id_publicacion;
}
public void setId_publicacion(Long id_publicacion) {
    this.id_publicacion = id_publicacion;
}
public String getUrl() {
    return url;
}
public void setUrl(String url) {
    this.url = url;
}
public String getTipo() {
    return tipo;
}
public void setTipo(String tipo) {
    this.tipo = tipo;
}
public Integer getOrden() {
    return orden;
}
public void setOrden(Integer orden) {
    this.orden = orden;
}
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará vinculada con una tabla de la base de datos. El atributo `id_multimedia`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal de cada archivo multimedia. La anotación `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que este identificador sea generado automáticamente por la base de datos.

Notificaciones.java:

La clase Notificaciones, ubicada en el paquete com.back.carfind.model, representa los avisos o mensajes automáticos que el sistema CarFind envía a los usuarios. Su función principal es

registrar información relevante sobre distintos eventos de la plataforma, como cambios en una publicación, mensajes recibidos, actualizaciones de operaciones o estados de una gestión



```java
package com.back.carfind.model;
import java.time.LocalDate;

@Entity
public class Notificaciones {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_notificacion;
@Column(nullable = false)
private Long usuario_id;
@Column
private String tipo;
@Column
private String mensaje;
@Column
private Boolean leido;
@Column
private Long id_entidad;
@Column
private LocalDate fecha_notificacion;

public Long getId_notificacion() {
    return id_notificacion;
}
public void setId_notificacion(Long id_notificacion) {
    this.id_notificacion = id_notificacion;
}
public Long getUsuario_id() {
    return usuario_id;
}
public void setUsuario_id(Long usuario_id) {
    this.usuario_id = usuario_id;
}
public String getTipo() {
    return tipo;
}
public void setTipo(String tipo) {
    this.tipo = tipo;
}
public String getMensaje() {
    return mensaje;
}
public void setMensaje(String mensaje) {
    this.mensaje = mensaje;
}
public Boolean getLeido() {
    return leido;
}
public void setLeido(Boolean leido) {
    this.leido = leido;
}
```

La anotación @Entity indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo id_notificacion, de tipo Long, está marcado con @Id, por lo que funciona como identificador principal de cada notificación. Además, @GeneratedValue(strategy = GenerationType.IDENTITY) permite que este valor sea generado automáticamente por la base de datos.

Ofertas.java:

La clase Ofertas, ubicada en el paquete com.back.carfind.model, representa las propuestas económicas realizadas por los compradores sobre las publicaciones de vehículos dentro de la plataforma CarFind. Su función es registrar y administrar las ofertas realizadas durante el proceso de compra y venta.



```java
package com.back.carfind.model;
import java.time.LocalDate;

@Entity
public class Ofertas{

@Column
private Long id_publicacion;
@Column
private Long id_comprador;
@Column
private Float monto_reporte;
@Column
private String estado;
@Column
private LocalDate fecha_oferta;

public Long getId_publicacion() {
    return id_publicacion;
}
public void setId_publicacion(Long id_publicacion) {
    this.id_publicacion = id_publicacion;
}
public Long getId_comprador() {
    return id_comprador;
}
public void setId_comprador(Long id_comprador) {
    this.id_comprador = id_comprador;
}
public Float getMonto_reporte() {
    return monto_reporte;
}
public void setMonto_reporte(Float monto_reporte) {
    this.monto_reporte = monto_reporte;
}
public String getEstado() {
    return estado;
}
public void setEstado(String estado) {
    this.estado = estado;
}
public LocalDate getFecha_oferta() {
    return fecha_oferta;
}
public void setFecha_oferta(LocalDate fecha_oferta) {
    this.fecha_oferta = fecha_oferta;
}
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. Los atributos están acompañados por `@Column`, lo que indica que sus valores serán almacenados en las columnas correspondientes.

Pagos.java:

La clase `Pagos`, ubicada en el paquete `com.back.carfind.model`, representa los pagos realizados dentro de la plataforma CarFind. Su función principal es registrar y administrar la información relacionada con las operaciones de pago asociadas a una compra.



```java
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class Pagos {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_pago;
@Column
private Long id_compra;
@Column
private Long tipo_pago_id;
@Column
private String estado;
@Column
private Integer numero_transaccion;
@Column
private Long id_externo;
@Column
private LocalDate fecha_pago;
public Long getId_pago() {
    return id_pago;
}
public void setId_pago(Long id_pago) {
    this.id_pago = id_pago;
}
public Long getId_compra() {
    return id_compra;
}
public void setId_compra(Long id_compra) {
    this.id_compra = id_compra;
}
public Long getTipo_pago_id() {
    return tipo_pago_id;
}
public void setTipo_pago_id(Long tipo_pago_id) {
    this.tipo_pago_id = tipo_pago_id;
}
public String getEstado() {
    return estado;
}
public void setEstado(String estado) {
    this.estado = estado;
}
public Integer getNumero_transaccion() {
    return numero_transaccion;
}
public void setNumero_transaccion(Integer numero_transaccion) {
    this.numero_transaccion = numero_transaccion;
}
public Long getId_externo() {
    return id_externo;
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `id_pago`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal de cada pago. Además, la anotación `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que este valor sea generado automáticamente por la base de datos.

PagosCuotas.java:

La clase PagosCuotas, ubicada en el paquete com.back.carfind.model, representa la información relacionada con los pagos realizados en cuotas dentro de la plataforma CarFind. Su función es almacenar los datos del plan de financiación, los importes involucrados y las condiciones aplicadas a una compra abonada en varias cuotas.



La anotación @Entity indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo id_pagos_cuotas, de tipo Long, está marcado con @Id, por lo que funciona como identificador principal del registro. Además, la anotación @GeneratedValue(strategy = GenerationType.IDENTITY) permite que este valor se genere automáticamente.

Preguntas.java:

La clase Preguntas, ubicada en el paquete com.back.carfind.model, representa las consultas realizadas por los usuarios sobre las publicaciones de vehículos dentro de la plataforma CarFind. Su función principal es registrar las preguntas y asociarlas con la publicación y el usuario correspondiente.



```java
package com.back.carfind.model;

import jakarta.persistence.Column;


@Entity
public class Preguntas {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_pregunta;
@Column
private Long id_publicacion;
@Column
private Long id_usuario;
@Column
private String descripcion;
@Column
private LocalDate fecha_pregunta;

public Long getId_pregunta() {
    return id_pregunta;
}
public void setId_pregunta(Long id_pregunta) {
    this.id_pregunta = id_pregunta;
}
public Long getId_publicacion() {
    return id_publicacion;
}
public void setId_publicacion(Long id_publicacion) {
    this.id_publicacion = id_publicacion;
}
public Long getId_usuario() {
    return id_usuario;
}
public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}
public String getDescripcion() {
    return descripcion;
}
public void setDescripcion(String descripcion) {
    this.descripcion = descripcion;
}
public LocalDate getFecha_pregunta() {
    return fecha_pregunta;
}
public void setFecha_pregunta(LocalDate fecha_pregunta) {
    this.fecha_pregunta = fecha_pregunta;
}
}
```

La anotación @Entity indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo id_pregunta, de tipo Long, está marcado con @Id, por lo que funciona como identificador principal de cada pregunta. Además, @GeneratedValue(strategy =

GenerationType.IDENTITY) permite que su valor se genere automáticamente al crear un nuevo registro.

Promociones.java:

La clase Promociones, ubicada en el paquete com.back.carfind.model, representa las promociones aplicadas a las publicaciones de vehículos dentro de la plataforma CarFind. Su función es relacionar una publicación con un tipo de promoción y controlar el período durante el cual se encuentra vigente.



```
package com.back.carfind.model;
import jakarta.persistence.Column;
@Entity
public class Promociones {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_publicacion;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_tipo_promocion;

@Column
private Integer fecha_inicio;

@Column
private Integer fehca_fin;

@Column
private String estado;

public Long getId_publicacion() {
    return id_publicacion;
}

public void setId_publicacion(Long id_publicacion) {
    this.id_publicacion = id_publicacion;
}

public Long getId_tipo_promocion() {
    return id_tipo_promocion;
}

public void setId_tipo_promocion(Long id_tipo_promocion) {
    this.id_tipo_promocion = id_tipo_promocion;
}

public Integer getFecha_inicio() {
    return fecha_inicio;
}

public void setFecha_inicio(Integer fecha_inicio) {
    this.fecha_inicio = fecha_inicio;
}

public Integer getFehca_fin() {
    return fehca_fin;
}

public void setFehca_fin(Integer fehca_fin) {
    this.fehca_fin = fehca_fin;
```

La anotación @Entity indica que la clase será administrada por JPA y estará vinculada con una tabla de la base de datos. Los atributos id_publicacion e id_tipo_promocion permiten identificar

la publicación y el tipo de promoción asociado. El atributo estado, de tipo String, indica la situación de la promoción, por ejemplo, activa, vencida o inactiva.

PromocionesBancarias.java:

La clase PromocionesBancarias, ubicada en el paquete com.back.carfind.model, representa las promociones de financiación ofrecidas por los bancos dentro de la plataforma CarFind. Su función principal es almacenar las condiciones de una promoción bancaria, como el tipo de tarjeta, la cantidad de cuotas y el interés aplicado.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;
@Entity
public class PromocionesBancarias{

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_promociones_bancarias;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_banco;

@Column
private String tarjeta;

@Column
private Integer cuotas;

@Column
private Float interes;

@Column
private Boolean activa;

public Long getId_promociones_bancarias() {
    return id_promociones_bancarias;
}

public void setId_promociones_bancarias(Long id_promociones_bancarias) {
    this.id_promociones_bancarias = id_promociones_bancarias;
}

public Long getId_banco() {
    return id_banco;
}

public void setId_banco(Long id_banco) {
    this.id_banco = id_banco;
}

public String getTarjeta() {
    return tarjeta;
}

public void setTarjeta(String tarjeta) {
    this.tarjeta = tarjeta;
}

public Integer getCuotas() {
    return cuotas;
```

La anotación @Entity indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo id_promociones_bancarias, de tipo Long, identifica cada promoción y está marcado con @Id, por lo que funciona como clave primaria. Además, @GeneratedValue(strategy = GenerationType.IDENTITY) permite que su valor sea generado automáticamente por la base de datos.

Publicaciones.java:

La clase `Publicaciones`, ubicada en el paquete `com.back.carfind.model`, representa los avisos de vehículos publicados dentro de la plataforma CarFind. Es uno de los modelos principales del sistema, ya que reúne la información necesaria para mostrar, administrar y controlar cada publicación.



La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `id_publicacion`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal de cada publicación. Además, `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que su valor sea generado automáticamente por la base de datos.

Reembolsos.java:

La clase `Reembolsos`, ubicada en el paquete `com.back.carfind.model`, representa las devoluciones de dinero asociadas a los pagos realizados dentro de la plataforma CarFind. Su función es registrar la información necesaria para identificar el pago involucrado, el monto devuelto, el motivo de la devolución y la fecha en que se realizó.



La anotación `@Entity` indica que la clase será administrada por JPA y estará vinculada con una tabla de la base de datos. El atributo `id_reembolsos`, de tipo `Long`, identifica cada reembolso y está marcado con `@Id`, por lo que funciona como clave primaria. Además, `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite generar automáticamente su valor.

Reportes.java:

La clase Reportes, ubicada en el paquete com.back.carfind.model, representa los reportes realizados por los usuarios sobre las publicaciones de vehículos dentro de la plataforma CarFind.

Su función es registrar posibles inconvenientes, incumplimientos o contenidos que deban ser revisados por los responsables del sistema.



La anotación @Entity indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. Los atributos están acompañados por @Column, lo que indica que sus valores serán almacenados en las columnas correspondientes.

Resenias.java:

La clase Resenias, ubicada en el paquete com.back.carfind.model, representa las reseñas o valoraciones realizadas por los usuarios después de una operación de compra y venta dentro de la plataforma CarFind. Su función es almacenar la opinión del comprador sobre el vendedor y permitir evaluar la experiencia realizada.

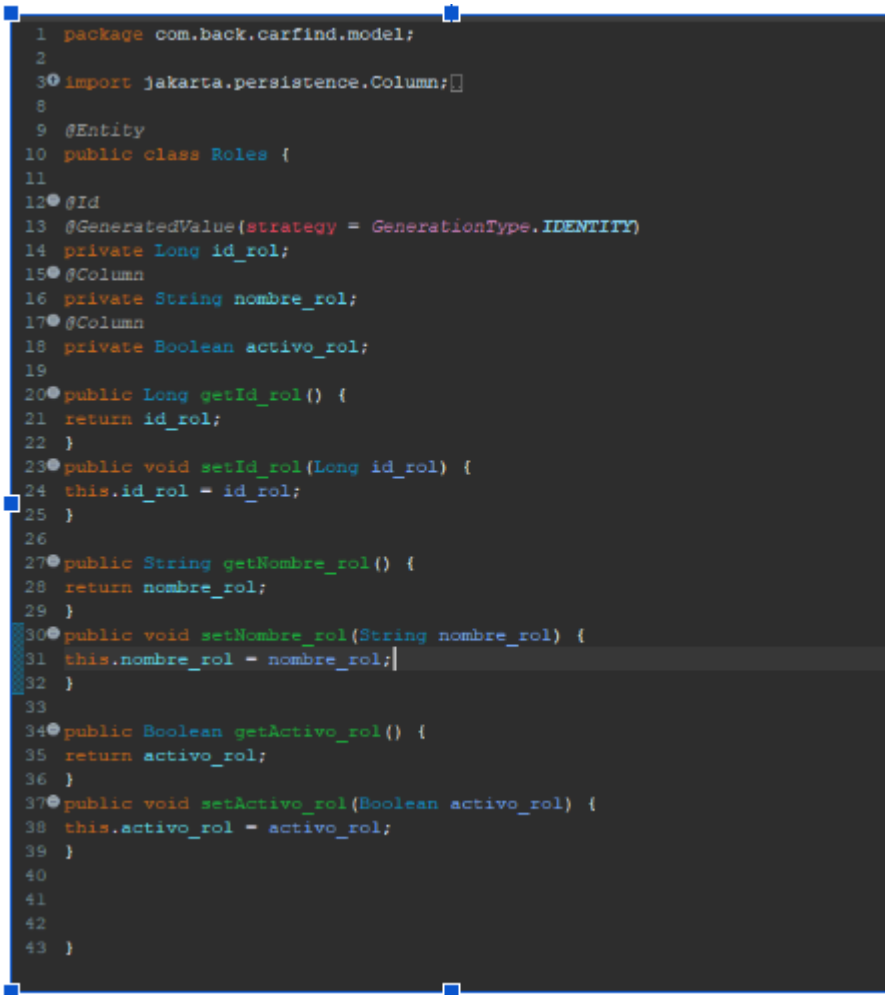


La anotación @Entity indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo id_resenia, de tipo Long, está marcado con @Id, por lo que funciona como identificador principal de cada reseña. Además, la anotación @GeneratedValue(strategy = GenerationType.IDENTITY) permite que este valor sea generado automáticamente por la base de datos.

Roles.java:

La clase `Roles`, ubicada en el paquete `com.back.carfind.model`, representa los diferentes roles o niveles de acceso que pueden tener los usuarios dentro de la plataforma

CarFind. Su función principal es organizar los permisos y determinar qué acciones puede realizar cada tipo de usuario en el sistema.



```java
package com.back.carfind.model;

import jakarta.persistence.Column;

@Entity
public class Roles {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_rol;
@Column
private String nombre_rol;
@Column
private Boolean activo_rol;

public Long getId_rol() {
return id_rol;
}
public void setId_rol(Long id_rol) {
this.id_rol = id_rol;
}

public String getNombre_rol() {
return nombre_rol;
}
public void setNombre_rol(String nombre_rol) {
this.nombre_rol = nombre_rol;
}

public Boolean getActivo_rol() {
return activo_rol;
}
public void setActivo_rol(Boolean activo_rol) {
this.activo_rol = activo_rol;
}



}
```

La anotación @Entity indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo id_rol, de tipo Long, está marcado con @Id, por lo que funciona como identificador principal de cada rol. Además, @GeneratedValue(strategy = GenerationType.IDENTITY) permite que este identificador sea generado automáticamente por la base de datos.

Sesiones.java:

La clase `Sesiones`, ubicada en el paquete `com.back.carfind.model`, representa las sesiones de acceso de los usuarios dentro de la plataforma CarFind. Su función principal es almacenar y controlar la información necesaria para mantener identificado a un usuario mientras utiliza el sistema.

```java
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class Sesiones {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_sesion;
@Column
private Long id_usuario;
@Column
private String token;
@Column
private LocalDate fecha_expiracion;

public Long getId_sesion() {
    return id_sesion;
}
public void setId_sesion(Long id_sesion) {
    this.id_sesion = id_sesion;
}
public Long getId_usuario() {
    return id_usuario;
}
public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}
public String getToken() {
    return token;
}
public void setToken(String token) {
    this.token = token;
}
public LocalDate getFecha_expiracion() {
    return fecha_expiracion;
}
public void setFecha_expiracion(LocalDate fecha_expiracion) {
    this.fecha_expiracion = fecha_expiracion;
}
}


```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `id_sesion`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal de cada sesión. Además, `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que este valor sea generado automáticamente por la base de datos.

TarjetasUsuario.java:

La clase `TarjetasUsuario`, ubicada en el paquete `com.back.carfind.model`, representa las tarjetas asociadas a los usuarios dentro de la plataforma CarFind. Su función principal es almacenar información identificativa y no sensible de las tarjetas para facilitar su gestión como medio de pago.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;
@Entity
public class TarjetasUsuario {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_tarjetas_usuario;

@Column
private String marca;

@Column
private String ultimos4;

@Column
private Long token;

@Column
private Integer vencimiento_mes;

@Column
private Integer vencmiento_anio;

public Long getId_tarjetas_usuario() {
    return id_tarjetas_usuario;
}

public void setId_tarjetas_usuario(Long id_tarjetas_usuario) {
    this.id_tarjetas_usuario = id_tarjetas_usuario;
}

public String getMarca() {
    return marca;
}

public void setMarca(String marca) {
    this.marca = marca;
}

public String getUltimos4() {
    return ultimos4;
}

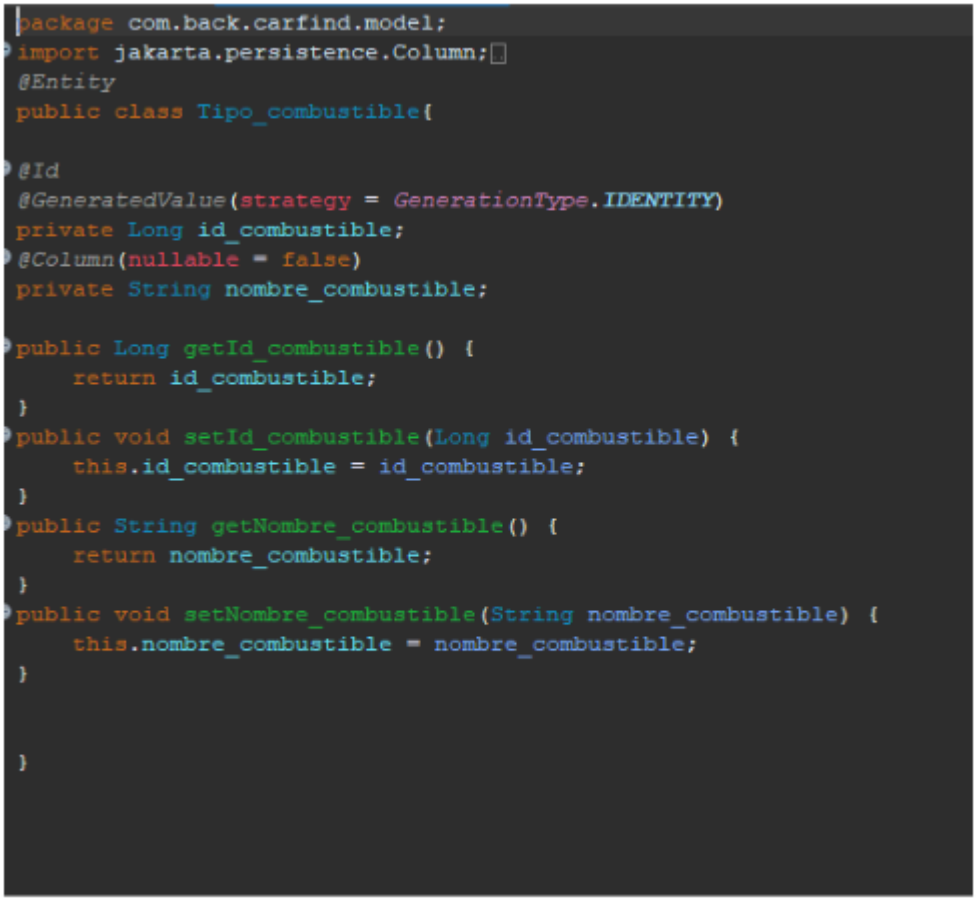
public void setUltimos4(String ultimos4) {
    this.ultimos4 = ultimos4;
}

public Long getToken() {
    return token;
}
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará vinculada con una tabla de la base de datos. El atributo `id_tarjetas_usuario`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal de cada tarjeta. Además, la anotación `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que este valor sea generado automáticamente.

Tipo_combustible.java:

La clase `Tipo_combustible`, ubicada en el paquete `com.back.carfind.model`, representa los distintos tipos de combustible disponibles para los vehículos registrados en la plataforma CarFind. Su función es organizar y almacenar las opciones de combustible que pueden asociarse a un automóvil.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;
@Entity
public class Tipo_combustible{

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_combustible;
@Column(nullable = false)
private String nombre_combustible;

public Long getId_combustible() {
    return id_combustible;
}
public void setId_combustible(Long id_combustible) {
    this.id_combustible = id_combustible;
}
public String getNombre_combustible() {
    return nombre_combustible;
}
public void setNombre_combustible(String nombre_combustible) {
    this.nombre_combustible = nombre_combustible;
}


}
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `id_combustible`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal de cada tipo de combustible. Además, `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que su valor sea generado automáticamente por la base de datos.

TipoPago.java:

La clase `TipoPago`, ubicada en el paquete `com.back.carfind.model`, representa los diferentes medios o tipos de pago disponibles dentro de la plataforma CarFind. Su función principal es organizar las opciones que pueden utilizar los usuarios para realizar operaciones de compra.

```java
package com.back.carfind.model;
import jakarta.persistence.Column;
@Entity
public class TipoPago {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_tipo_pago;
@Column
private String nombre_tipo_pago;


public Long getId_tipo_pago() {
    return id_tipo_pago;
}
public void setId_tipo_pago(Long id_tipo_pago) {
    this.id_tipo_pago = id_tipo_pago;
}
public String getNombre_tipo_pago() {
    return nombre_tipo_pago;
}
public void setNombre_tipo_pago(String nombre_tipo_pago) {
    this.nombre_tipo_pago = nombre_tipo_pago;
}



}
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `id_tipo_pago`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal de cada tipo de pago. Además, la anotación `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que su valor sea generado automáticamente por la base de datos.

TiposPromocion.java:

La clase `TiposPromocion`, ubicada en el paquete `com.back.carfind.model`, representa las distintas alternativas de promoción que pueden contratarse para destacar las publicaciones de vehículos dentro de la plataforma CarFind. Su objetivo es definir las características principales de cada promoción disponible.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;
@Entity
public class TiposPromocion {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id_tipos_promocion;

    @Column
    private String nombre;

    @Column
    private Integer prioridad;

    @Column
    private Integer duracion_dias;

    @Column
    private Float precio;

    public Long getId_tipos_promocion() {
        return id_tipos_promocion;
    }

    public void setId_tipos_promocion(Long id_tipos_promocion) {
        this.id_tipos_promocion = id_tipos_promocion;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public Integer getPrioridad() {
        return prioridad;
    }

    public void setPrioridad(Integer prioridad) {
        this.prioridad = prioridad;
    }

    public Integer getDuracion_dias() {
        return duracion_dias;
    }

    public void setDuracion_dias(Integer duracion_dias) {
        this.duracion_dias = duracion_dias;
    }
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `id_tipos_promocion`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal de cada tipo de promoción. La anotación `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que este identificador sea generado automáticamente.

Transmisiones.java:

La clase `Transmisiones`, ubicada en el paquete `com.back.carfind.model`, representa los diferentes tipos de transmisión disponibles para los vehículos registrados en la plataforma CarFind. Su función principal es organizar y almacenar las opciones de transmisión que pueden asociarse a un automóvil.

```java
package com.back.carfind.model;
import jakarta.persistence.Column;

@Entity
public class Transmisiones {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_transmision;
@Column(nullable = false)
private String nombre_transmision;

public Long getId_transmision() {
    return id_transmision;
}
public void setId_transmision(Long id_transmision) {
    this.id_transmision = id_transmision;
}
public String getNombre_transmision() {
    return nombre_transmision;
}
public void setNombre_transmision(String nombre_transmision) {
    this.nombre_transmision = nombre_transmision;
}


}
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `id_transmision`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal de cada tipo de transmisión. Además, `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que este identificador sea generado automáticamente por la base de datos.

Usuario.java:

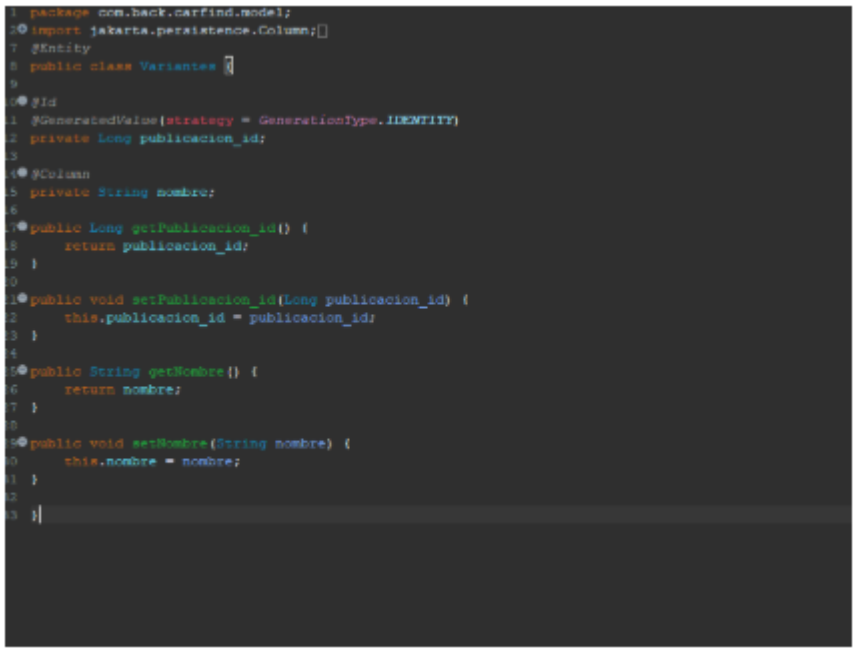
La clase `Usuario`, ubicada en el paquete `com.back.carfind.model`, representa a las personas que utilizan la plataforma CarFind. Es uno de los modelos principales del sistema, ya que almacena la información necesaria para identificar a cada usuario, gestionar su acceso y registrar su actividad dentro de la aplicación



La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `id`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal de cada usuario. Además, `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que su valor sea generado automáticamente por la base de datos.

Variantes.java:

La clase `Variantes`, ubicada en el paquete `com.back.carfind.model`, representa las diferentes variantes o versiones disponibles para los vehículos dentro de la plataforma CarFind. Su función es almacenar el nombre de cada variante y permitir identificarla dentro del sistema.



```java
package com.back.carfind.model;
import jakarta.persistence.Column;
@Entity
public class Variantes {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long publicacion_id;

@Column
private String nombre;

public Long getPublicacion_id() {
    return publicacion_id;
}

public void setPublicacion_id(Long publicacion_id) {
    this.publicacion_id = publicacion_id;
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

}
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `publicacion_id`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal del registro. Además, la anotación `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que su valor sea generado automáticamente por la base de datos.

VariantesOpciones.java:

La clase `VariantesOpciones`, ubicada en el paquete `com.back.carfind.model`, representa las opciones o valores específicos asociados a una variante de vehículo dentro de la plataforma CarFind. Su función es almacenar los detalles que permiten diferenciar una variante de otra, como el color, el equipamiento, el tipo de acabado u otra característica particular.

```java
package com.back.carfind.model;
import jakarta.persistence.Column;
@Entity
public class VariantesOpciones {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long variante_id;

public Long getVariante_id() {
    return variante_id;
}

public void setVariante_id(Long variante_id) {
    this.variante_id = variante_id;
}

public String getValor() {
    return valor;
}

public void setValor(String valor) {
    this.valor = valor;
}

@Column
private String valor;

}
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `variante_id`, de tipo `Long`, está marcado con `@Id`, por lo que funciona como identificador principal del registro. Además, la anotación

`@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que su valor sea generado automáticamente por la base de datos.

Verificaciones.java:

La clase `Verificaciones`, ubicada en el paquete `com.back.carfind.model`, representa los procesos de validación realizados sobre los usuarios dentro de la plataforma CarFind. Su función es registrar qué tipo de verificación se realizó, cuál es su estado y cuándo fue efectuada.

```java
package com.back.carfind.model;
import java.time.LocalDate;

@Entity
public class Verificaciones {

@Column
private Long id_usuario;
@Column
private String tipo_verificacion;
@Column
private String estado_verificacion;
@Column
private LocalDate fecha_verificacion;

public Long getId_usuario() {
    return id_usuario;
}
public void setId_usuario(Long id_usuario) {
    this.id_usuario = id_usuario;
}
public String getTipo_verificacion() {
    return tipo_verificacion;
}
public void setTipo_verificacion(String tipo_verificacion) {
    this.tipo_verificacion = tipo_verificacion;
}
public String getEstado_verificacion() {
    return estado_verificacion;
}
public void setEstado_verificacion(String estado_verificacion) {
    this.estado_verificacion = estado_verificacion;
}
public LocalDate getFecha_verificacion() {
    return fecha_verificacion;

public void setFecha_verificacion(LocalDate fecha_verificacion) {
    this.fecha_verificacion = fecha_verificacion;
}



}
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `id_usuario`, de tipo `Long`, permite asociar la

verificación con el usuario correspondiente. El campo `tipo_verificacion` almacena la clase de validación realizada, como la comprobación de identidad, documentación o datos personales.

VisitasAuto.java:

La clase `VisitasAuto`, ubicada en el paquete `com.back.carfind.model`, representa las visitas o encuentros coordinados entre compradores y vehículos publicados dentro de la plataforma CarFind. Su función es registrar cuándo se realizó una visita, qué vehículo fue consultado, qué comprador participó y cuál fue el resultado de la experiencia.



```java
package com.back.carfind.model;
import java.time.LocalDate;
@Entity
public class VisitasAuto{

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_visitas_auto;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_auto;

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id_comprador;

@Column
private LocalDate fecha;

@Column
private String resultado;

public Long getId_visitas_auto() {
    return id_visitas_auto;
}

public void setId_visitas_auto(Long id_visitas_auto) {
    this.id_visitas_auto = id_visitas_auto;
}

public Long getId_auto() {
    return id_auto;
}

public void setId_auto(Long id_auto) {
    this.id_auto = id_auto;
}

public Long getId_comprador() {
    return id_comprador;
}

public void setId_comprador(Long id_comprador) {
    this.id_comprador = id_comprador;
}

public LocalDate getFecha() {
    return fecha;
}

public void setFecha(LocalDate fecha) {
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `id_visitas_auto`, de tipo `Long`, identifica cada registro de visita y está marcado con `@Id`, por lo que funciona como clave primaria. La anotación `@GeneratedValue(strategy = GenerationType.IDENTITY)` permite que su valor sea generado automáticamente por la base de datos.

VistasPublicacion.java:

La clase `VistasPublicacion`, ubicada en el paquete `com.back.carfind.model`, representa los registros de visualización de las publicaciones dentro de la plataforma CarFind. Su función es almacenar qué usuario visitó una publicación y en qué fecha se produjo esa visualización.



```java
package com.back.carfind.model;
import java.time.LocalDate;

@Entity
public class VistasPublicacion {

    @Column
    private Long id_usuario;
    @Column
    private Long id_publicacion;
    @Column
    private LocalDate fecha_vista;
    public Long getId_usuario() {
        return id_usuario;
    }
    public void setId_usuario(Long id_usuario) {
        this.id_usuario = id_usuario;
    }
    public Long getId_publicacion() {
        return id_publicacion;
    }
    public void setId_publicacion(Long id_publicacion) {
        this.id_publicacion = id_publicacion;
    }
    public LocalDate getFecha_vista() {
        return fecha_vista;
    }
    public void setFecha_vista(LocalDate fecha_vista) {
        this.fecha_vista = fecha_vista;
    }



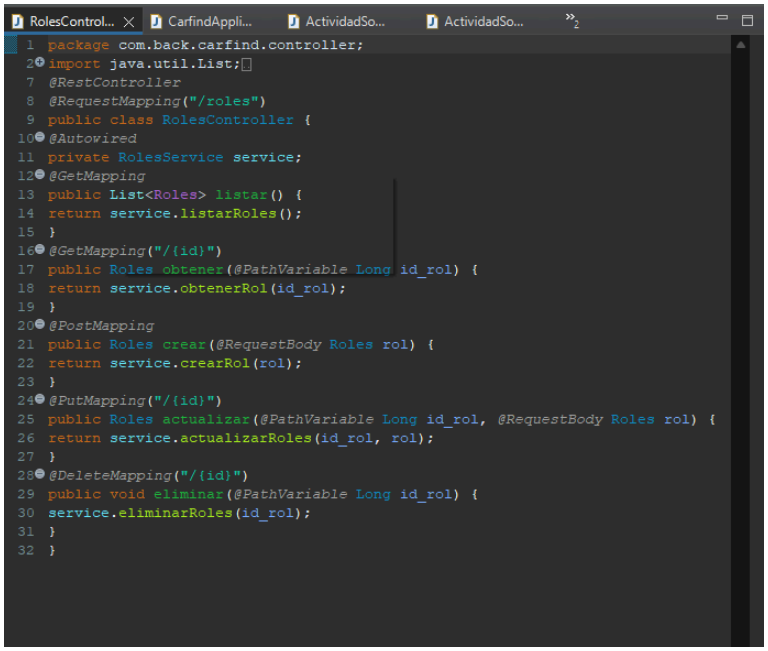
}
```

La anotación `@Entity` indica que la clase será administrada por JPA y estará relacionada con una tabla de la base de datos. El atributo `id_usuario`, de tipo `Long`, identifica al usuario que visualizó la publicación, mientras que `id_publicacion` permite identificar el aviso consultado.

Capa Controller

RolesController.java:

La clase RolesController pertenece a la capa controlador y se encarga de gestionar las solicitudes HTTP relacionadas con la entidad Roles. Su función es recibir las peticiones del cliente, redirigirlas al servicio correspondiente y devolver la respuesta obtenida. De esta manera, actúa como intermediario entre la interfaz de usuario y la lógica del sistema.



La anotación @RestController indica que la clase responde mediante servicios REST, mientras que @RequestMapping("/roles") establece la ruta base para acceder a sus operaciones. Dentro de la clase, el servicio RolesService se inyecta mediante @Autowired, lo que permite reutilizar la lógica de negocio sin instanciarla manualmente. Los métodos definidos permiten listar, obtener, crear, actualizar y eliminar roles mediante los verbos HTTP GET, POST, PUT y DELETE.

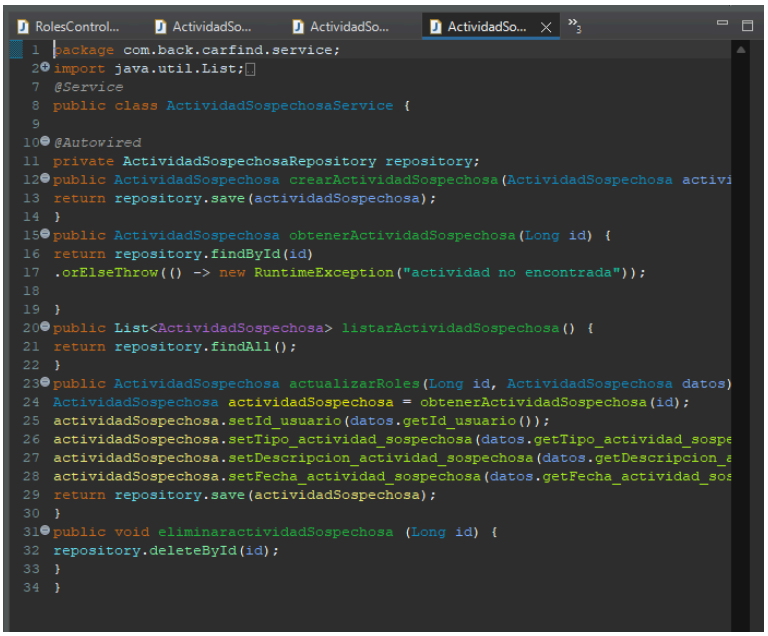
Esta capa resulta indispensable porque organiza la comunicación entre el usuario y el sistema, facilitando el acceso a las funcionalidades de manera estructurada. Además, contribuye a mantener una arquitectura limpia, ya que separa el manejo de peticiones de la lógica interna de la aplicación.

Capa Service

ActividadSospechosaService.java:

La clase ActividadSospechosaService pertenece a la capa de servicio y tiene como objetivo manejar la lógica de negocio asociada a la entidad ActividadSospechosa. Esta capa se ubica entre el controlador y el repositorio, cumpliendo un rol fundamental en la organización del sistema, ya que procesa la información antes de enviarla a la base de datos o devolverla a la capa superior.

CarFind_C



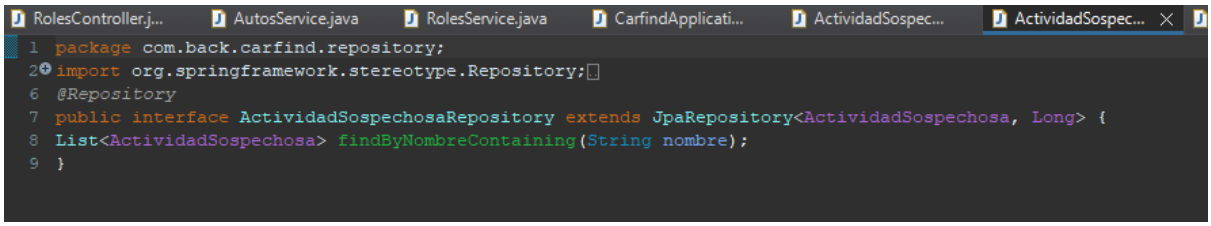
La anotación @Service permite que Spring reconozca esta clase como un componente del sistema. El repositorio se inyecta mediante @Autowired, facilitando la interacción con la base de datos. Los métodos definidos permiten crear, obtener, listar, actualizar y eliminar actividades sospechosas, asegurando así el correcto manejo de la información. Esta capa resulta indispensable porque centraliza la lógica operativa del sistema y contribuye a una arquitectura más clara, mantenible y escalable.

Capa Repository

ActividadSospechosaRepository.java:

La interfaz ActividadSospechosaRepository pertenece a la capa repository y se encarga del acceso directo a la base de datos. Al extender de JpaRepository, hereda automáticamente los

métodos básicos de persistencia y consulta ofrecidos por Spring Data JPA, lo que simplifica el desarrollo y evita la necesidad de implementar manualmente operaciones repetitivas.

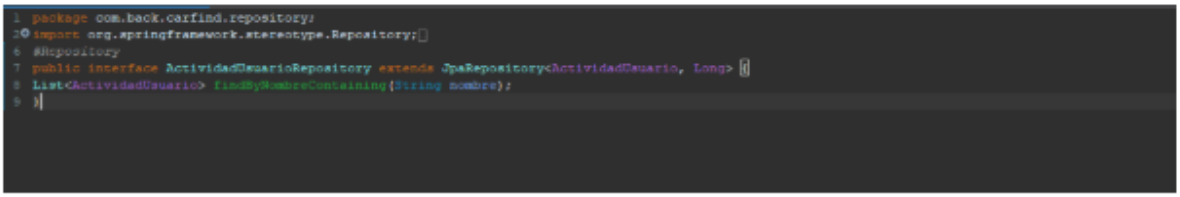


La anotación @Repository permite que Spring identifique esta interfaz como parte de la capa de acceso a datos. Además, el método findByNombreContaining define una consulta personalizada para realizar búsquedas parciales por nombre, aunque sería recomendable revisar su coherencia con los atributos reales de la entidad. Esta capa resulta esencial dentro de la arquitectura del sistema, ya que actúa como puente entre la aplicación y la base de datos, garantizando un manejo eficiente, organizado y reutilizable de la información.

ActividadUsuarioRepository.java:

La interfaz ActividadUsuarioRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad ActividadUsuario dentro de la base de datos.

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y que será administrada por Spring. Al extender JpaRepository<ActividadUsuario, Long>, el repositorio trabaja con objetos de tipo ActividadUsuario y utiliza identificadores de tipo Long.

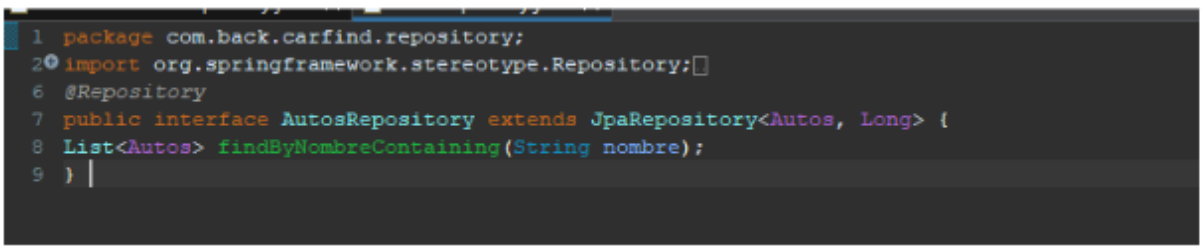


Gracias a JpaRepository, la interfaz obtiene automáticamente métodos para realizar operaciones básicas, como guardar, buscar, listar, actualizar y eliminar registros. Entre ellos se encuentran save(), findById(), findAll(), deleteById() y existsById().

AutosRepository.java:

La interfaz AutosRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Autos dentro de la base de datos de CarFind.

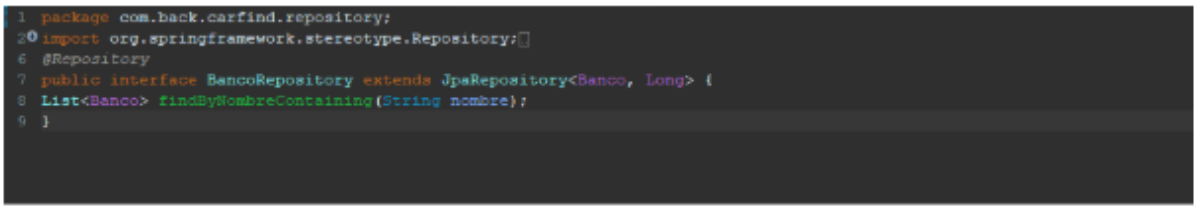
La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y que será administrada por Spring. Al extender JpaRepository<Autos, Long>, el repositorio trabaja con objetos de tipo Autos y utiliza identificadores de tipo Long.



Gracias a JpaRepository, la interfaz obtiene automáticamente métodos para realizar operaciones básicas, como guardar, buscar, listar, actualizar y eliminar vehículos. Algunos de estos métodos son save(), findById(), findAll(), deleteById() y existsById().

BancoRepository.java:

La interfaz BancoRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Banco dentro de la base de datos de CarFind.
La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y que será administrada por Spring. Al extender JpaRepository<Banco, Long>, el repositorio trabaja con objetos de tipo Banco y utiliza identificadores de tipo Long.



```
1 package com.back.carfind.repository;
2 import org.springframework.stereotype.Repository;
6 @Repository
7 public interface BancoRepository extends JpaRepository<Banco, Long> {
8 List<Banco> findByNombreContaining(String nombre);
9 }
```

Gracias a JpaRepository, se obtienen automáticamente métodos para realizar operaciones básicas sobre los bancos registrados, como guardar, buscar, listar, actualizar y eliminar información. Algunos ejemplos son save(), findById(), findAll(), deleteById() y existsById().

BilleterasRepository.java:

La interfaz BilleterasRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Billeteras dentro de la base de datos de CarFind.

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y que será administrada por Spring. Al extender JpaRepository<Billeteras, Long>, el repositorio trabaja con objetos de tipo Billeteras y utiliza identificadores de tipo Long.

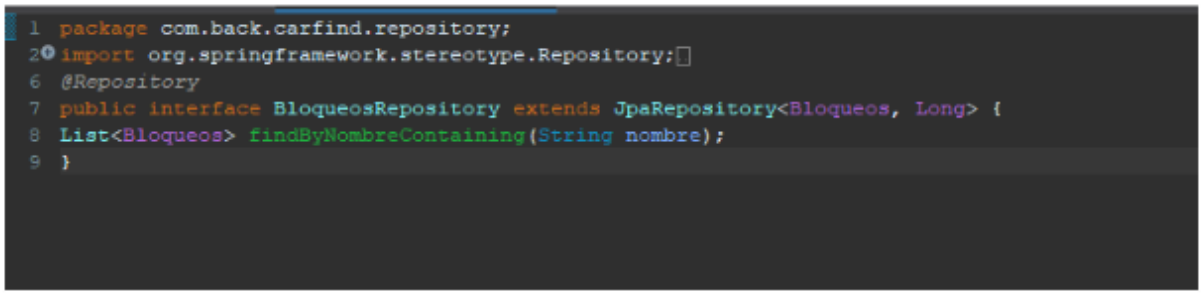


Gracias a JpaRepository, se obtienen automáticamente métodos para realizar operaciones básicas sobre las billeteras registradas, como guardar, buscar, listar, actualizar y eliminar información. Entre estos métodos se encuentran save(), findById(), findAll(), deleteById() y existsById().

BloqueosRepository.java:

La interfaz `BloqueosRepository`, ubicada en el paquete `com.back.carfind.repository`, se encarga de gestionar el acceso a los datos de la entidad `Bloqueos` dentro de la base de datos de CarFind.

La anotación `@Repository` indica que la interfaz pertenece a la capa de persistencia y que será administrada por Spring. Al extender `JpaRepository<Bloqueos, Long>`, el repositorio trabaja con objetos de tipo `Bloqueos` y utiliza identificadores de tipo `Long`.

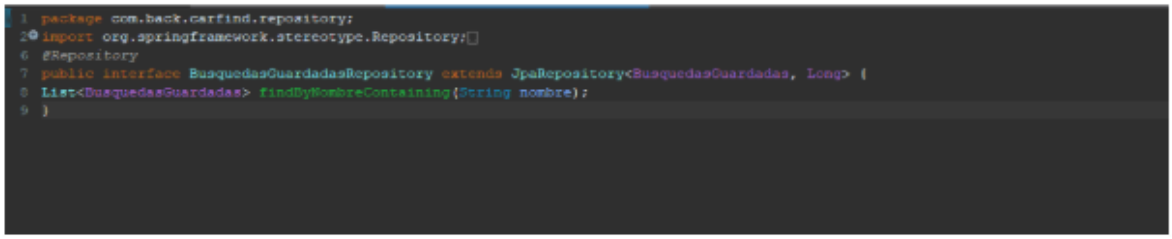


Gracias a JpaRepository, la interfaz obtiene automáticamente métodos para realizar operaciones básicas sobre los bloqueos registrados, como guardar, buscar, listar, actualizar y eliminar información. Algunos métodos disponibles son save(), findById(), findAll(), deleteById() y existsById().

BusquedasGuardadasRepository.java:

La interfaz BusquedasGuardadasRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad BusquedasGuardadas dentro de la base de datos de CarFind.

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y que será administrada por Spring. Al extender JpaRepository<BusquedasGuardadas, Long>, el repositorio trabaja con objetos de tipo BusquedasGuardadas y utiliza identificadores de tipo Long.

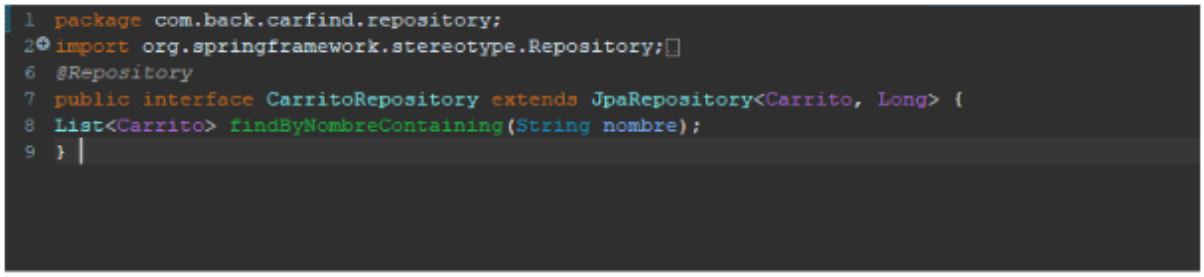


Gracias a JpaRepository, la interfaz obtiene automáticamente métodos para realizar operaciones básicas sobre las búsquedas guardadas, como guardar, consultar, listar, actualizar y eliminar registros. Entre ellos se encuentran save(), findById(), findAll(), deleteById() y existsById().

CarritoRepository.java:

La interfaz CarritoRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Carrito dentro de la base de datos de CarFind.

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y será administrada por Spring. Al extender JpaRepository<Carrito, Long>, el repositorio trabaja con objetos de tipo Carrito y utiliza identificadores de tipo Long.



Gracias a JpaRepository, se obtienen automáticamente métodos para realizar operaciones básicas sobre los carritos registrados, como guardar, buscar, listar, actualizar y eliminar datos. Entre estos métodos se encuentran save(), findById(), findAll(), deleteById() y existsById().

ChatRepository.java:

La interfaz ChatRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Chat dentro de la base de datos de CarFind.

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y será administrada por Spring. Al extender JpaRepository<Chat, Long>, el repositorio trabaja con objetos de tipo Chat y utiliza identificadores de tipo Long.



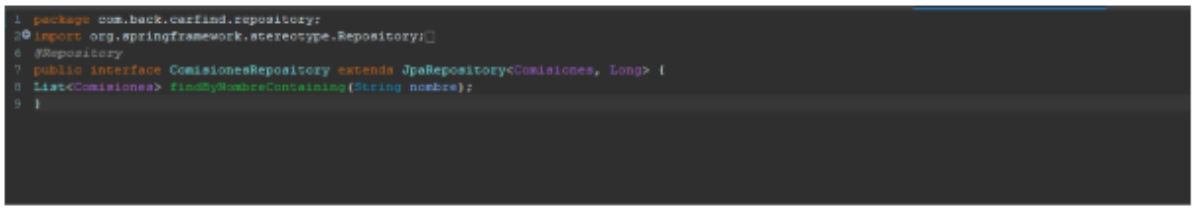
```java
package com.back.carfind.repository;
import org.springframework.stereotype.Repository;
@Repository
public interface ChatRepository extends JpaRepository<Chat, Long> {
List<Chat> findByNombreContaining(String nombre);
}
```

Gracias a JpaRepository, el repositorio obtiene automáticamente métodos para realizar operaciones básicas sobre los chats registrados, como guardar, buscar, listar, actualizar y eliminar información. Entre estos métodos se encuentran save(), findById(), findAll(), deleteById() y existsById().

ComisionesRepository.java:

La interfaz ComisionesRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Comisiones dentro de la base de datos de CarFind.

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y que será administrada por Spring. Al extender JpaRepository<Comisiones, Long>, el repositorio trabaja con objetos de tipo Comisiones y utiliza identificadores de tipo Long.



```java
package com.back.carfind.repository;
import org.springframework.stereotype.Repository;
@Repository
public interface ComisionesRepository extends JpaRepository<Comisiones, Long> {
List<Comisiones> findByNombreContaining(String nombre);
}
```

Gracias a JpaRepository, la interfaz obtiene automáticamente métodos para realizar operaciones básicas sobre las comisiones registradas, como guardar, buscar, listar, actualizar y eliminar información. Algunos de estos métodos son save(), findById(), findAll(), deleteById() y existsById().

ComprasRepository.java:

La interfaz ComprasRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Compras dentro de la base de datos de CarFind.

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y que será administrada por Spring. Al extender JpaRepository<Compras, Long>, el repositorio trabaja con objetos de tipo Compras y utiliza identificadores de tipo Long.

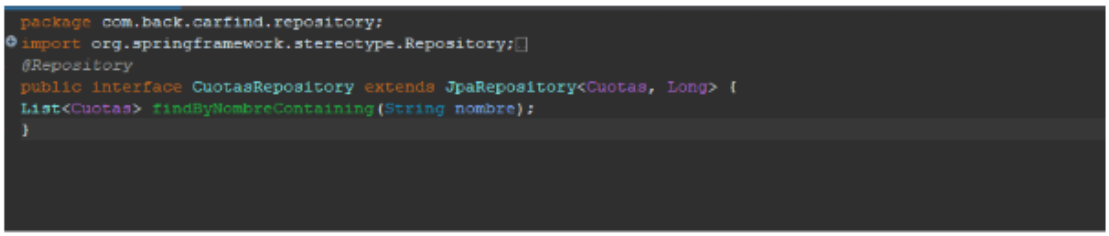


Gracias a JpaRepository, la interfaz obtiene automáticamente métodos para realizar operaciones básicas sobre las compras registradas, como guardar, buscar, listar, actualizar y eliminar información. Entre ellos se encuentran save(), findById(), findAll(), deleteById() y existsById().

CuotasRepository.java:

La interfaz CuotasRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Cuotas dentro de la base de datos de CarFind.

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y será administrada por Spring. Al extender JpaRepository<Cuotas, Long>, el repositorio trabaja con objetos de tipo Cuotas y utiliza identificadores de tipo Long.



Gracias a JpaRepository, la interfaz obtiene automáticamente métodos para realizar operaciones básicas sobre las cuotas registradas, como guardar, buscar, listar, actualizar y eliminar información. Entre ellos se encuentran save(), findById(), findAll(), deleteById() y existsById().

DireccionesRepository.java:

La interfaz DireccionesRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Direcciones dentro de la base de datos de CarFind.

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y será administrada por Spring. Al extender JpaRepository<Direcciones, Long>, el repositorio trabaja con objetos de tipo Direcciones y utiliza identificadores de tipo Long.



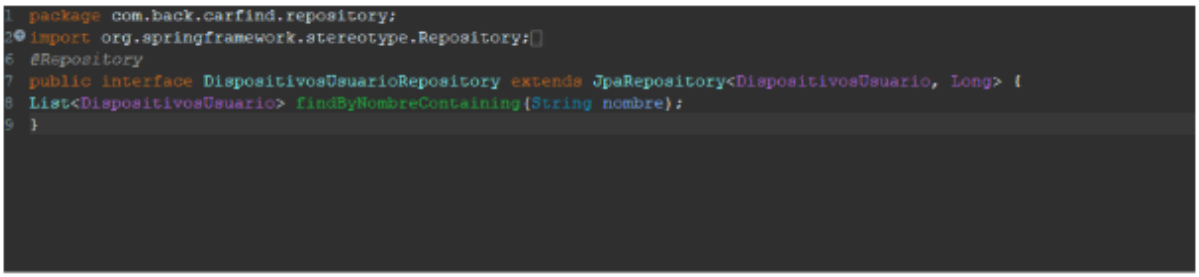
```java
package com.back.carfind.repository;
import org.springframework.stereotype.Repository;
@Repository
public interface DireccionesRepository extends JpaRepository<Direcciones, Long> {
List<Direcciones> findByNombreContaining(String nombre);
}
```

Gracias a JpaRepository, el repositorio obtiene automáticamente métodos para realizar operaciones básicas sobre las direcciones registradas, como guardar, buscar, listar, actualizar y eliminar información. Algunos métodos disponibles son save(), findById(), findAll(), deleteById() y existsById().

DispositivosUsuarioRepository.java:

La interfaz DispositivosUsuarioRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad DispositivosUsuario dentro de la base de datos de CarFind.

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y será administrada por Spring. Al extender JpaRepository<DispositivosUsuario, Long>, trabaja con objetos de tipo DispositivosUsuario y utiliza identificadores de tipo Long.



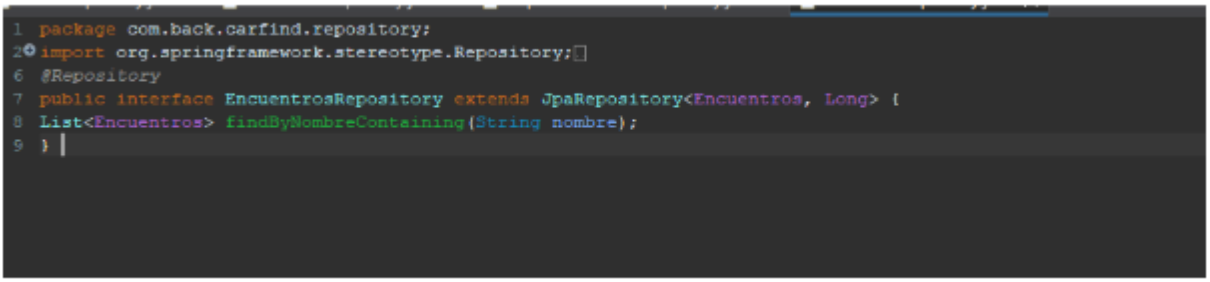
```java
package com.back.carfind.repository;
import org.springframework.stereotype.Repository;
@Repository
public interface DispositivosUsuarioRepository extends JpaRepository<DispositivosUsuario, Long> {
List<DispositivosUsuario> findByNombreContaining(String nombre);
}
```

Gracias a JpaRepository, se incorporan automáticamente métodos para guardar, buscar, listar, actualizar y eliminar dispositivos asociados a los usuarios, como save(), findById(), findAll(), deleteById() y existsById()

EncuentrosRepository.java:

La interfaz EncuentrosRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Encuentros dentro de la base de datos de CarFind.

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y será administrada por Spring. Al extender JpaRepository<Encuentros, Long>, trabaja con objetos de tipo Encuentros y utiliza identificadores de tipo Long.



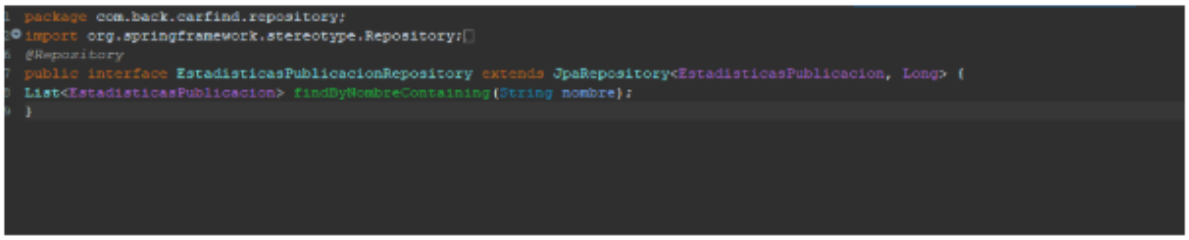
```java
package com.back.carfind.repository;
import org.springframework.stereotype.Repository;
@Repository
public interface EncuentrosRepository extends JpaRepository<Encuentros, Long> {
List<Encuentros> findByNombreContaining(String nombre);
}
```

Gracias a JpaRepository, se incorporan automáticamente métodos para guardar, buscar, listar, actualizar y eliminar encuentros, como save(), findById(), findAll(), deleteById() y existsById().

EstadisticasPublicacionRepository.java:

La interfaz EstadisticasPublicacionRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad EstadisticasPublicacion dentro de la base de datos de CarFind.

La anotación @Repository indica que esta interfaz pertenece a la capa de persistencia y será administrada por Spring. Al extender JpaRepository<EstadisticasPublicacion, Long>, trabaja con objetos de tipo EstadisticasPublicacion y utiliza identificadores de tipo Long.



Gracias a JpaRepository, se incorporan automáticamente métodos para guardar, buscar, listar, actualizar y eliminar estadísticas de publicaciones, como save(), findById(), findAll(), deleteById() y existsById().

FacturasRepository.java:

La interfaz FacturasRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Facturas dentro de la base de datos de CarFind.

La anotación @Repository indica que esta interfaz pertenece a la capa de persistencia y será administrada por Spring. Al extender JpaRepository<Facturas, Long>, trabaja con objetos de tipo Facturas y utiliza identificadores de tipo Long.



Gracias a JpaRepository, se incorporan automáticamente métodos para guardar, buscar, listar, actualizar y eliminar facturas, como save(), findById(), findAll(), deleteById() y existsById().

FavoritosRepository.java:

La interfaz FavoritosRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad Favoritos dentro de la base de datos de CarFind.

La anotación @Repository indica que esta interfaz pertenece a la capa de persistencia y será administrada por Spring. Al extender JpaRepository<Favoritos, Long>, trabaja con objetos de tipo Favoritos y utiliza identificadores de tipo Long.

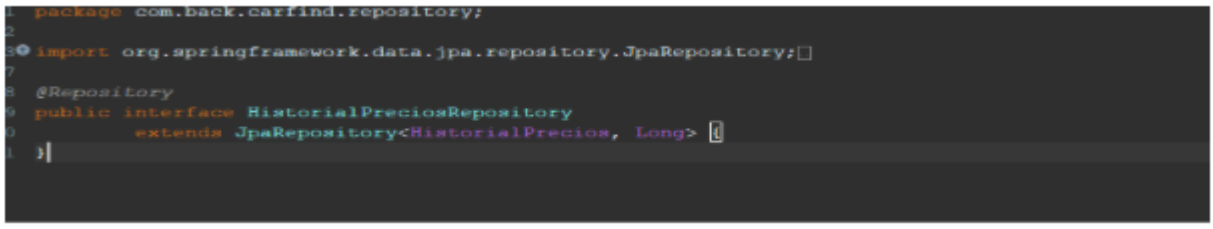


```java
package com.back.carfind.repository;
import org.springframework.stereotype.Repository;
@Repository
public interface FavoritosRepository extends JpaRepository<Favoritos, Long> {
List<Favoritos> findByNombreContaining(String nombre);
}
```

Gracias a JpaRepository, se incorporan automáticamente métodos para guardar, buscar, listar, actualizar y eliminar favoritos, como save(), findById(), findAll(), deleteById() y existsById().

HistorialPreciosRepository.java:

La interfaz HistorialPreciosRepository, ubicada en el paquete com.back.carfind.repository, permite gestionar los registros de la entidad HistorialPrecios en la base de datos de CarFind.



```java
package com.back.carfind.repository;

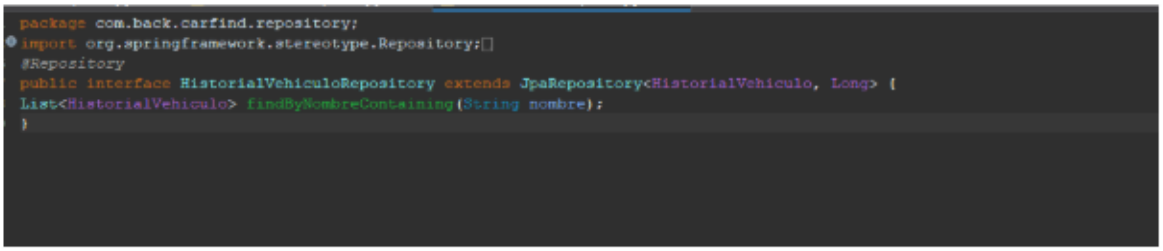
import org.springframework.data.jpa.repository.JpaRepository;

@Repository
public interface HistorialPreciosRepository
        extends JpaRepository<HistorialPrecios, Long> {
}
```

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y que será administrada automáticamente por Spring.

HistorialVehiculosRepository.java:

La interfaz HistorialVehiculoRepository, ubicada en el paquete com.back.carfind.repository, se encarga de gestionar el acceso a los datos de la entidad HistorialVehiculo dentro de la base de datos de CarFind.



```java
package com.back.carfind.repository;
import org.springframework.stereotype.Repository;
@Repository
public interface HistorialVehiculoRepository extends JpaRepository<HistorialVehiculo, Long> {
List<HistorialVehiculo> findByNombreContaining(String nombre);
}
```

La anotación @Repository indica que la interfaz pertenece a la capa de persistencia y será administrada por Spring. Al extender JpaRepository<HistorialVehiculo, Long>, trabaja con objetos de tipo HistorialVehiculo y utiliza identificadores de tipo Long.

Capa DTOs (Data Transfer Objects)

[Entidad]RequestDTO.java y [Entidad]ResponseDTO.java

Las clases orientadas a DTO (Data Transfer Objects) pertenecen a la capa de transferencia de datos y se encargan de transportar la información exclusivamente entre el cliente (por ejemplo, el frontend u otra API) y el servidor, desacoplando la estructura interna de las entidades de la base de datos de lo que se expone públicamente. Su función principal es definir contratos de datos específicos para las peticiones de entrada (*Request*) y las respuestas de salida (*Response*), evitando exponer atributos sensibles (como contraseñas, tokens o campos de auditoría interna) y previniendo problemas de referencias circulares o sobrecarga de datos (*over-posting* / *under-posting*).

La implementación de esta capa se estructura comúnmente separando las responsabilidades de entrada y salida:

- **RequestDTO (`[Entidad]RequestDTO`):** Contiene los campos estrictamente necesarios y las validaciones correspondientes (mediante anotaciones de Bean Validation como `@NotBlank`, `@NotNull`, `@Email`, `@Size`, etc.) para recibir los datos cuando el cliente realiza una operación de creación (`POST`) o actualización (`PUT`).
- **ResponseDTO (`[Entidad]ResponseDTO`):** Define el formato optimizado y limpio con el que el servidor devuelve los datos hacia la interfaz de usuario tras procesar una consulta (`GET`) o una transacción exitosa.

Ejemplo Práctico de Implementación en el Proyecto (`carfind`)

1. `AutoRequestDTO.java` (Capa de Entrada con Validaciones):

Java

```java
package com.back.carfind.dto;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;

public class AutoRequestDTO {

    @NotBlank(message = "La marca no puede estar en blanco")
    private String marca;

    @NotBlank(message = "El modelo no puede estar en blanco")
    private String modelo;
```

```
    @NotNull(message = "El año es obligatorio")
    @Min(value = 1950, message = "El año debe ser válido y posterior
a 1950")
    private Integer anio;

    @NotNull(message = "El precio es obligatorio")
    private Double precio;

    // Constructores, Getters y Setters
    public AutoRequestDTO() {}

    public String getMarca() { return marca; }
    public void setMarca(String marca) { this.marca = marca; }

    public String getModelo() { return modelo; }
    public void setModelo(String modelo) { this.modelo = modelo; }

    public Integer getAnio() { return anio; }
    public void setAnio(Integer anio) { this.anio = anio; }

    public Double getPrecio() { return precio; }
    public void setPrecio(Double precio) { this.precio = precio; }
}
```

2. `AutoResponseDTO.java` (Capa de Salida optimizada):

Java

```
package com.back.carfind.dto;

public class AutoResponseDTO {

    private Long id;
    private String marca;
```

```
    private String modelo;
    private Integer anio;
    private Double precio;
    private String estado; // Campo calculado o derivado que no viene
directamente en el request

    // Constructores, Getters y Setters
    public AutoResponseDTO() {}

    public AutoResponseDTO(Long id, String marca, String modelo,
Integer anio, Double precio, String estado) {
        this.id = id;
        this.marca = marca;
        this.modelo = modelo;
        this.anio = anio;
        this.precio = precio;
        this.estado = estado;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getMarca() { return marca; }
    public void setMarca(String marca) { this.marca = marca; }

    public String getModelo() { return modelo; }
    public void setModelo(String modelo) { this.modelo = modelo; }

    public Integer getAnio() { return anio; }
    public void setAnio(Integer anio) { this.anio = anio; }

    public Double getPrecio() { return precio; }
    public void setPrecio(Double precio) { this.precio = precio; }

    public String getEstado() { return estado; }
    public void setEstado(String estado) { this.estado = estado; }
}
```

Esta capa resulta indispensable dentro de la arquitectura limpia de Spring Boot porque aísla el modelo de persistencia (`@Entity`) de la API REST, permitiendo que la base de datos evolucione o cambie su esquema interno de manera independiente sin romper contratos ni alterar los datos que consumen los clientes del sistema.

Prueba de Compilación y Resolución de Errores

Finalizada la migración de las capas principales, se realizó una prueba de compilación para verificar la integridad del proyecto. Esta prueba se ejecutó utilizando el controlador de roles (RolesController) como caso de prueba inicial. Durante la compilación se identificó un error crítico relacionado con la estructura de paquetes del proyecto: los archivos de las distintas capas se encontraban fuera del paquete principal (carfind), lo que impedía que Spring Boot los detectará automáticamente mediante el mecanismo de escaneo de componentes.

La resolución de este error requirió mover todos los archivos al paquete correcto y actualizar las declaraciones de importación (import) en todas las clases afectadas, dado que el cambio de paquete invalida las referencias entre clases. Si bien este proceso demandó tiempo adicional, su resolución permitió comprender en profundidad el mecanismo de escaneo de componentes de Spring Boot y la importancia de definir correctamente la estructura de paquetes desde el inicio del proyecto.

Investigación sobre APIs de Marcas y Modelos de Vehículos

Durante el proceso de revisión del prototipado se identificó una problemática adicional: la necesidad de cargar en el sistema todas las marcas y modelos de vehículos existentes en el mercado. Dado que esto representaría una tarea de ingreso manual extremadamente extensa y propensa a errores, el equipo investigó la existencia de APIs públicas que provean esta información de forma estructurada y actualizada.

La investigación llevó a identificar varias APIs disponibles en el mercado que ofrecen catálogos completos de marcas y modelos de vehículos, incluyendo datos como el año de fabricación, el tipo de carrocería y las especificaciones técnicas. La integración de una de estas APIs en el sistema permitirá que el formulario de carga de publicaciones ofrezca al vendedor una lista dinámica y actualizada de marcas y modelos, garantizando la consistencia y precisión de los datos cargados sin requerir un mantenimiento manual del catálogo.

PROBLEMÁTICA DEL EQUIPO

El desarrollo de un proyecto de software en equipo implica no solo desafíos técnicos, sino también organizativos y comunicacionales. A lo largo del período de trabajo comprendido entre

marzo y julio de 2026, el equipo de CarFind atravesó distintas situaciones problemáticas que requirieron discusión, negociación y adaptación. La documentación de estas problemáticas forma parte del proceso reflexivo propio de la metodología Scrum y contribuye al aprendizaje del equipo para futuros proyectos.

Distribución Desigual de Tareas

Una de las principales problemáticas identificadas fue la distribución desigual de las responsabilidades de desarrollo entre los miembros del equipo, principalmente durante las etapas de construcción de la base de datos y el frontend. La base de datos fue completada en su totalidad para inicios de mayo, lo que generó un desequilibrio ya que varios miembros del equipo no pudieron participar activamente en su construcción. Esta situación se repitió con el frontend, que estuvo terminado para inicios de junio. La ausencia de participación de parte del equipo en estas etapas no solo generó disconformidades internas, sino que también limitó la distribución del conocimiento sobre el sistema, creando dependencias riesgosas en torno a los miembros que sí trabajaron en esas capas.

Como reflexión de esta experiencia, el equipo identificó la necesidad de planificar las tareas con mayor granularidad desde el inicio del proyecto, asignando responsabilidades específicas a cada miembro en cada sprint y estableciendo criterios claros de definición de hecho (Definition of Done) que aseguren la participación activa de todos en cada etapa del desarrollo.

Revisión Crítica de la Base de Datos

En el período comprendido entre mayo y junio, durante una reunión de equipo surgió una observación crítica respecto al diseño de la base de datos: algunos miembros identificaron la presencia de tablas redundantes o con información innecesaria para los requerimientos del sistema. Esta situación generó un intercambio de opiniones dentro del equipo que derivó en la decisión de revisar y depurar el modelo de datos. La revisión implicó eliminar tablas redundantes, fusionar entidades relacionadas y redefinir las relaciones entre tablas para garantizar un modelo más limpio, eficiente y alineado con los requerimientos reales del sistema.

Correcciones en el Prototipado

El 8 de julio de 2026 se realizó una revisión formal de las vistas del prototipado, en la cual se identificaron varios puntos de mejora que dieron lugar a correcciones concretas. En la vista de publicación se eliminó la opción de comprar directamente a través de la página, ya que esta funcionalidad no había sido acordada con el cliente durante la entrevista de relevamiento. Adicionalmente, se corrigió el tamaño de las publicaciones sugeridas en la grilla inferior y se reorganizó la galería de fotos de la publicación para mejorar la experiencia visual del usuario.

A la vista de subir la publicación se identificó y corrigió un error conceptual: el formulario presentaba 'marca' y 'modelo' como si fueran el mismo campo. Se realizó la separación correspondiente, dejando el campo de marca como único selector. Esta corrección derivó en la problemática descrita en el punto 6.5 respecto a la necesidad de una API de marcas y modelos, ya que cargar manualmente todos los modelos existentes en el mercado resultaría inviable.

Las vistas corregidas fueron incorporadas al documento y al prototipado digital, y los cambios fueron comunicados al equipo para asegurar que el desarrollo del frontend refleje las decisiones actualizadas de diseño.

Evaluación de alternativas

Antes de definir una solución analicé distintas posibilidades.

Una de ellas consistía en mantener la selección manual del estado del vehículo incorporando únicamente un mensaje de advertencia para recordar al usuario la importancia de completar correctamente la información.

También evalué la posibilidad de solicitar documentación adicional o algún tipo de certificado que acreditara que un vehículo realmente podía considerarse nuevo.

Sin embargo, luego de analizar ambas opciones comprendí que seguían dependiendo de la intervención directa del usuario y no eliminaban completamente el riesgo de errores o información incorrecta.

El 2 de septiembre del 2026 se realizó una reunión de organización en la que se definieron los roles vigentes del equipo y se clasificaron las problemáticas a resolver en tres niveles de prioridad. Como Scrum Master, usé esa clasificación como criterio para ordenar el trabajo:

- Prioridad máxima: que el primer sprint quedará terminado, con todas las tareas desde marzo hasta la fecha organizadas en Jira, y que se documentara el trabajo de Front-end y Back-end.

- Prioridad media: organizar el sprint y lograr un avance importante en el Back-end, completando los DTOs y, si era posible, la mitad de los controllers.
- Prioridad baja: definir la paleta de colores y avanzar en el Front-end.

Fundamenté este orden en que el Back-end condicionaba al resto del sistema. Mientras los DTOs, controllers y servicios no estuvieran completos, las vistas del Front-end seguirían siendo un prototipo sin datos reales, como se había comprobado en las pruebas de junio (punto 1.6). Además, el Backlog desordenado impedía saber qué estaba realmente terminado, así que ordenarlo era la condición para poder medir el avance.

Tabla 1. Distribución de roles y estado de participación del equipo

Integrante	Rol redefinido el 02/09/2026	Situación durante el tercer bimestre
Daiana Rivera	Scrum Master	Organización del Jira, planificación de sprints, seguimiento y control del equipo
Samuel Vera	Base de datos / Back-end	Asistencia irregular a clase; continuó trabajando de forma remota desde su casa
Mauricio Arzabe	Back-end	Trabajo intensivo en DTOs, controllers y servicios
Ignacio Córdoba	Apoyo en Scrum / Back-end	Trabajo en el Back-end junto a Mauricio

Leandro Delgado	Front-end	Desarrollo de las vistas
Kiara Hernández	Documentación Front-end / Front-end	Trabajo en conjunto con Leandro
Ludmila Rovner	Documentación Back-end	Dejó de asistir y no continuó con las tareas del proyecto

Reorganización del Backlog en Jira

El mayor desafío técnico de mi gestión fue la reorganización del Jira. Como se detalló en el punto 1.7, el tablero venía con errores de estructura desde las primeras etapas, con actividades individuales cargadas como Épicas. Aunque Ignacio había iniciado la reestructuración, el tablero seguía incompleto y la numeración de tareas era irregular. Mi trabajo consistió en ordenar la información desde marzo hasta la fecha, reconstruir el historial de tareas por gestión y cargar las nuevas tareas de los sprints a mi cargo.

Para ordenar este trabajo se elaboró un documento de separación por sprints que distingue las tareas según el Scrum Master que las gestionó. Su análisis muestra la evolución del volumen de planificación y es el respaldo de lo que detallo a continuación.

Tabla 2. Tareas del Backlog según la gestión de cada Scrum Master

Gestión	Numeración en Jira	Sprints	Tareas identificadas

Ludmila Rovner	18 a 59 (con la 32 sin registrar)	2	41
Ignacio Córdoba	60 a 139	4	80
Daiana Rivera	1 a 15, 140 a 192, 194 a 224	4 sprints y un bloque final	99

El conjunto del Backlog registra 220 tareas identificadas, por encima de la meta de aproximadamente doscientas que el equipo se había propuesto al adoptar la gestión por microtareas (punto 1.3, "Del cronograma tradicional a la gestión mediante microtareas"). Aclaro que esta cifra mide la cantidad de tareas cargadas y organizadas, no las completadas.

Planificación de los sprints de mi gestión

Organicé las 99 tareas de mi gestión en cinco bloques, cada uno con un objetivo propio. Al cargarlas, apliqué el criterio de tareas atómicas ya adoptado por el equipo: cada ticket describe una única acción verificable.

Seguimiento y control del trabajo del equipo

Mi función durante el bimestre fue de control. El 1 de septiembre asigné tareas a Ludmila y verifiqué el avance de Mauricio. El 2 de septiembre verifique que Leandro y Mauricio habían avanzado en el proyecto. El 8 de septiembre supervisé que cada integrante cumpliera las tareas asignadas y, en paralelo, diseñe las tareas del sprint siguiente. Para sostener este seguimiento

incluí en el Backlog la tarea de establecer las reuniones diarias y el registro de sus métricas (semana 1 a semana 4).

Fui estricta en el control del cumplimiento. Cuando un integrante no avanzaba con lo asignado, lo comunicaba al docente, que me indicó reasignar la tarea a otro compañero, el caso de Ludmila, reasigne todas sus tareas a Mauricio. Esta decisión se sostuvo en dos razones. La primera es que una tarea detenida bloquea las posteriores, de acuerdo con las dependencias del camino crítico. La segunda es que dejar tareas sin responsable activo contradecía el criterio de Scrum de que cada ticket tenga un dueño. Las reasignaciones fueron una consecuencia directa del nivel de participación de cada integrante, y quedaron registradas en el Jira.

La composición del equipo cambió durante el bimestre:

- Ludmila dejó de asistir y no continuó con el proyecto, por lo que sus tareas debieron redistribuirse.
- Samuel registró inasistencias, pero siguió trabajando desde su casa. Entendí que la modalidad remota era suficiente para mantener su participación, siempre que sus avances quedaran documentados.
- Kiara se incorporó a trabajar junto a Leandro en el Front-end.
- Ignacio y Mauricio concentraron el esfuerzo en el Back-end (DTOs, controllers y servicios).

Replanificación ante el plazo del 31 de octubre

El 29 de septiembre los profesores nos advirtieron que todo el proyecto debía estar terminado para el 31 de octubre, y el 30 de septiembre se confirmó que la situación era crítica. Este dato

modifica la planificación, porque el cronograma original (Gantt y camino crítico) había estimado plazos más amplios. Corresponde ahora recalcular qué tareas del Backlog son indispensables para el MVP y cuáles pasarán a ser mejoras futuras.

Estado actual y dificultades

Hasta la fecha de entrega de esta carpeta el 1 de octubre, el Jira está organizado a nivel de tareas, sprints y asignaciones, y lo único que resta es completar las Épicas. Considero esa carga el último paso para que el tablero refleje la estructura correcta de Scrum.

Durante el bimestre debí gestionar tres dificultades: la reorganización del tablero heredado, las ausencias y la salida de integrantes, y la reducción del plazo. Estas experiencias me permitieron comprender que el Scrum Master no desarrolla funcionalidades, sino que crea las condiciones para que el equipo pueda hacerlo.

Evaluación Técnica Final

Al finalizar esta primera etapa del proyecto CarFind (1er Cuatrimestre), considero que los objetivos principales se cumplieron satisfactoriamente, logrando transformar un relevamiento inicial en un prototipo funcional y solo me falta testear que la arquitectura de base de datos funcione a la perfección para así continuar. Como limitaciones, reconozco que la falta de conexión total entre el backend y el frontend impide que el sistema sea 100% operativo actualmente. A futuro, el proyecto podría ampliarse integrando muy bien la parte de pagos/transferencias y un sistema de validación de identidad con bases de datos gubernamentales. En lo personal, este proceso me permitió fortalecer mis capacidades de análisis funcional y

comprender la importancia de las reglas de negocio para garantizar la integridad de los datos en un sistema complejo

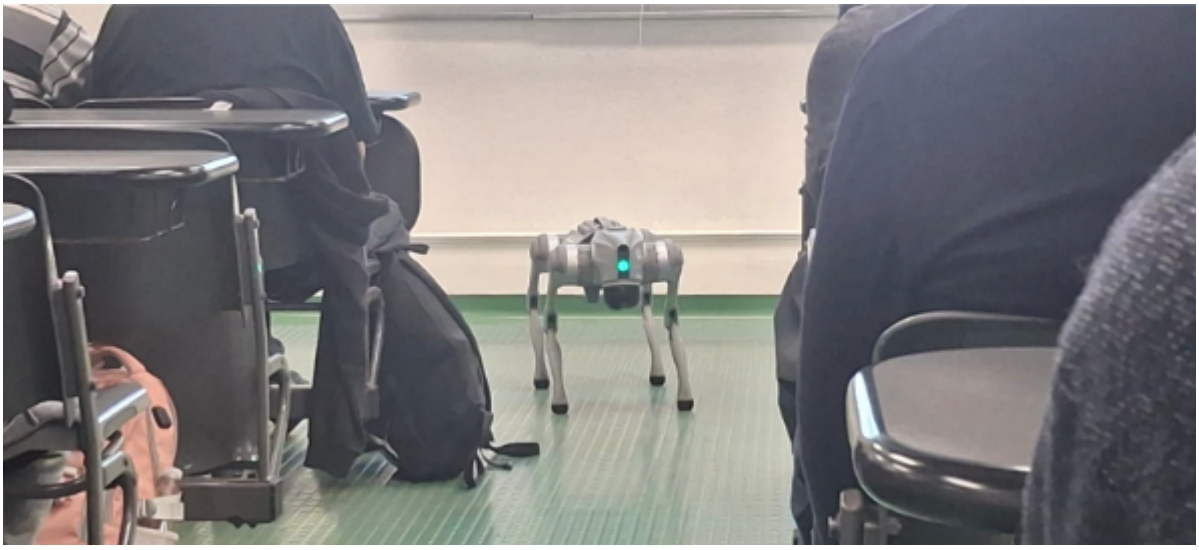
SALIDAS DIDÁCTICAS

En esta etapa se realizaron salidas didácticas con el objetivo de conocer distintos espacios de formación y observar de manera directa diferentes propuestas académicas y profesionales. Estas experiencias permitieron ampliar la visión sobre las alternativas que existen después de la escuela y reflexionar acerca de las opciones de estudio y trabajo disponibles.

UADE:

Asistí a una presentación académica y técnica realizada por representantes de la UADE, en la cual se explicó la propuesta educativa de su Facultad de Ingeniería y las distintas oportunidades de formación vinculadas al área de tecnología. Durante la charla se presentaron diversos trayectos educativos, desde tecnicaturas orientadas al desarrollo de software hasta carreras relacionadas con inteligencia artificial y ciencia de datos.

A lo largo de la presentación pude conocer las diferentes áreas en las que se puede desempeñar una persona dentro del mundo tecnológico, así como también las diferencias entre perfiles profesionales. Por un lado, se explicaron roles más técnicos y operativos, vinculados con la programación, el desarrollo y el mantenimiento de sistemas. Por otro lado, también se mencionaron roles de liderazgo, enfocados en la organización de proyectos, la toma de decisiones y la coordinación de equipos de trabajo.



Luego de la explicación sobre las carreras, se realizó una demostración práctica de robótica en la que pude observar el funcionamiento de distintos dispositivos de la marca Unitree, entre ellos un perro robot. Durante esta actividad, junto con mi equipo, tuvimos la oportunidad de interactuar con el robot y darle instrucciones para controlar sus movimientos. A partir de las indicaciones que organizamos entre todos, logramos que el perro robot realizará diferentes acciones, lo que permitió ver en la práctica cómo la programación se aplica directamente al control de una máquina.

Durante la demostración aprendí conceptos relacionados con lenguajes de programación como C y Python, el uso de sensores como LiDAR para detectar y analizar el ambiente, y la utilización de SDK para enviar instrucciones y controlar los movimientos de los robots. Esto permitió comprender cómo la programación, la inteligencia artificial y la robótica trabajan en conjunto para crear sistemas capaces de responder a diferentes órdenes y realizar tareas de manera autónoma

Para finalizar la actividad, realizamos una práctica en equipo donde aplicamos conceptos de algoritmos para crear secuencias de movimiento y estabilidad en el dispositivo. Esta experiencia me permitió relacionar los conocimientos teóricos de programación con una aplicación real, entendiendo mejor cómo la tecnología, la inteligencia artificial y la robótica forman parte de soluciones utilizadas actualmente en distintas áreas

escuela técnica N°11

En esta exposición de la Escuela Técnica N°11 pudimos conocer una variedad de propuestas relacionadas con diferentes áreas de estudio y con posibles salidas laborales a futuro. Durante la jornada se presentaron distintas universidades e instituciones que ofrecían carreras y capacitaciones de diversos tipos, como computación, informática, cocina y también áreas vinculadas con el ejército, entre otras opciones.

Además, la exposición estaba organizada por módulos y horarios. En cada franja había distintas charlas o actividades con temáticas variadas, y cada estudiante podía elegir a cuál asistir según sus intereses o curiosidad. Esto permitió conocer mejor las posibilidades que existen después de terminar la escuela y tener una idea más clara sobre los caminos que se pueden seguir.

Una de las charlas a las que asistí trataba sobre cómo elegir qué hacer en el futuro cuando todavía no se tiene una decisión definida. En esa actividad se destacó la importancia de reconocer nuestros gustos, habilidades e intereses para poder elegir una carrera o profesión que se adapte mejor a cada persona. Esta experiencia me ayudó a reflexionar sobre las distintas opciones disponibles y a pensar con más claridad en mi futuro académico y laboral

Salida didáctica a la Expo-Universidades

Fecha: 04/06/2026

Lugar: Cochabamba 2830, ET N° 11 "Manuel Belgrano".

En esta salida participé de la Expo-Universidades, un evento que reúne en un mismo espacio a numerosas instituciones educativas mediante stands informativos, pensado para que los estudiantes del último tramo del secundario podamos conocer de primera mano las distintas propuestas académicas antes de decidir qué carrera seguir. Entre los stands que recorrí se encontraban la Universidad Torcuato Di Tella, la UADE, la UAI (Universidad Abierta Interamericana), y también un instituto orientado específicamente a programas de viajes de aprendizaje de idiomas, además de otras casas de estudio que tenían presencia en la feria. La exposición de stands se desarrollaba en la planta baja del edificio, mientras que las charlas específicas se dictaban en las aulas del piso superior, cada una con su propia aula asignada. Fue una jornada larga, en la que aproveché el tiempo para pasar por varios stands, pero hubo dos actividades puntuales que marcaron especialmente mi experiencia: una charla de orientación vocacional a las 10:00 hs, y más tarde, a las 11:20 hs, una charla relacionada con programas de intercambio educativo en el extranjero.

Charla de orientación vocacional (10:00 hs)

La primera actividad a la que asistí fue una charla de orientación vocacional organizada por el Gobierno de la Ciudad de Buenos Aires, en el marco de su plataforma "Conocete", disponible en línea a través del sitio orientacionvocacional.bue.edu.ar. Me pareció una propuesta muy completa, porque no se limita únicamente a tomar un test, sino que plantea un recorrido en tres etapas pensado para acompañar a quien todavía no tiene decidido su futuro académico o laboral: primero invita a explorar los distintos campos ocupacionales a través de testimonios reales de profesionales y noticias relacionadas con cada área; luego, propone realizar un test de intereses vocacionales que arroja como resultado las áreas de mayor afinidad de la persona; y finalmente, ofrece información concreta sobre dónde estudiar cada carrera dentro de la Ciudad de Buenos Aires y el AMBA, cruzando los resultados obtenidos con la oferta educativa real disponible.

Me gustó muchísimo la forma en que estructuraron esta charla, que se dictó en una de las aulas del piso de arriba del edificio, ya que la feria en sí ocupaba únicamente la planta baja. Quienes la dieron fueron un hombre y una mujer, ambos con mucha soltura para hablarle a un grupo grande de estudiantes secundarios sin que la charla se volviera aburrida ni demasiado formal. Para empezar, en lugar de arrancar directamente con una explicación teórica sobre orientación vocacional, propusieron una dinámica grupal bastante divertida: un juego en el que, por turnos, cada uno de los presentes tenía que mencionar una profesión, un estudio, un posgrado o una diplomatura sin repetir ninguna de las que ya habían sido dichas por compañeros anteriores. Esto generó que, entre risas y algunos silencios incómodos cuando a alguien no se le ocurría nada nuevo, termináramos nombrando entre todos una cantidad enorme de carreras y formaciones que ni siquiera sabíamos que existían, desde profesiones más tradicionales hasta especializaciones bastante específicas y posgrados que la mayoría del grupo desconocía por completo. Esa dinámica me pareció una manera muy inteligente de romper el hielo y, al mismo tiempo, de

demostrarnos en la práctica lo amplio que es el abanico de opciones que existen más allá de las carreras "clásicas" que uno suele tener en mente al terminar el secundario.

Luego de romper el hielo con el juego anterior, los dos profesionales que dieron la charla explicaron cómo funcionaba el test de la plataforma Conocete y nos invitaron a completarlo ahí mismo desde nuestros celulares. En mi caso, realicé el test completo y los resultados que obtuve, ordenados según mi nivel de afinidad, fueron los siguientes: en primer lugar, Ciencias Ambientales y Agronomía / Agropecuaria; en segundo lugar, Arquitectura y Diseño; y en tercer lugar, Ciencias Exactas y Naturales. A diferencia de lo que podría suponerse, este resultado no me tomó por sorpresa: yo ya tenía bastante en claro que mis intereses estaban orientados hacia lo ambiental, y que lo puramente técnico o "duro" del desarrollo de software (el hardcodeo, la lógica estricta de programación) no era lo que más me apasionaba dentro de todo lo que veníamos trabajando en la escuela. De hecho, ese resultado terminó confirmando una decisión que ya venía masticando: actualmente me encuentro cursando el CBC para ingresar a la UBA, en Ciencias Ambientales o en Gestión de Agroalimentos. Lo que sí me pareció valioso de la charla fue ver esa afinidad reflejada de forma tan clara y ordenada en un test relativamente corto, y poder compartir con mis compañeros de curso cuánto se alejaban (o se acercaban) sus propios resultados del camino técnico que estamos recorriendo actualmente con el proyecto CarFind.

Charla sobre programas de intercambio educativo

EF Education First (11:20 hs)

La segunda charla a la que asistí, y la que definitivamente más me gustó de toda la jornada, fue la organizada por EF Education First (ef.com.ar), una institución internacional dedicada a la enseñanza de idiomas, los intercambios culturales y los programas académicos en el extranjero, con más de 60 años de trayectoria en el rubro. Esta charla también se dictó en una de las aulas del piso superior. Junto con mi compañera Aisana, pasamos prácticamente todo el día juntas recorriendo la feria, y ambas coincidimos en que esta fue, sin dudas, la propuesta que más nos entusiasmó de toda la jornada.

Durante la charla, nos explicaron la enorme variedad de propuestas que ofrece EF según la edad y el objetivo de cada estudiante. Vimos que existen desde campamentos de idiomas de corta duración (de una a cuatro semanas) pensados para adolescentes, con supervisión permanente de un líder de grupo durante las 24 horas, hasta programas mucho más extensos, de un semestre o un año completo, pensados para quienes buscan una inmersión total en un nuevo idioma y una nueva cultura, con la posibilidad de obtener certificaciones oficiales durante la estadía. También nos comentaron sobre programas orientados específicamente a jóvenes de nuestra edad que quieren cursar parte de la secundaria en el exterior, conviviendo con estudiantes de decenas de nacionalidades distintas dentro de colegios privados en el extranjero, así como programas de preparación universitaria con admisión garantizada a un número muy amplio de universidades asociadas alrededor del mundo.

Lo que más me entusiasmó de la propuesta de EF fue la cantidad de destinos e idiomas disponibles: mencionaron que se puede elegir entre decenas de destinos en distintos continentes y aprender idiomas como inglés, francés, alemán, italiano, chino mandarín, coreano, japonés, portugués o árabe, entre otros. A mí en lo personal me quedó dando vueltas la idea de, en algún momento, poder viajar para aprender alemán o chino mandarín, dos idiomas que siempre me generaron mucha curiosidad, pero que nunca había considerado seriamente como una posibilidad real hasta escuchar esta charla. También me pareció muy atractivo que los cursos incluyan, dentro de un mismo paquete, el alojamiento, las comidas, el traslado y las actividades culturales organizadas en el destino, además de soporte permanente durante toda la estadía, lo cual le da bastante tranquilidad a un estudiante que viaja por primera vez sin su familia.

En resumen, esta segunda charla me abrió la cabeza a la idea de que aprender un idioma no tiene por qué limitarse a una clase semanal en un instituto, sino que puede convertirse en una experiencia de vida completa, combinando el aprendizaje académico con el intercambio cultural real.

Salida didáctica al Instituto Nacional Superior del Profesorado Técnico (INSPT) de la Universidad Tecnológica Nacional

Fecha: 09/09/2026

Lugar: Av. Triunvirato 3174, CABA – Instituto Nacional Superior del Profesorado Técnico de la Universidad Tecnológica Nacional (UTN)

La salida didáctica al Instituto Nacional Superior del Profesorado Técnico (INSPT) de la Universidad Tecnológica Nacional fue una de las salidas más cortas para mí, ya que tuve que

retirarme antes de que finalizara la jornada. Por este motivo, mi experiencia se concentró principalmente en el taller al que pude asistir y en la forma en la que el profesor a cargo de la charla presentó sus conocimientos y su proyecto final. A pesar de haber permanecido menos tiempo, considero que fue una experiencia interesante porque pude observar un espacio educativo relacionado directamente con el área de Informática y, además, participar de una clase que se sintió bastante diferente a una charla tradicional.

Durante la visita asistí a la charla **“Mi primera huella web”**, correspondiente a la carrera de Tecnicatura Superior en Informática Aplicada. Uno de los aspectos que más me llamó la atención fue la forma en la que el profesor llevó adelante el taller. La explicación tuvo una dinámica bastante similar a una clase real, ya que antes de comenzar directamente con la parte práctica se presentó una introducción y se explicó la teoría necesaria para comprender lo que se iba a realizar posteriormente. Esto me resultó interesante porque permitió entender primero el concepto y después llevarlo a la práctica, en lugar de simplemente seguir una serie de pasos sin saber qué estaba sucediendo.

El objetivo principal de la actividad estaba relacionado con la publicación o deploy de una página web utilizando Vercel. Este tema me resultó especialmente interesante porque anteriormente ya había intentado utilizar Vercel por mi cuenta, aunque en ese momento no había conseguido que el proceso funcionara correctamente. Por eso, tener la posibilidad de ver nuevamente este procedimiento dentro de un contexto educativo me permitió relacionarlo con una experiencia previa que ya había tenido y entender mejor la utilidad de esta herramienta dentro del desarrollo web.

Durante el taller también pude observar parte del entorno de desarrollo que se estaba utilizando. En la computadora se encontraba abierto Visual Studio Code, con un proyecto organizado mediante diferentes archivos y componentes. Se podían observar archivos con extensión .jsx, como Education.jsx, Experience.jsx, Footer.jsx, Header.jsx y Skills.jsx, además de otros archivos como App.jsx, main.jsx, index.html y index.css. Esto permitió identificar que se estaba trabajando con React, utilizando componentes para organizar diferentes partes de la página web.

También pude observar el uso de Vite como herramienta para ejecutar el proyecto durante el desarrollo. En la terminal se encontraba ejecutado el comando npm run dev, y aparecía el servidor local funcionando mediante una dirección localhost. Esto me permitió relacionar diferentes herramientas que forman parte del desarrollo web actual, ya que no se trataba solamente de escribir código, sino también de utilizar un entorno de desarrollo, administrar el proyecto y ejecutarlo localmente para poder comprobar su funcionamiento antes de realizar el deploy.

La imagen que tomé durante la actividad permite registrar parte de este entorno de trabajo. Se puede observar Visual Studio Code junto con la estructura de carpetas del proyecto y los distintos componentes utilizados. También aparece la terminal con Vite ejecutándose y el proyecto disponible en localhost. Considero que este registro es importante porque permite mostrar de manera concreta qué tecnologías y herramientas estaban siendo utilizadas durante el taller y no solamente describirlas de manera teórica.

Además de Vercel, React y Vite, también pude relacionar la actividad con conocimientos previos sobre desarrollo web y programación. Durante mi formación en la escuela técnica tuve contacto con diferentes tecnologías y conceptos relacionados con el desarrollo de páginas y aplicaciones,

por lo que algunos elementos que aparecieron durante el taller no eran completamente nuevos para mí. Sin embargo, la experiencia permitió observar cómo estas herramientas pueden utilizarse juntas dentro de un proyecto concreto y cómo se organiza un proyecto web utilizando componentes.

Otro aspecto que me resultó interesante fue la manera en la que el profesor explicaba los contenidos. Se mostró carismático y profesional al momento de presentar su trabajo, pero al mismo tiempo la actividad mantuvo una dinámica bastante cercana a una clase. Esto hizo que la explicación no se sintiera solamente como una presentación sobre una tecnología, sino como una instancia en la que se podía comprender el proceso y posteriormente llevarlo a la práctica. Para mí, esto también permitió observar cómo es una clase dentro de una institución de formación técnica relacionada con Informática.

En relación con la institución, el INSPT pertenece a la Universidad Tecnológica Nacional y ofrece diferentes propuestas de formación vinculadas con áreas técnicas. En particular, la Tecnicatura Superior en Informática Aplicada está orientada principalmente a la construcción de software, incluyendo tareas como análisis, diseño y programación. También busca brindar una base conceptual que permita participar en proyectos de desarrollo de aplicaciones utilizando diferentes paradigmas de programación y herramientas. Esto se relaciona directamente con lo que pude observar durante el taller, ya que la actividad no se centró únicamente en aprender una herramienta puntual, sino que estuvo relacionada con una parte concreta del proceso de desarrollo de una aplicación web.

Según la información presentada sobre esta tecnicatura, también se busca que quienes se formen en ella puedan adaptarse a nuevas herramientas y tecnologías a medida que estas aparecen en el

ámbito laboral. Este punto me pareció especialmente relacionado con la actividad que realicé, ya que tecnologías como React, Vite y Vercel forman parte de un ecosistema que puede cambiar y actualizarse con el tiempo. Por eso, además de conocer herramientas específicas, considero importante desarrollar la capacidad de aprender nuevas tecnologías y entender cómo se relacionan entre sí.

A partir de lo observado, considero que para trabajar profesionalmente en este tipo de ámbito se necesitan conocimientos de programación y desarrollo de software, pero también otras competencias. Es necesario poder comprender los requerimientos de un proyecto, organizar el trabajo, utilizar diferentes herramientas y resolver problemas cuando algo no funciona como se esperaba. También es importante tener capacidad para investigar y aprender de manera autónoma, ya que las tecnologías utilizadas en el desarrollo web pueden cambiar y requieren mantenerse actualizadas. La experiencia del taller también me permitió observar la importancia de poder explicar conocimientos técnicos de manera clara, especialmente cuando se trabaja con otras personas o se tiene que enseñar un procedimiento.

Uno de los aspectos que más me llamó la atención fue justamente poder ver un proyecto web funcionando desde el entorno de desarrollo hasta su preparación para ser publicado. Antes de esta experiencia, Vercel era una herramienta que ya había intentado utilizar, pero mi experiencia había sido principalmente desde el intento personal. En cambio, durante el taller pude verla presentada dentro de un proceso de desarrollo más estructurado. También me resultó interesante reconocer herramientas como React y Vite en un proyecto real, ya que permitió conectar conceptos que había visto anteriormente con una situación concreta.

La salida también me permitió observar de una manera diferente el ámbito de formación técnica. Más allá de los contenidos específicos del taller, pude ver cómo un profesor puede combinar teoría y práctica para explicar una tecnología y cómo los conocimientos de una carrera como Informática Aplicada pueden trasladarse a proyectos concretos. Esto me ayudó a comprender mejor que el desarrollo de software requiere tanto conocimientos técnicos como la capacidad de adaptarse, investigar y resolver problemas.

Aunque tuve que retirarme antes de finalizar la actividad y por ese motivo no pude observar todas las propuestas de la salida, considero que la experiencia que pude realizar fue suficiente para obtener algunos aprendizajes concretos. Principalmente, pude profundizar mi comprensión sobre el proceso de publicación de una página web, relacionar Vercel con conocimientos previos y reconocer dentro de un proyecto herramientas como React, JSX, Vite, npm y Visual Studio Code. Además, pude ver un entorno de enseñanza relacionado directamente con una posible salida profesional dentro del área de Informática.

En conclusión, esta salida me permitió relacionar conocimientos que ya había tenido anteriormente con una experiencia más cercana al ámbito profesional y educativo de la informática. La charla “Mi primera huella web” fue especialmente útil porque no se limitó a mostrar una herramienta, sino que presentó primero los conceptos y luego los llevó a una aplicación práctica. Para mí, lo más importante de la experiencia fue poder ver cómo diferentes tecnologías trabajan en conjunto dentro de un proyecto web y entender un poco más el tipo de conocimientos y competencias que se desarrollan en la Tecnicatura Superior en Informática Aplicada.

Salida didáctica a la EXPO CARRERAS UTN.BA

Fecha: 16/09/2026

Lugar: Sede Campus UTN.BA, Mozart 2300

La salida didáctica a la EXPO CARRERAS UTN.BA se realizó el 16 de septiembre de 2026, en la Sede Campus de la Universidad Tecnológica Nacional, ubicada en Mozart 2300. La actividad se desarrolló aproximadamente desde las 13:00 hasta las 16:00 y asistí a las charlas junto con mis compañeros Ignacio Córdoba, Kiara Hernández y Samuel Vera. Personalmente, me gustó mucho realizar esta salida porque no suelo viajar demasiado y nunca había ido hasta esta sede de la UTN en Lugano, por lo que también fue una oportunidad para conocer un espacio diferente al que estoy acostumbrada.

Una de las primeras cosas que me llamó la atención fue la infraestructura del predio. El campus es bastante grande y cuenta con edificios nuevos, pero al mismo tiempo se mantienen en funcionamiento construcciones anteriores. También tiene espacios al aire libre, entre ellos un patio bastante amplio y una cancha de fútbol. Con mis compañeros nos emocionó tener un espacio de estas dimensiones, ya que no estamos acostumbrados a contar con un patio tan grande dentro de una institución educativa. Esto hizo que la primera impresión del lugar fuera bastante diferente a la de otros establecimientos que habíamos visitado anteriormente.

Durante la jornada asistimos a dos charlas relacionadas con distintas propuestas de formación de la UTN.

Charla N° 1: **Tecnicatura Universitaria en Desarrollo y Producción de Videojuegos**

Sala 2, Edificio Física, 1° piso

La primera fue a las 14:00 y estuvo relacionada con la **Tecnicatura Universitaria en Desarrollo y Producción de Videojuegos**. La charla se realizó en la Sala 2 del edificio Física, en el primer piso. Había una gran cantidad de personas dentro de la sala, aunque el aula era amplia y permitió que todos pudiéramos ubicarnos.

Durante esta charla se explicó principalmente en qué consiste la tecnicatura, cómo está organizada y cuáles son los conocimientos y áreas que se trabajan a lo largo de la formación. Se explicó que la carrera tiene una duración de dos años, organizada en cuatro cuatrimestres, y que se dicta completamente a distancia mediante el Campus Virtual de la Universidad. También se mencionó que para ingresar es necesario contar con el título secundario legalizado o convalidado.

Uno de los puntos que más me interesó de la explicación fue conocer cómo se encuentra organizado el plan de estudios. La formación combina diferentes áreas que normalmente pueden verse por separado, como programación, diseño de juegos, arte, narrativa, producción y desarrollo de experiencias interactivas. Durante el primer año se incluyen materias como Programación de Videojuegos 1 y 2, Probabilidad, Diseño de Juegos, Arte 2D, Inglés, Game Design, Motores de Desarrollo I y II, Audio Interactivo, Arte 3D, Technical Art, Narrativa y Estructura de Datos.

También se explicó que, a medida que avanza la carrera, existe la posibilidad de orientar el perfil hacia diferentes áreas. Entre las opciones mencionadas se encuentran Programación, Producción y Diseño.

Esto me permitió entender que la formación no apunta solamente a que todas las personas terminen realizando exactamente el mismo tipo de trabajo, sino que existe la posibilidad de desarrollar distintos perfiles dentro de una misma industria.

En relación con el objetivo de la carrera, durante la charla se explicó que la formación busca preparar profesionales capaces de participar en el desarrollo integral de proyectos de videojuegos, comprendiendo las diferentes etapas que forman parte de su creación. Esto incluye desde la idea inicial y el diseño hasta el desarrollo de prototipos y la producción de una experiencia interactiva. También se mencionó la integración de diferentes recursos y conocimientos, incluyendo tecnología, inteligencia artificial, arte y narrativa.

Otro aspecto que se explicó fue el perfil profesional que se busca desarrollar. Además de los conocimientos técnicos específicos, se planteó la importancia de poder trabajar de manera colaborativa, participar en equipos y utilizar metodologías ágiles y herramientas relacionadas con la industria. También se mencionó algo que me llama la atención que es la posibilidad de desarrollar experiencias visuales en 2D y 3D y participar en procesos creativos completos, desde la concepción de una idea hasta su transformación en un prototipo o proyecto.

La charla también permitió conocer con mayor claridad las posibles áreas laborales relacionadas con esta formación. Se mencionó que quienes se desarrollen profesionalmente en este campo pueden participar en estudios de videojuegos, desempeñándose en áreas como programación, arte, diseño o producción. Además, los conocimientos pueden relacionarse con otras industrias creativas, como el cine, la animación digital y la publicidad, así como con proyectos de programación y diseño interactivo. También se mencionó la posibilidad de trabajar en proyectos independientes y emprendimientos propios.

Me resultó interesante esta parte porque permitió observar que una formación relacionada con videojuegos no se limita necesariamente al desarrollo de un videojuego tradicional. Los conocimientos de programación, diseño, arte, producción y narrativa pueden utilizarse en diferentes tipos de proyectos interactivos y en distintas áreas de la industria.

Otro punto que se explicó durante la charla fue la modalidad a distancia. Al tratarse de una carrera que se cursa completamente mediante el Campus Virtual de la UTN, se explicó que el entorno se encuentra disponible las 24 horas y permite organizar los tiempos de estudio con mayor autonomía.

Además, se explicó que existe acompañamiento académico por parte de docentes especializados y que las materias cuentan con diferentes instancias de evaluación.

Esta información me permitió conocer no solamente qué materias forman parte de la carrera, sino también cómo está pensada la formación y qué tipo de perfil profesional busca desarrollar. Me pareció importante que durante la charla se explicará el recorrido completo, porque de esta manera pude entender mejor la relación entre las materias de los primeros cuatrimestres y los contenidos que aparecen posteriormente. Por ejemplo, algunas materias están orientadas a desarrollar conocimientos técnicos específicos, mientras que otras se relacionan con el diseño, el arte, la narrativa o la producción, formando diferentes partes de un mismo proceso de desarrollo.

Y se nos dejó claro que el objetivo de la formación está relacionado con la preparación para una industria que se encuentra en constante evolución. En este sentido, además de aprender herramientas concretas, se destacó la importancia de poder adaptarse a nuevas tecnologías y formas de trabajo. Esto me pareció relacionado con el ámbito de la informática en general, ya que

las herramientas utilizadas pueden cambiar con el tiempo y es necesario continuar aprendiendo para poder desenvolverse profesionalmente.

En cuanto a los conocimientos y competencias que requiere este ámbito, pude identificar que no alcanza solamente con saber programar. También son necesarios conocimientos relacionados con el diseño, la resolución de problemas, el trabajo en equipo, la comunicación y la organización de proyectos. Dependiendo del área elegida, pueden adquirir mayor importancia la programación, el diseño, el arte o la producción, pero todos estos perfiles forman parte de un mismo proceso de trabajo y necesitan poder relacionarse entre sí.

Por último, durante la charla también se explicaron aspectos relacionados con el ingreso a la carrera y la documentación necesaria. Se mencionó como requisito contar con el título secundario legalizado o convalidado y que el proceso de inscripción se realiza de manera online. Esta parte fue más administrativa, pero permitió conocer cómo continúa el proceso después de conocer la propuesta académica y qué documentación se requiere para iniciar una carrera de este tipo.

Charla N° 2: **Ingeniería en Sistemas de Información**

Sala 1, Edificio Física, 1° piso

La segunda charla a la que asistimos fue sobre la **Ingeniería en Sistemas de Información** y se realizó a las 15:00 en la Sala 1 del Edificio Física, en el primer piso. Esta charla estuvo mucho más llena que la anterior, había muchísimas personas dentro de la sala y, por la cantidad de gente interesada, me dio la impresión de que es una carrera que genera bastante interés.

La explicación comenzó con una introducción sencilla sobre qué es la carrera y qué tipo de actividades puede realizar un profesional de esta área. Aunque durante gran parte de la charla me costó escuchar y entender todo lo que se explicaba debido a la cantidad de personas y al ruido del lugar, pude obtener una idea general de la propuesta. La **Ingeniería en Sistemas de Información** es una carrera de grado de aproximadamente cinco años, orientada a la planificación, el diseño, el desarrollo, la implementación y el mantenimiento de sistemas de información y software para diferentes organizaciones.

También se explicó que la carrera combina diferentes áreas de conocimiento, entre ellas programación, matemática, redes, bases de datos y gestión de procesos de negocio. Esto me pareció interesante porque muestra que el trabajo dentro de Sistemas de Información no se limita únicamente a programar, sino que también implica comprender cómo funcionan las organizaciones y cómo la tecnología puede utilizarse para resolver diferentes necesidades.

Dentro de la explicación también se mencionó que la carrera cuenta con un título intermedio de Analista Universitario de Sistemas, al que se puede acceder al cumplir los requisitos correspondientes y las prácticas necesarias. Para completar la carrera de Ingeniería se requiere realizar una Práctica Profesional Supervisada de alrededor de 200 horas y un Proyecto Final. Estos aspectos me permitieron tener una idea más concreta de cómo se organiza una carrera de grado de este tipo y de las diferentes etapas que debe atravesar una persona hasta completar su formación.

La definición que pude rescatar de la charla es que la Ingeniería en Sistemas de Información se relaciona con planificar, dirigir, ejecutar y controlar diferentes etapas de los sistemas de información y del software, incluyendo el relevamiento, análisis, diseño, desarrollo, prueba,

implementación y mantenimiento. Me pareció importante esta parte porque permite entender que un sistema no empieza directamente con la programación, sino que existe un proceso previo de análisis y planificación y, posteriormente, también hay etapas relacionadas con su implementación y mantenimiento.

Antes de que terminara la actividad pude conseguir uno de los folletos que contenía información sobre la carrera y sus materias, ya que se estaban acabando rápidamente. Esto me permitió tener un registro más completo de la propuesta y revisar posteriormente algunos de los contenidos que se habían mencionado durante la charla. Considero que fue útil porque, debido a que no pude escuchar con claridad toda la explicación, el material me permitió recuperar parte de la información presentada.

En general, la salida me gustó bastante, aunque en mi caso creo que me interesó más conocer el espacio físico y el ambiente universitario que las charlas en sí, principalmente porque la cantidad de personas dificultó bastante poder escuchar la segunda exposición. Aun así, pude conocer una parte de la UTN que no había visitado anteriormente y observar cómo es el ambiente de una universidad orientada principalmente a carreras técnicas e ingenierías.

También pude enterarme de que la UTN cuenta con un Centro de e-Learning, mediante el cual se pueden realizar propuestas de formación de manera virtual. Me llamó la atención conocer esta posibilidad, ya que no sabía que la universidad contaba con una modalidad de este tipo. También pude observar otros espacios del campus, como la cafetería, que contaba con diferentes opciones, entre ellas un kiosco, cafetería y hamburguesería. Estos detalles pueden parecer secundarios en comparación con las charlas, pero para mí también formaron parte de la experiencia, porque permitieron conocer cómo es el entorno y los espacios que forman parte de la vida universitaria.

Lo que más rescato de esta salida fue justamente poder conocer de manera presencial un campus universitario de la UTN y observar un ambiente académico orientado a la tecnología, las carreras técnicas y las ingenierías. Además, las dos charlas me permitieron conocer propuestas de formación diferentes, especialmente la Tecnicatura Universitaria en Desarrollo y Producción de Videojuegos y la Ingeniería en Sistemas de Información, y comparar los distintos enfoques que puede tener el área de informática.

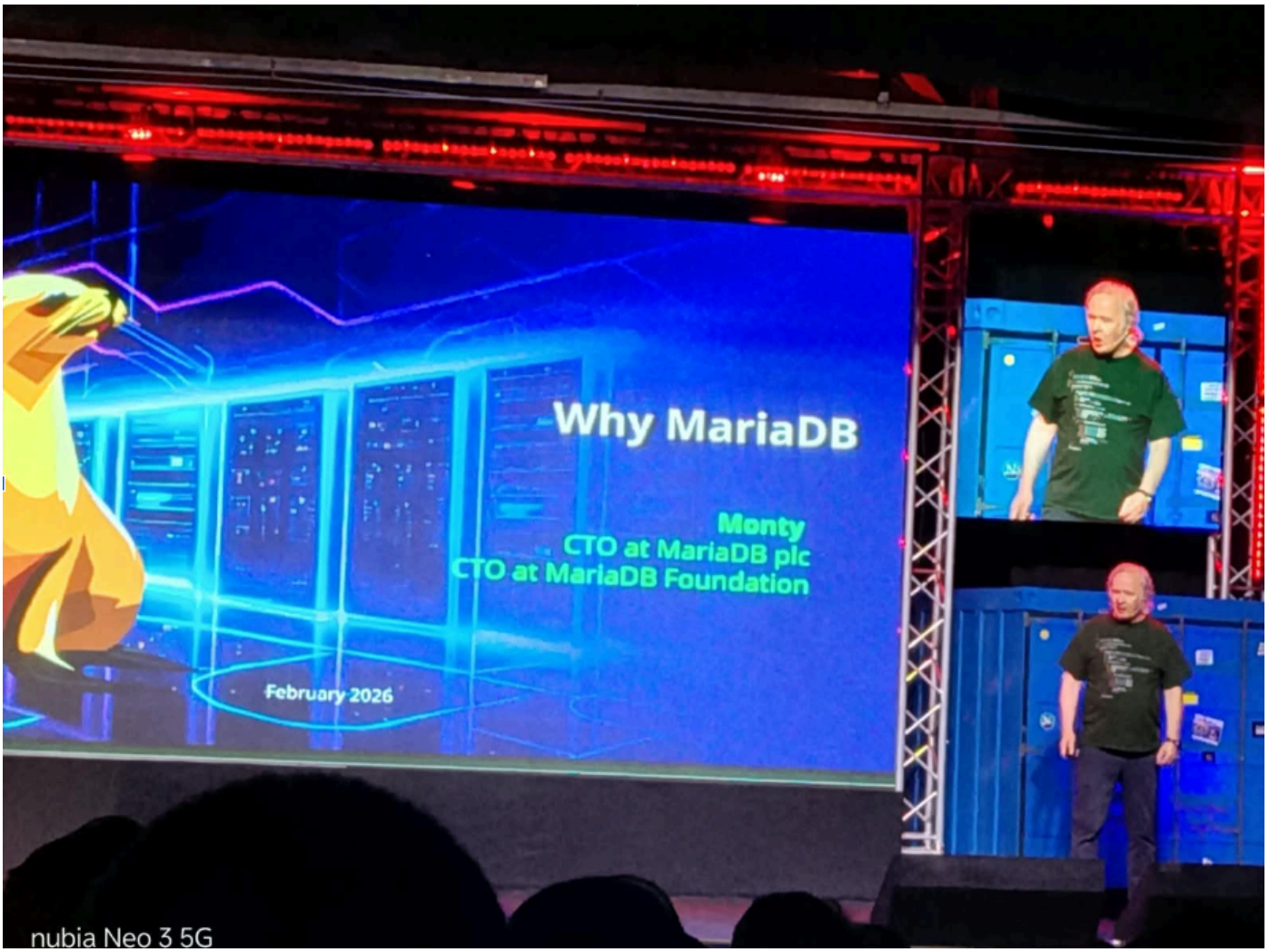
Salida didáctica al NERDEARLA, Primera Jornada del 24 de Septiembre

Fecha: 24/09/2026

Lugar: Ciudad Cultural Konex, Ciudad Autonoma de Buenos Aires

Charla N° 1: **Monty Widenius opening keynote - Data Science / AI**

Jueves 24/09/2026 10:10 AM - 10:50 AM



Esta fue la primera charla a la que asistí, y estuvo a cargo de Monty Widenius, creador de MySQL y MariaDB y desarrollador reconocido dentro del ámbito de las bases de datos de código abierto. Durante la charla se trataron principalmente temas relacionados con las bases de datos, el desarrollo de software y el impacto que está teniendo la Inteligencia Artificial en proyectos de gran escala.

Uno de los temas principales fue el desarrollo de MariaDB y la importancia de las bases de datos de código abierto. Además, Monty explicó de manera más teórica las diferencias entre las

distintas bases de datos que existen, lo que permitió comprender que no todas funcionan de la misma manera ni están pensadas para exactamente los mismos usos. También habló sobre la historia de MySQL y sobre su experiencia con Oracle, mencionando de forma crítica que, en sus propias palabras, Oracle “despidió a casi todos los desarrolladores” de MySQL. Esto permitió conocer no solamente el aspecto técnico de las tecnologías, sino también parte del contexto que llevó al desarrollo de MariaDB.

Otro punto que me llamó la atención fue la relación entre las bases de datos y la Inteligencia Artificial. Se habló sobre el impacto y el uso de la IA en proyectos de desarrollo de gran envergadura, mostrando que estas tecnologías ya forman parte de proyectos complejos y que su incorporación también plantea nuevos desafíos para las personas que trabajan en el área. Me resultó interesante poder relacionar este tema con conocimientos que ya había trabajado anteriormente sobre Inteligencia Artificial y desarrollo tecnológico, pero aplicado en un contexto más profesional y relacionado directamente con sistemas de gran escala.

A partir de la charla pude relacionar algunos conocimientos previos de computación y de bases de datos con situaciones más cercanas al ámbito profesional. También pude entender mejor que el trabajo relacionado con el desarrollo de software no consiste solamente en programar, sino que requiere conocer las tecnologías utilizadas, comprender sus diferencias y poder tomar decisiones según las necesidades de cada proyecto. En este ámbito también son importantes la capacidad de aprendizaje, la adaptación a nuevas tecnologías y la comprensión de cómo funcionan las herramientas que forman parte de un sistema.

La experiencia me permitió conocer una perspectiva profesional sobre tecnologías que ya había visto de manera más general y, al mismo tiempo, ampliar mi comprensión sobre las bases de

datos de código abierto. También aprendí más sobre MariaDB, su relación con MySQL y algunos de los motivos que llevaron a su desarrollo. Además, pude observar cómo una persona con una trayectoria extensa dentro del área analiza la evolución de estas tecnologías y su relación con los cambios actuales, especialmente con la incorporación de la Inteligencia Artificial en proyectos de gran envergadura.

Charla N° 2: **El Manager que no usa IA ya perdió - Management**

Jueves 24/09/2026 10:25 AM - 11:05 AM



Participé de la charla “El Manager que no usa IA ya perdió”, dictada por Maximiliano Contieri . La actividad estuvo relacionada con el uso de Inteligencia Artificial dentro de la gestión de equipos de software y con los cambios que estas herramientas están generando en la forma de planificar, organizar, documentar y tomar decisiones dentro de un proyecto tecnológico.

Durante la charla se presentó una comparación entre la forma de gestionar proyectos de software en la década de 1990 y las características de los equipos actuales, que pueden trabajar de manera remota o híbrida y estar distribuidos entre diferentes países, culturas y zonas horarias. En este contexto, la Inteligencia Artificial aparece como una herramienta que puede ayudar a los profesionales a procesar grandes cantidades de información, detectar patrones, resumir datos, identificar riesgos, organizar tareas y generar documentación. Sin embargo, uno de los puntos principales de la charla fue que la IA no debe reemplazar el criterio humano, sino ampliar las capacidades de las personas. El manager continúa siendo responsable de comprender el contexto, establecer prioridades, tomar decisiones y asumir la responsabilidad por las acciones realizadas.

Entre las tecnologías y herramientas mencionadas se encontraron diferentes usos de Inteligencia Artificial aplicados a la gestión y al desarrollo de software. Se habló de prompts y habilidades reutilizables para tareas como la priorización, la generación de reportes, la documentación, la elaboración de minutas de reuniones, el análisis de riesgos, el seguimiento de problemas y la preparación de conversaciones difíciles. También se mencionaron herramientas no-code para realizar prototipos, sistemas de conocimiento interno o “company brain”, automatización de pruebas, integración y despliegue continuo, análisis de rendimiento, monitoreo de incidentes y herramientas de IA capaces de detectar patrones o realizar determinadas acciones dentro de límites establecidos.

La charla también permitió observar que en este ámbito participan distintos perfiles profesionales y que el trabajo no depende de una sola persona. Se mencionaron managers, desarrolladores, profesionales de QA, equipos técnicos, clientes y otros stakeholders. Cada uno cumple diferentes funciones, pero existe una relación constante entre la parte técnica y la gestión del proyecto. Por ejemplo, un manager puede utilizar IA para analizar tareas y detectar bloqueos, pero necesita comprender el trabajo del equipo para determinar si esas sugerencias realmente tienen sentido. De la misma manera, los profesionales de QA pueden utilizar herramientas de IA para generar posibles planes de prueba o analizar riesgos, aunque la validación final sigue dependiendo de las personas.

Varios de los contenidos presentados se relacionan con conocimientos que ya había trabajado anteriormente. Principalmente, pude relacionarlos con la planificación y organización de proyectos, la división de tareas, la comunicación dentro de un equipo y el seguimiento del trabajo. También encontré relación con los contenidos de Inteligencia Artificial que ya había trabajado, especialmente en cuanto al uso de prompts, el análisis de información y la importancia de mantener supervisión humana. Además, algunos conceptos se relacionaron con la forma de trabajo de CarFind, ya que en el proyecto es necesario organizar tareas, establecer prioridades, documentar el trabajo y trabajar con diferentes integrantes del equipo. La idea de dividir una tarea grande en partes más pequeñas también se relaciona con la organización del proyecto y con la necesidad de que las tareas puedan ser comprendidas, desarrolladas y verificadas de manera más clara.

A partir de lo observado, pude identificar que en este ámbito no alcanza solamente con conocer herramientas tecnológicas. También son necesarias competencias relacionadas con la

planificación, la organización, la comunicación, la resolución de problemas, la identificación de riesgos y la capacidad de tomar decisiones. A esto se suma la necesidad de comprender cómo utilizar la Inteligencia Artificial de manera responsable. Por ejemplo, se destacó la importancia de no cargar información confidencial en herramientas que no estén autorizadas, revisar la información generada por la IA y mantener una persona responsable de las decisiones. Esto me pareció especialmente importante porque demuestra que saber utilizar una herramienta no significa necesariamente saber utilizarla correctamente.

Uno de los aspectos que más me llamó la atención fue justamente la diferencia entre utilizar IA para automatizar tareas y utilizarla como una herramienta de apoyo para ampliar las capacidades de una persona. Durante la charla se explicó que la IA puede encargarse de tareas repetitivas o ayudar a organizar información, pero también puede funcionar como una especie de compañero de análisis o “sparring” para preparar decisiones, detectar riesgos o incluso practicar conversaciones difíciles. Me resultó interesante porque muestra un uso de la IA diferente al que normalmente se piensa cuando se habla de generación de contenido o programación. En este caso, la herramienta se utiliza principalmente para ayudar al profesional a pensar, analizar y prepararse mejor.

También me llamó la atención el concepto de “company brain”, es decir, un sistema de conocimiento apoyado por IA que puede reunir información de proyectos, decisiones, reuniones, documentación técnica, riesgos y aprendizajes anteriores. La idea de que el conocimiento importante de una organización no dependa únicamente de una persona me pareció relevante, especialmente en equipos donde diferentes integrantes pueden incorporarse, cambiar de proyecto o dejar la organización. Aun así, se aclaró que este sistema no debe considerarse automáticamente

como una fuente de verdad, ya que la información debe mantenerse actualizada y ser revisada por personas.

Otro punto que considero importante de la experiencia fue el concepto de human-in-the-loop. La IA puede analizar información, proponer prioridades, identificar problemas o generar una posible solución, pero la decisión final continúa estando en manos de las personas. Esto se aplica especialmente en situaciones relacionadas con contrataciones, evaluaciones, salarios, comunicación con clientes, incidentes técnicos y otros procesos donde una decisión incorrecta puede tener consecuencias importantes. De esta manera, la tecnología puede aportar velocidad y capacidad de análisis sin eliminar la responsabilidad profesional.

A partir de esta experiencia aprendí que el rol de un manager dentro de un equipo tecnológico está cambiando y que la Inteligencia Artificial puede modificar muchas de las tareas que tradicionalmente se realizaban de forma manual. La planificación, la documentación, los reportes, las reuniones, el seguimiento de tareas y el análisis de riesgos pueden apoyarse en estas herramientas, pero eso también requiere que los profesionales sepan utilizarlas con criterio. Además, pude comprender que trabajar con IA no consiste solamente en saber escribir prompts, sino también en saber qué información proporcionarle, cómo revisar sus resultados, qué decisiones no deberían delegarse y cómo mantener la responsabilidad humana durante todo el proceso.

En general, la charla me permitió relacionar conocimientos que ya venía trabajando sobre tecnología, proyectos e Inteligencia Artificial con una aplicación más orientada a la gestión de equipos de software. También me ayudó a entender que, dentro de un ámbito profesional, el valor de la IA no está únicamente en hacer una tarea más rápido, sino en utilizarla para obtener

información, detectar problemas y facilitar el trabajo sin perder el criterio y la responsabilidad de las personas que forman parte del proyecto.

Charla N° 3: **Más allá del prompt: principios para diseñar experiencias agentivas centradas en las personas**

Jueves 24/09/2026 11:40 AM - 12:20 PM - UX



La charla “Más allá del prompt: principios para diseñar experiencias agentivas centradas en las personas”, estuvo a cargo de Laura Passera, quien se presentó como product designer y user

experience designer. Además, comentó que también trabaja como docente de comunicación transmedia en la Universidad de Buenos Aires. La charla estuvo enfocada principalmente en el diseño de experiencias con inteligencia artificial y, especialmente, en cómo pensar los sistemas de IA desde una perspectiva centrada en las personas, en lugar de enfocarse únicamente en sus capacidades técnicas.

Uno de los primeros aspectos que me llamó la atención fue la relación que se planteó entre la tecnología y las necesidades reales de las personas. Para explicar esta idea se utilizó como ejemplo el LaserDisc, una tecnología que para su época ofrecía una muy buena calidad de video, pero que no logró incorporarse de la misma manera que el VHS. Esto se debía, entre otras cosas, a que el VHS podía grabar contenido, transportarse con mayor facilidad, alquilarse, prestarse y adaptarse mejor a los hábitos que ya tenían las personas. Este ejemplo permitió mostrar que una tecnología puede ser técnicamente avanzada y, aun así, no tener el mismo impacto si no encuentra una utilidad concreta dentro de la vida cotidiana de sus usuarios.

Esta idea se relaciona con conocimientos que ya venía trabajando sobre inteligencia artificial, especialmente con la importancia de utilizar la tecnología de manera responsable y teniendo en cuenta sus límites. También pude relacionarlo con proyectos anteriores en los que trabajé con herramientas de IA, ya que muchas veces el foco puede estar puesto en qué puede hacer una herramienta, pero no necesariamente en si realmente es la solución más adecuada para el problema que se quiere resolver. En la charla se remarcó justamente que no se debería incorporar inteligencia artificial solamente porque es una tecnología actual o porque puede hacer determinada tarea, sino que primero es necesario identificar qué problema existe, quién lo tiene y qué valor aportaría realmente utilizar IA para resolverlo.

A partir de esto, se explicó que los agentes de inteligencia artificial presentan una diferencia importante respecto de las experiencias digitales tradicionales. En una experiencia tradicional, el diseñador puede definir de manera bastante específica los pasos que seguirá el usuario, las pantallas que verá y las posibles respuestas del sistema. En cambio, un agente puede interpretar objetivos, tomar decisiones y ejecutar acciones con cierto grado de autonomía. Por este motivo, el trabajo de diseño no consiste solamente en crear una interfaz, sino también en definir qué puede hacer el agente, qué límites tiene, qué decisiones puede tomar por sí mismo y en qué situaciones debe intervenir una persona.

Este punto me resultó especialmente interesante porque cambia la forma de pensar la relación entre una persona y una herramienta tecnológica. El diseñador ya no tiene que pensar solamente en qué va a hacer el usuario dentro de una aplicación, sino también en qué acciones va a delegar en el sistema y cuáles deberían permanecer bajo control humano. Esto se relaciona con temas de ética y responsabilidad que ya había visto anteriormente, ya que una herramienta de IA puede cometer errores o tomar decisiones que afecten a una persona, pero eso no significa que la responsabilidad desaparezca. Durante la charla se remarcó que la responsabilidad continúa siendo humana, incluso cuando una decisión haya sido tomada o ejecutada por un sistema.

También se presentaron siete principios para diseñar experiencias con agentes de inteligencia artificial. El primero consiste en definir la necesidad y el valor que realmente aporta la tecnología. Después se debe definir el rol del agente, estableciendo qué tareas puede realizar, cuáles no y qué decisiones requieren intervención humana. Otro punto importante es calibrar su autonomía según el impacto que pueda tener una decisión y según qué tan fácil sea revertirla. Por ejemplo, una acción sencilla y reversible puede tener un mayor grado de autonomía, mientras que

una decisión relacionada con dinero, salud, seguridad o información sensible requiere mayores controles y supervisión.

Un ejemplo que me pareció claro fue el caso de un cliente que había comprado unos auriculares que dejaron de funcionar después de 45 días. La política de la empresa permitía evaluar reclamos realizados dentro de los 90 días, pero el agente le informó al cliente que tenía derecho a un reembolso, generando la expectativa de que el reembolso ya estaba aprobado. La diferencia entre poder solicitar una evaluación y tener garantizado un resultado muestra por qué es importante establecer límites claros para un agente. El problema no estaba solamente en si la información era correcta, sino también en la expectativa que el sistema había generado en el usuario.

Otro principio trabajado fue el diseño de la confianza. Se explicó que una persona tiene más posibilidades de confiar en un sistema cuando puede comprender qué está haciendo, por qué lo está haciendo y cuáles son sus límites. Esto también se relaciona con los temas de ética de la información y uso responsable de la inteligencia artificial que ya había trabajado anteriormente. Una respuesta generada por IA no debería presentarse automáticamente como una verdad absoluta, especialmente cuando puede existir información incompleta, sesgos o diferentes alternativas que el sistema no está mostrando. Por eso, el diseño también debe ocuparse de comunicar incertidumbre y brindar información suficiente para que la persona pueda tomar sus propias decisiones.

La charla también mostró que los sistemas de IA necesitan estar preparados para situaciones inesperadas. A diferencia de una experiencia completamente definida de antemano, un sistema basado en IA puede interpretar una solicitud de una manera diferente a la esperada o realizar una modificación que la persona no había pedido. Se presentó como ejemplo una herramienta de

generación de imágenes a la que se le solicitó cambiar solamente el color de una camisa, pero que además agregó un bolsillo. Aunque el sistema había realizado la tarea principal, también había modificado algo que no formaba parte de la intención original. Esto demuestra que no alcanza con que un agente complete una tarea: también es necesario evaluar si respetó lo que realmente quería la persona y si existe una manera sencilla de corregir o revertir el resultado.

Otro aspecto importante fue la forma de medir el éxito de un agente. Se planteó que completar una tarea no debería ser la única medida utilizada para determinar si un sistema funciona correctamente. También se pueden considerar la cantidad de correcciones necesarias, las desviaciones respecto de la intención del usuario, el tiempo utilizado para solucionar errores, el esfuerzo que tuvo que realizar la persona y la frecuencia con la que fue necesaria una intervención humana. Esto me pareció importante porque permite entender que una herramienta puede funcionar técnicamente y, al mismo tiempo, generar una experiencia poco útil para el usuario.

Además, se habló de cómo los sistemas pueden evolucionar con el tiempo cuando tienen memoria y utilizan información de interacciones anteriores. El ejemplo presentado fue el de un agente que organizaba viajes y que continuaba recomendando hoteles con determinadas características porque recordaba las necesidades que una persona había tenido cuando viajaba con sus hijos, aunque con el tiempo esas circunstancias habían cambiado. Esto plantea la necesidad de que los usuarios puedan saber qué información recuerda el sistema, modificarla, eliminarla o actualizarla. En este sentido, el diseño también debe considerar que las personas cambian y que una información que era correcta anteriormente puede dejar de serlo.

Entre las competencias profesionales que pude identificar a partir de la charla se encuentran el análisis de necesidades, la capacidad de comprender a los usuarios, el diseño de experiencias, la comunicación, el pensamiento crítico, la resolución de problemas y la capacidad de trabajar con inteligencia artificial teniendo en cuenta sus límites. También resulta importante comprender aspectos éticos y de responsabilidad, ya que trabajar con agentes implica decidir qué acciones pueden delegarse y cuáles necesitan supervisión humana. Además, la charla mostró que estos campos pueden cambiar con el tiempo. La propia experiencia de Laura Passera permitió observar cómo las profesiones relacionadas con la comunicación, el diseño web, UX y product design fueron evolucionando junto con las necesidades del mercado y la aparición de nuevas tecnologías.

En mi caso, varios de estos temas se relacionan con conocimientos que ya fui incorporando mediante distintos proyectos y experiencias vinculadas con inteligencia artificial. Por ejemplo, ya había trabajado sobre los límites de la IA, los sesgos, la necesidad de supervisión humana y el uso responsable de diferentes herramientas. También tuve experiencias utilizando herramientas de generación de contenido, análisis y creación de prototipos, por lo que la charla me permitió observar estos temas desde otra perspectiva, más relacionada con el diseño de la experiencia y con la relación entre la tecnología y las personas.

Lo que más me llamó la atención fue que el desarrollo de una solución con inteligencia artificial no debería centrarse solamente en conseguir que el sistema haga algo. También es necesario pensar qué consecuencias puede tener, qué grado de autonomía debería tener, cómo se va a comportar cuando se equivoque y cómo va a interactuar con la persona a lo largo del tiempo. Me pareció interesante porque muchas veces la atención está puesta en mostrar lo que una

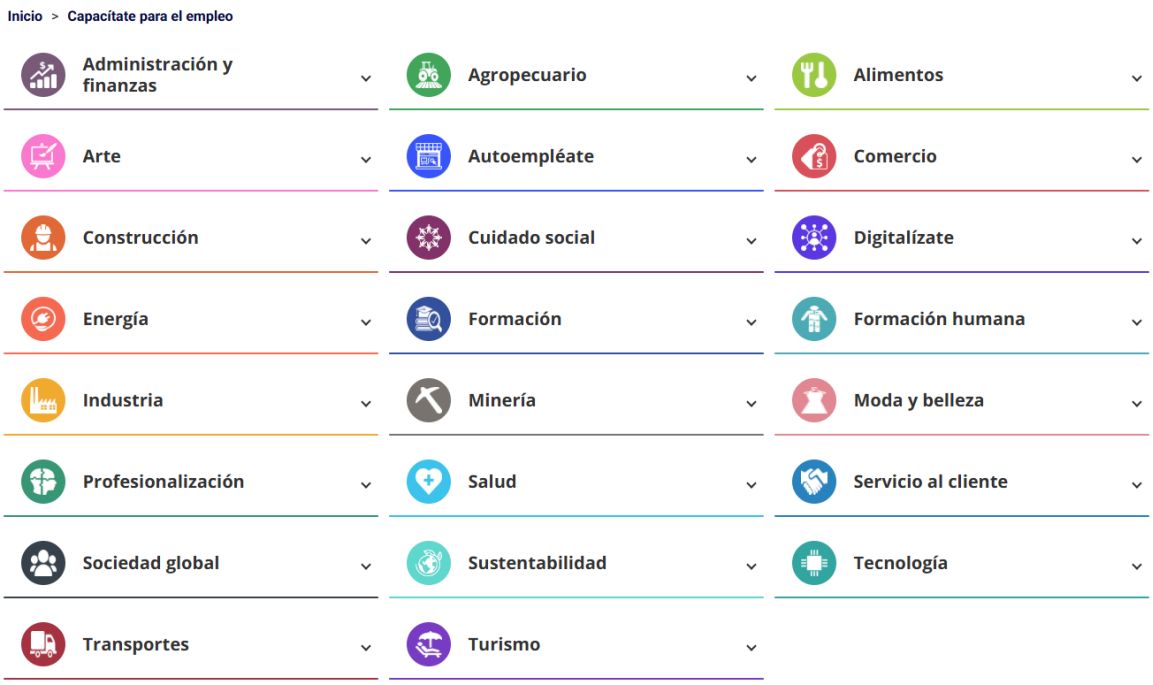
herramienta de IA puede hacer, mientras que en esta charla el enfoque estuvo más relacionado con decidir cuándo realmente conviene utilizarla y cómo hacerlo sin perder de vista las necesidades humanas.

A partir de esta experiencia aprendí que diseñar con inteligencia artificial implica mucho más que utilizar una herramienta o escribir buenos prompts. También requiere comprender el problema, definir claramente el rol de la tecnología, establecer límites, considerar la intervención humana y pensar en situaciones que pueden ocurrir incluso cuando no fueron previstas inicialmente. Además, entendí que una solución tecnológica no necesariamente es mejor por ser más avanzada, sino que su valor depende de cómo se adapta a las necesidades reales de las personas y de si logra aportar una mejora concreta a su experiencia. La charla me permitió ampliar la forma en la que pienso la inteligencia artificial, pasando de verla solamente como una herramienta capaz de realizar determinadas tareas a considerarla también como parte de una experiencia que debe ser diseñada, supervisada y evaluada teniendo siempre en cuenta a las personas que van a utilizarla.

DIPLOMATURAS CARLOS SLIMM

La Fundación Carlos Slim es una organización sin fines de lucro que desarrolla programas sociales en América Latina, principalmente en México, con foco en educación, empleo, salud y desarrollo económico y social. Su objetivo principal es ofrecer oportunidades de formación y mejorar la calidad de vida de las personas, especialmente de quienes se encuentran en contextos vulnerables.

A través de sus plataformas digitales, la Fundación ofrece cursos, diplomados y contenidos formativos gratuitos en áreas como educación, cultura, salud y capacitación para el empleo. Estos programas buscan que las personas adquieran competencias productivas acordes a los nuevos trabajos del siglo XXI, promoviendo la inclusión social, el autoempleo y la creación de pequeños negocios.



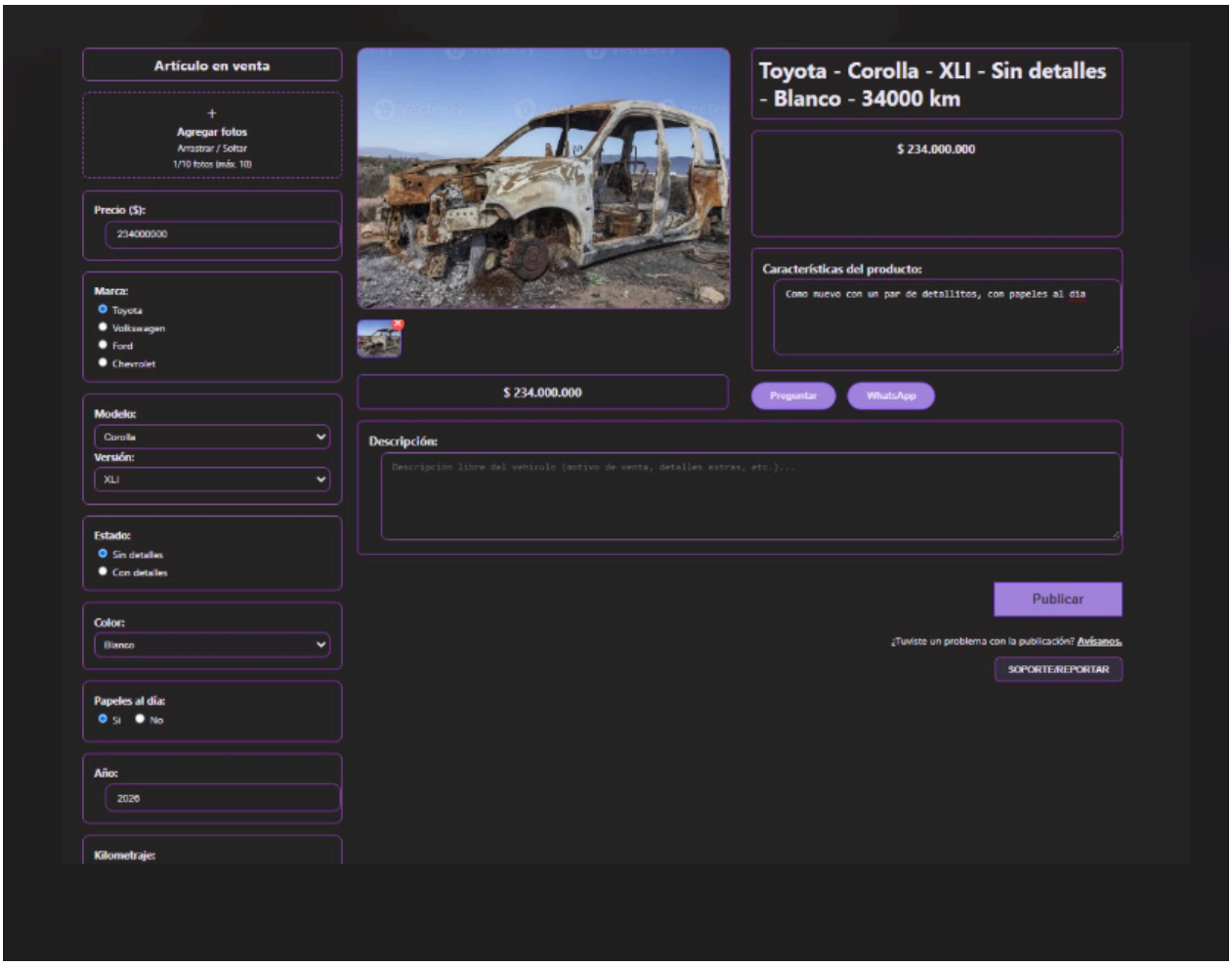
Dentro de su oferta educativa, la Fundación Carlos Slim ha desarrollado más de 300 cursos y al menos 15 diplomados oficiales, muchos de ellos avalados por la Secretaría de Educación Pública de México. Estas propuestas han beneficiado a millones de personas en Latinoamérica, permitiendo que puedan capacitarse y certificarse en línea, de manera gratuita y a su propio ritmo

Objetivo de la diplomatura elegida

La diplomatura que se cursa a través de la plataforma "Capacítate para el Empleo" de la Fundación Carlos Slim tiene como objetivo ofrecer capacitación gratuita en ocupaciones técnico–operativas y en áreas tecnológicas de alta demanda, como el manejo y análisis de datos. Busca que las personas puedan adquirir conocimientos actualizados y habilidades prácticas que les permitan mejorar sus oportunidades de empleo, autoempleo o emprendimiento, fortaleciendo así su autonomía financiera y su preparación para los nuevos modelos de trabajo del siglo XXI.
En mi caso, la elección de esta diplomatura se centra en el campo de los datos, ya que cursos como Finder y Curador de Datos contribuyen a desarrollar competencias para localizar información relevante, organizarla, limpiarla y prepararla para su uso en proyectos tecnológicos. De esta manera, la formación recibida se relaciona directamente con el perfil técnico que necesito para participar en proyectos como CarFind, donde el tratamiento de datos y la toma de decisiones basadas en información confiable son fundamentales.

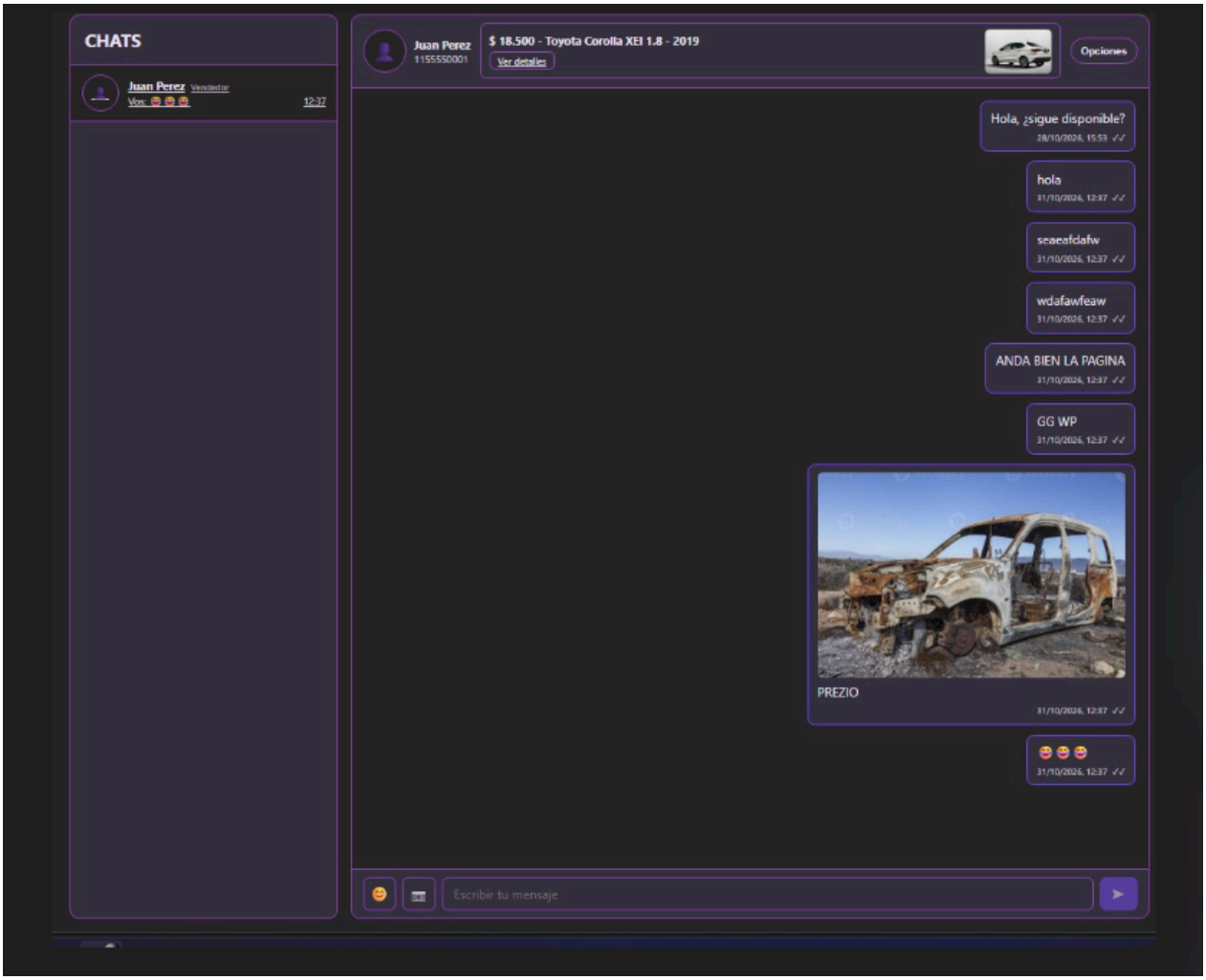


PÁGINA WEB FINAL



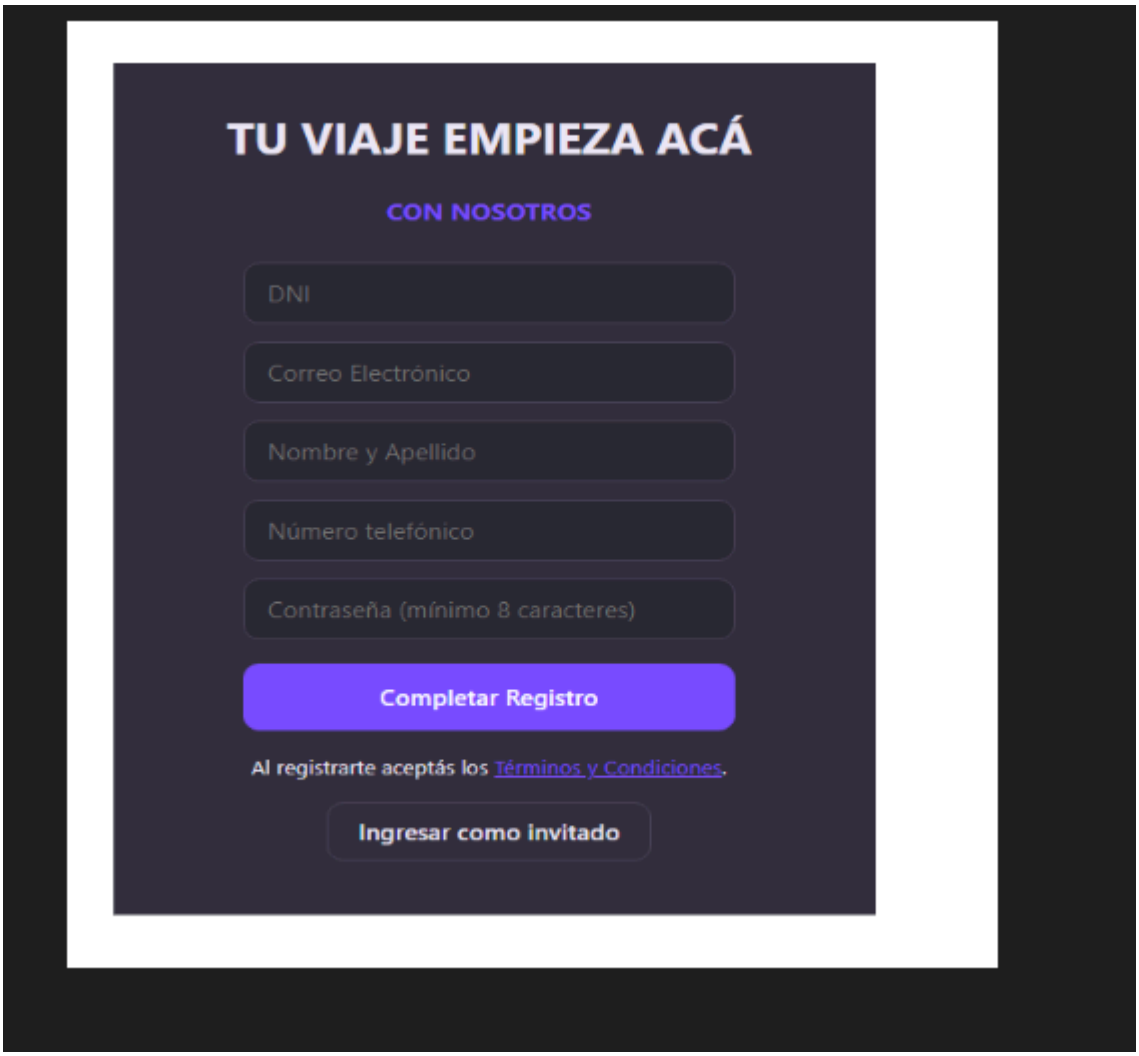
Pantalla de Formulario de Publicación de Vehículos: Esta interfaz representa el módulo operativo destinado a los usuarios vendedores para dar de alta un nuevo artículo en venta dentro de la plataforma. El formulario se encuentra estructurado de manera modular y vertical para optimizar la experiencia de carga de datos. En la sección superior izquierda cuenta con un componente de carga y arrastre de archivos fotográficos (con un límite de hasta 10 imágenes). Debajo, dispone de campos de entrada estructurados para definir el precio, la selección de marcas mediante botones de opción (*radio buttons*), menús desplegables (*select*) para modelos, versiones, estados de conservación, colores y la situación legal de los papeles del vehículo

("Papeles al día"). Asimismo, incluye campos numéricos específicos para el año y el kilometraje. En la sección derecha, la interfaz ofrece una vista previa en tiempo real (*live preview*) de cómo se visualizará la tarjeta del producto, acompañada de un cuadro de texto libre para la descripción detallada de características adicionales, un botón principal de confirmación ("Publicar") y accesos directos de soporte técnico o reporte de inconvenientes.



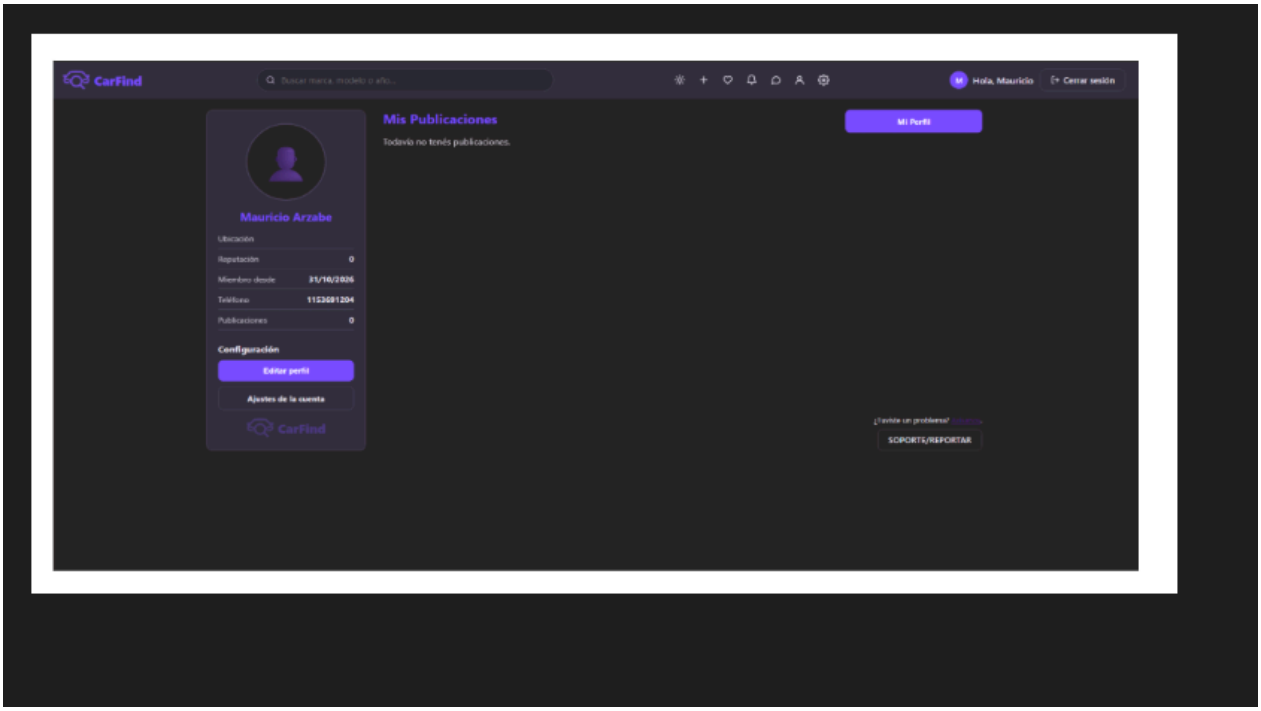
Módulo de Comunicación y Chat Interno entre Usuarios: Esta pantalla corresponde a la interfaz de mensajería en tiempo real integrada en la plataforma, diseñada para facilitar la

negociación directa y segura entre compradores y vendedores sin necesidad de salir del entorno web. La estructura de la interfaz adopta un diseño de tres paneles o columnas adaptativas: el panel izquierdo muestra el listado general de chats activos ordenados por interlocutor y rol; el panel superior central e informativo fija los datos del producto objeto de la consulta (con un enlace directo de acceso rápido a los detalles del vehículo y los datos de contacto del vendedor); y el cuerpo central despliega el historial cronológico de la conversación con marcas de estado de entrega de mensajes. La interfaz incluye soporte para la visualización integrada de archivos multimedia e imágenes adjuntas dentro de las burbujas de chat, rematando en la parte inferior con una barra de escritura flotante equipada con selectores de emojis, archivos y botón de envío, garantizando una experiencia de comunicación fluida y centralizada.



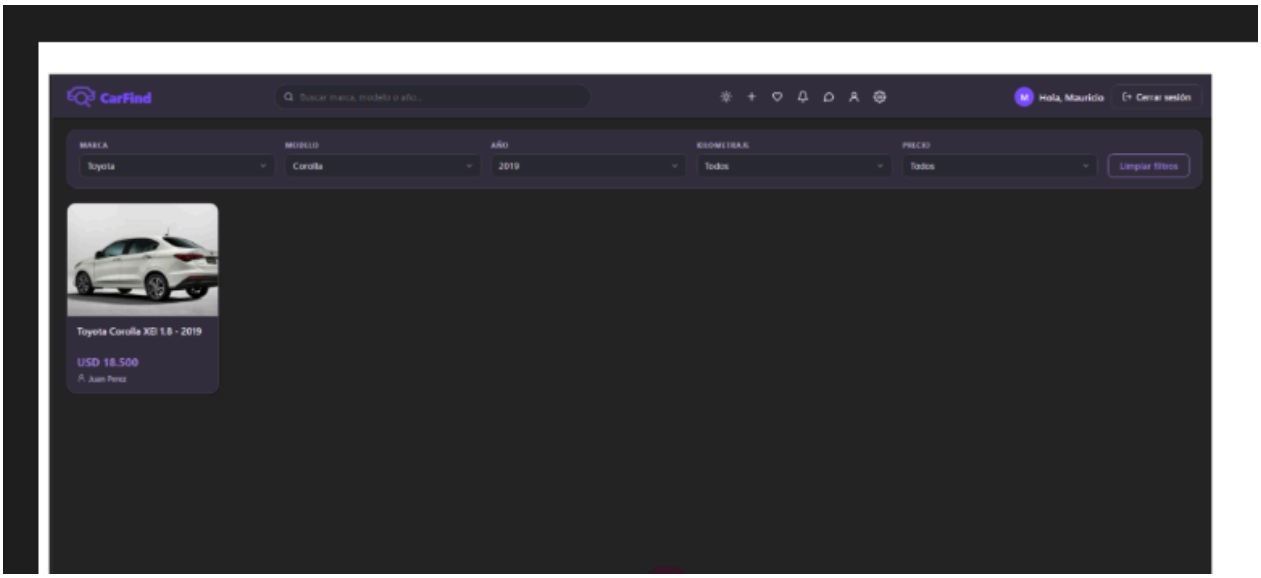
Pantalla de Registro e Inicio de Sesión de Nuevos Usuarios: Esta interfaz corresponde al módulo de autenticación y alta de cuentas de la plataforma, diseñada bajo un enfoque minimalista y centrado en la usabilidad. La pantalla presenta un contenedor central con el encabezado institucional *"TU VIAJE EMPIEZA ACÁ / CON NOSOTROS"*, seguido de un formulario estructurado en campos de entrada vertical para la recolección de datos personales y credenciales: número de DNI, correo electrónico, nombre y apellido, número telefónico y contraseña con un requisito mínimo de seguridad (8 caracteres). En la zona inferior, cuenta con un botón principal

de acción violeta titulado *"Completar Registro"*, una cláusula de aceptación vinculada a los términos y condiciones de uso del servicio, y una alternativa de navegación rápida mediante el botón *"Ingresar como invitado"* para aquellos usuarios que deseen explorar el catálogo sin registrarse formalmente.

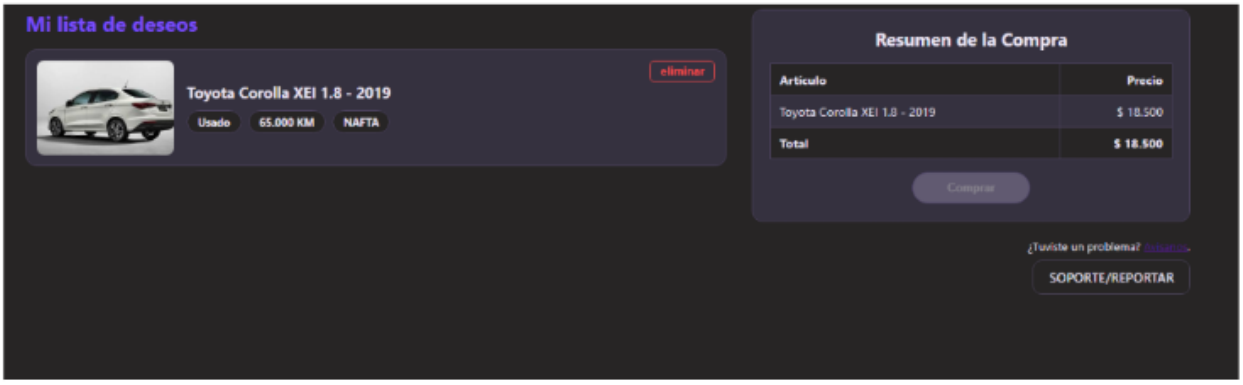


Pantalla de Perfil de Usuario y Panel de Control de Publicaciones: Esta interfaz muestra la vista del panel personal del usuario dentro de la plataforma (en este caso, autenticado como Mauricio Arzabe). El diseño se divide estructuralmente en dos secciones: un bloque lateral izquierdo que funciona como tarjeta de presentación y control de cuenta, el cual incluye un avatar de usuario, su nombre completo, datos de reputación, fecha de registro (*miembro desde*), número telefónico verificado, contador de publicaciones activas y un panel de *"Configuración"* con botones interactivos para *"Editar perfil"* y *"Ajustes de la cuenta"*. El área central y principal de la pantalla está destinada a la sección *"Mis Publicaciones"*, donde el sistema gestiona y lista el

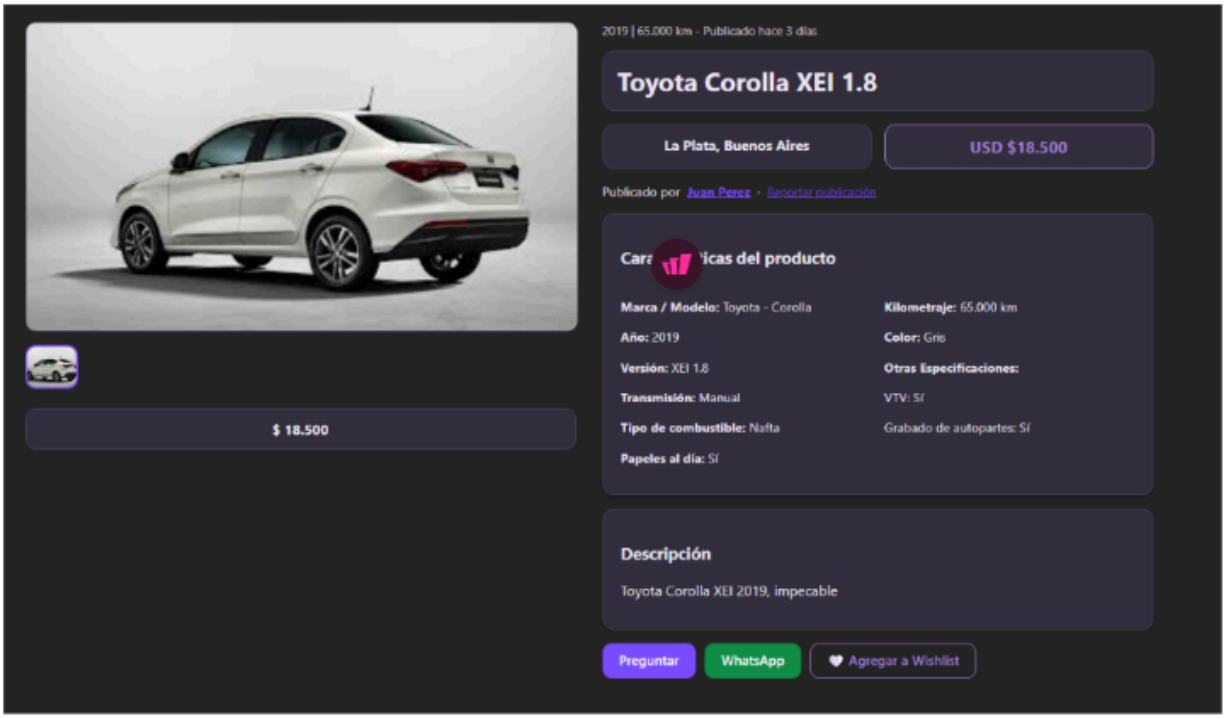
inventario de vehículos ofrecidos por el usuario (mostrando en este estado inicial la leyenda de control de que todavía no posee publicaciones registradas), complementado en la esquina superior derecha con accesos directos al perfil, botón de cierre de sesión y la central de soporte técnico o reporte de anomalías.



Pantalla de Catálogo con Sistema de Filtros Avanzados: Esta vista representa el motor de búsqueda y filtrado de la plataforma e-commerce, diseñada para permitir a los compradores acotar los resultados de forma precisa según sus preferencias de interés. En la parte superior cuenta con una barra de búsqueda global y una botonera de accesos directos del sistema. Justo debajo del encabezado se despliega una barra horizontal interactiva de filtros estructurados por categorías clave: **Marca** (con selección para *Toyota*), **Modelo** (configurado en *Corolla*), **Año** (filtrado en *2019*), **Kilometraje** y **Precio** (ambos configurados inicialmente en el parámetro general *"Todos"*), acompañada de un botón de restablecimiento rápido denominado *"Limpiar filtros"*. En el área de resultados inferior se muestra la tarjeta de producto correspondiente al vehículo que cumple con dichos criterios (un *Toyota Corolla XEI 1.8 - 2019* comercializado en USD 18.500 por el usuario Juan Perez), facilitando una experiencia de navegación ágil y directa.



Pantalla de Lista de Deseos y Carrito / Resumen de Compra: Esta interfaz gestiona el módulo de favoritos (*Mi lista de deseos*) y el flujo preliminar de transacción o cierre de compra del usuario. El diseño se organiza en dos bloques funcionales: a la izquierda, el listado de artículos guardados donde se despliega una tarjeta detallada del vehículo seleccionado (*Toyota Corolla XEI 1.8 - 2019*) con etiquetas indicativas de su condición (*Usado*, kilometraje y tipo de combustible *NAFTA*), acompañada de un botón de acción en rojo rotulado como *"eliminar"*. A la derecha, se ubica el panel de **"Resumen de la Compra"**, el cual detalla de forma tabular el artículo seleccionado, su respectivo precio unitario en dólares, el cálculo total de la transacción y un botón de confirmación de pago o adquisición (*"Comprar"*). La interfaz se completa en su sección inferior con los enlaces de asistencia y el botón de reporte técnico general.



Pantalla de Detalle de Publicación e Interacción Comercial (Vista de Comprador): Esta interfaz muestra la vista ampliada de un vehículo en venta dentro del marketplace, optimizada para ofrecer toda la información técnica y de confianza necesaria antes de concretar una compra. En la sección superior se indica la antigüedad de la publicación y los datos del rodado (*Toyota Corolla XEI 1.8*), con la ubicación geográfica (*La Plata, Buenos Aires*) y el precio en moneda extranjera (*USD $18.500*). El diseño central se estructura en dos columnas: a la izquierda, un visor de imágenes del vehículo con miniaturas de navegación y el precio destacado; a la derecha, una ficha técnica detallada (*Características del producto*) que especifica marca, modelo, año, versión, transmisión, tipo de combustible, estado de los papeles, kilometraje, color, especificaciones adicionales, vigencia de la VTV y grabado de autopartes. Debajo de la ficha técnica se ubica el cuadro de descripción redactado por el vendedor y una barra de botones de

interacción directa que incluye opciones para *"Preguntar"*, chat vía *"WhatsApp"* y la función de agregar el ítem a la lista de deseos (*"Agregar a Wishlist"*).



Pantalla del Módulo de Reportes y Moderación de Contenido: Esta interfaz interactiva corresponde al sistema de seguridad y control de calidad de la plataforma, diseñada para permitir a los usuarios reportar publicaciones dudosas o fraudulentas. La estructura de la página presenta un título destacado (*"REPORTAR PUBLICACIÓN"*) acompañado de un mensaje guía de

asistencia comunitaria. En la sección superior se muestra una vista previa resumida de la publicación objeto de la denuncia (con imagen, título, estado de disponibilidad y contadores de interacción). Debajo, el sistema provee un panel de selección de motivos mediante botones de opción estructurados en cuadrícula, cubriendo causales comunes como: *"Sospecho de fraude o estafa"*, *"Precio engañoso"*, *"Publicación duplicada"*, *"La información no coincide con el auto"*, *"Contenido inapropiado"* u *"Otro motivo"*. Adicionalmente, cuenta con un área de texto libre para detallar el problema, sugerencias o dudas (con un contador de caracteres de 0 a 500), y los botones de acción para *"Cancelar"* o enviar formalmente el reporte (*"Reportar"*).

Mi aporte en esta etapa final del proyecto: Capa DTOs y Lógica de Backend

Dentro del equipo de desarrollo, me responsabilicé de implementar y estructurar la Capa de DTOs (Data Transfer Objects) para los módulos correspondientes y de desarrollar funciones clave de la lógica del backend en Spring Boot.

En lo relativo a la capa de transferencia de datos, me encargué de diseñar los pares de objetos [Entidad]RequestDTO y [Entidad]ResponseDTO para desacoplar el modelo de datos de la base de datos de la API pública. Mi trabajo consistió en:

- Entrada de datos (RequestDTO): Definir la estructura de las peticiones que ingresan al sistema, aplicando validaciones mediante *Bean Validation* (como @NotBlank, @NotNull y @Size) para garantizar que la información sea correcta antes de ser procesada por los servicios.

- Salida de datos (ResponseDTO): Modelar las respuestas de la API para devolver únicamente la información requerida por el cliente, protegiendo datos sensibles del sistema y evitando problemas de ciclos de serialización JSON.

Adicionalmente, participé en la implementación de funcionalidades del backend, trabajando en las capas de servicio (Service) y controladores (Controller). Me encargué de escribir la lógica de negocio para gestionar operaciones del sistema (como creación, actualización, búsquedas filtradas y validaciones internas) y mapear los endpoints REST necesarios para conectar correctamente el flujo de datos entre el cliente, las reglas de negocio y los repositorios de persistencia JPA.

Esta responsabilidad me permitió no solo afianzar el uso de Java y Spring Boot en un entorno real, sino también asegurar que los contratos de la API sean limpios, seguros y mantenibles para el resto del equipo.

Análisis técnico del desarrollo y trabajo colaborativo

Analizando en detalle el impacto técnico de este proyecto, la implementación de un modelo de datos extendido a 56 entidades representó un hito clave en mi formación. Trabajar con patrones de diseño como DTO (Data Transfer Objects) para desacoplar la capa de persistencia de la capa de presentación no solo mejoró la calidad del código, sino que me obligó a pensar la aplicación en términos de mantenibilidad, seguridad y escalabilidad a largo plazo.

Asimismo, la resolución de incidencias en el control de versiones a través de Git y GitHub Desktop fue una de las instancias más formativas de todo el proceso. Enfrentar la corrupción de estados locales, gestionar archivos de compilación no deseados como target o bin, y reestructurar

adecuadamente el archivo .gitignore me permitió comprender que la administración de entornos de desarrollo y la configuración de repositorios son disciplinas tan cruciales como la propia escritura de código Java.

Por otro lado, la experiencia puso en evidencia el valor del trabajo en equipo dentro de un entorno técnico. Intercambiar soluciones sobre errores de compilación, validar scripts de inserción masiva en MySQL y coordinar ramas de trabajo me enseñó que el desarrollo de software no es una tarea aislada. La comunicación clara, la documentación de cambios y la capacidad de diagnosticar fallos de manera metódica son competencias transversales que fortalecieron mi rol dentro del grupo de trabajo y que resultan indispensables para cualquier proyecto tecnológico futuro.

Reflexión personal y próximos pasos: Desarrollo Backend y Arquitectura de Sistemas

En lo personal, desarrollar este backend completo de 56 entidades y construir una arquitectura limpia basada en controladores, servicios, repositorios y DTOs ha sido una experiencia sumamente desafiante y valiosa, aunque en varios momentos el camino técnico haya estado lleno de obstáculos complejos. Enfrentarme a la integración de bases de datos relacionales, configurar entornos locales con XAMPP y phpMyAdmin, y lidiar con la depuración de errores a bajo nivel me permitió tener una idea mucho más concreta de todo el esfuerzo que implica diseñar software

robusto, la cantidad de componentes que interactúan por debajo y lo amplio que puede llegar a ser el desarrollo backend.

Este proceso también me hizo reflexionar profundamente sobre mi propio aprendizaje en la secundaria técnica y en la programación. A veces, cuando nos sumergimos en un proyecto ambicioso o conocemos tecnologías atractivas como Spring Boot, queremos avanzar directamente hacia las capas más complejas, implementar soluciones avanzadas o estructurar decenas de modelos de golpe antes de tiempo. Sin embargo, los tropiezos técnicos a los que me enfrenté —especialmente los problemas de control de versiones con Git y la correcta gestión de dependencias y compilación— me recordaron que no tiene sentido intentar construir sistemas avanzados sobre bases que todavía no están completamente consolidadas.

Esto coincide directamente con lo que vengo notando en mi formación en administración de sistemas, redes y desarrollo: cuando una base conceptual o estructural falla, los problemas de sincronización, arquitectura o configuración aparecen inevitablemente más adelante. Por eso, me parece fundamental no saltear etapas, comprender a fondo los fundamentos de la programación orientada a objetos en Java, el manejo de sockets, el diseño de redes y el funcionamiento de las bases de datos antes de apresurarme a abarcar más de lo necesario.

Desarrollar este proyecto también me ayudó a descubrir que el mundo IT ofrece múltiples puertas de entrada y áreas complementarias muy interesantes. Trabajar tanto en la lógica de backend como en la estructuración de redes y servidores me permite ampliar mi perspectiva y entender que el perfil profesional en informática es muy versátil. Me interesa seguir explorando y conectando áreas como el desarrollo de software, la administración de sistemas y redes, el

soporte y la gestión de bases de datos, comprendiendo qué requiere cada rol y cómo se complementan en el día a día de un entorno tecnológico real.

Como próximos pasos, tengo pensado reforzar aún más mis conocimientos en Java backend y persistencia de datos, optimizar la estructuración de proyectos grandes aplicando buenas prácticas desde el inicio, y seguir adelante con mis prácticas y asignaturas técnicas de la secundaria. También considero importante documentar mejor mis propios procesos de desarrollo y resolución de errores para enfrentar futuros proyectos con mayor orden y criterio técnico.

Finalmente, al dar por concluidas las etapas críticas de este desarrollo, me voy con nuevos interrogantes técnicos por resolver, pero también con una visión mucho más clara de lo que implica construir una aplicación backend robusta y de los conocimientos sólidos que necesito seguir afianzando.

Mi principal aprendizaje de este proyecto fue que, antes de escalar hacia arquitecturas masivas o sistemas complejos, necesito dominar los fundamentos de la programación y la gestión de sistemas. Una base sólida es lo que permite comprender problemas difíciles, solucionar fallos de entorno con autonomía y tomar decisiones de diseño con criterio técnico. Pero, además, me llevo algo igual de valioso: la certeza de que el desarrollo tecnológico tiene muchos caminos posibles y que todavía tengo todo el recorrido de la escuela técnica para seguir explorándolos, construyendo y descubriendo hacia dónde quiero orientar mi futuro profesional.